

MotorX

Codebase Guide

Architecture, files, functions and automated tests

Part 1: Codebase walkthrough

Part 2: Complete file and function index

Combined from CODEBASE_WALKTHROUGH.md and CODEBASE_FILE_FUNCTION_INDEX.md. Source snapshot: 28 September 2026. Use PDF bookmarks to navigate sections and files.

Source file paths refer to D:/MotorX/MotorX. Paths are retained as readable references; the PDF does not embed the source files. Architecture arrows are reproduced as a diagram.

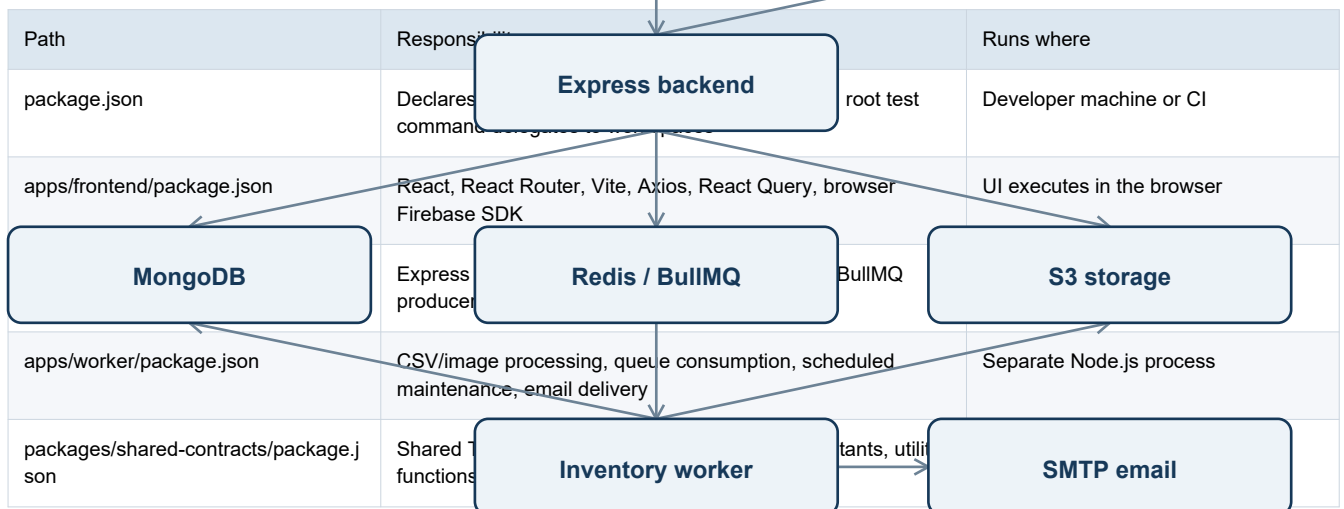
MotorX: codebase walkthrough, from startup to tests

This explains the local working tree at `D:\MotorX\MotorX`, inspected on 2026-09-28. All paths below are relative to that repository root. It describes code and configured automation, not a verification that a deployed site or GitHub workflow is currently healthy. The preceding task's uncommitted admin collection/account changes are included; their verification was interrupted.

Read this document for the architecture and important execution paths. Use [CODEBASE_FILE_FUNCTION_INDEX.md](#) for every indexed source file, named function/component, parameters, leading explanation comments, direct named calls, route registrations, and declared test cases. That companion covers 314 TypeScript/TSX/MJS files and 819 named functions/methods/components, including local helpers. Anonymous event/effect callbacks remain inside their containing functions. Generated dependencies, build outputs, binary assets, and private environment values are not reproduced.

1. What runs in this project

MotorX is an npm-workspace monorepo: several applications share one repository and one set of common contracts.



MongoDB stores application records. Redis supports queueing and rate limits. S3-compatible storage holds files. Firebase handles identity/password authentication. These are separate responsibilities; the frontend does not directly open the application's MongoDB connection.

2. Root files and infrastructure

File/directory	What to read it for
README.md, DEVELOPER_SETUP.md	Existing setup, operation, and development instructions
package-lock.json	Exact dependency resolution used by <code>npm ci</code> ; generated package metadata, not application logic
.nvmrc and workspace package.json files	Project Node version and runnable scripts; the root engine requires Node 24
.env.example, apps/backend/.env.example	Names/examples of configuration inputs; actual .env values stay private
compose.yml	Main service wiring: backend, worker, frontend, Redis, MinIO, initialization, dependencies, and health checks
compose.dev.yml	Development commands and source mounts
compose.watch.yml	Compose file watching/synchronization and development commands
compose.test.yml	Isolated test project, disposable Mongo replica set, test commands and temporary storage
compose.ec2.yml	EC2-specific Compose overrides
infrastructure/docker/backend.Dockerfile	Backend build/runtime stages
infrastructure/docker/worker.Dockerfile	Worker build/runtime stages
infrastructure/docker/frontend.Dockerfile, frontend.production.Dockerfile	Frontend development versus production image
infrastructure/nginx/nginx.conf	Production frontend web-server configuration
infrastructure/aws/github-actions-deploy-role.yml	AWS role configuration for deployment automation
.github/workflows/ci.yml	Build/test/security automation
.github/workflows/deploy.yml	Production release workflow and deployment conditions
.github/dependabot.yml	Dependency-update automation configuration
.gitignore, .dockerignore	Files excluded from Git or Docker build contexts
.gitleaks.toml, .gitleaksignore	Secret-scanner rules and recorded exclusions
scripts/smoke/live-stack-smoke.py	Script for checking a running stack
scripts/drills/kill-worker-mid-import.sh	Recovery drill for worker interruption during import
docs/, team-work-plan.md, root requirements/design PDFs	Documentation, reports, evidence and planning; not server request handlers

The IDE's `.npm-cache/_update-notifier-last-checked` is npm cache housekeeping. It is not MotorX business logic, database configuration, or a test scheduler.

3. How the backend connects to MongoDB

Read these in order:

1. `apps/backend/src/config/env.ts`: imports `dotenv/config`, defines `envSchema`, and executes `envSchema.parse(process.env)`. That validates configuration and exports `env`. `MONGODB_URI` is one of those inputs. Production-specific checks also validate the URI's suitability.
2. `apps/backend/src/config/database.ts`: `connectDatabase()` calls Mongoose using that URI.

3. `apps/backend/src/server.ts`: awaits `connectDatabase()` before calling `app.listen(...)`.

4. Model files, such as `listing.model.ts`, register schemas on Mongoose. Repository functions use those models through the established connection.

The actual connection function is:

```
export async function connectDatabase(): Promise<void> {
  await mongoose.connect(env.MONGODB_URI, {
    serverSelectionTimeoutMS: 10_000,
  });
}
```

`mongoose.connect` is asynchronous, so `await` waits for a successful connection. `serverSelectionTimeoutMS` limits how long server selection can wait. Because the startup connection is awaited before `app.listen`, failed database initialization prevents the API from starting to accept HTTP requests.

`disconnectDatabase()` closes Mongoose. `shutdown(signal)` in `server.ts` stops accepting requests, allows in-flight requests time to finish, and closes MongoDB, the queue, and Redis. It has a deadline so shutdown cannot wait indefinitely.

The worker is a separate process, so it opens its own connection. See `apps/worker/src/config/database.ts`: `connectWorkerDatabase()` and `disconnectWorkerDatabase()`. `startWorker()` in `worker.ts` awaits its connection before consuming jobs.

A database model is different from the connection. In `listing.model.ts`, `listingSchema` defines fields such as `dealerId`, `title`, `price`, `status`, `attributes`, `sourceUploadJobId`, and `sourceRowNumber`. Index declarations support lookup and uniqueness rules. `ListingModel` provides methods such as `find`, `create`, and `findOneAndUpdate`. Opening the connection belongs to `database.ts`; describing documents belongs to the model.

For example, the compound import-row index makes an upload/row pair unique, supporting safe import retries. The partial registration-number index prevents two draft/active listings from claiming the same normalized registration number.

4. Backend startup and request layers

`apps/backend/src/app.ts` constructs the Express application and registers middleware and routes. It is separate from `server.ts` so tests can import the app without running the production startup/listen code.

Its setup includes Helmet headers, request logging, CORS, JSON parsing, proxy trust configuration, API rate limiting, health endpoints, feature routers, and the error handler.

Most backend features follow this pattern:

```
HTTP request
-> *.routes.ts      URL + HTTP method + ordered middleware
-> *.validation.ts  input rules used by validateRequest
-> *.controller.ts  extract request values and send HTTP response
-> *.service.ts     business rules and multi-step operations
-> *.repository.ts  database reads/writes
-> *.model.ts       document schema and indexes
-> MongoDB
```

These are conventions, not absolute restrictions: the auth router has inline handlers, and some services directly use models.

Shared file	Important function / role
<code>shared/middleware/validateRequest.ts</code>	<code>validateRequest({ body, params, query })</code> returns middleware that parses chosen request sections with Zod and forwards HTTP 400 errors for invalid input
<code>shared/utls/asyncHandler.ts</code>	<code>asyncHandler</code> forwards asynchronous handler failures to Express error handling
<code>shared/responses/apiResponse.ts</code>	<code>sendSuccess</code> builds the standard success response
<code>shared/errors/AppError.ts</code>	Application error class carrying HTTP status, error code and optional field errors

Shared file	Important function / role
shared/errors/errorCodes.ts	Common machine-readable error identifiers
shared/middleware/errorHandler.ts	errorHandler turns known errors into the API error envelope, logs unexpected errors, and avoids exposing their internal messages
shared/utls/pagination.ts	buildPaginationMeta(page, limit, total) calculates total pages
shared/middleware/rateLimits.ts	API/upload budgets, daily quotas and concurrent-upload controls

Mounted paths come from app.ts:

Prefix	Router purpose
/api/v1/auth	Current account/profile
/api/v1/listings	Dealer listing operations and public buyer listing operations
/api/v1/search	Search
/api/v1/listing-images	Listing image responses
/api/v1/dealers	Dealer application/profile
/api/v1/dealer/uploads	Dealer CSV/image upload operations
/api/v1/admin	Administrator operations
/api/v1/notifications	Account notifications

The dealer listing router mounts before the buyer router so /mine is not mistaken for a public /:listingId route.

5. Login, identity, account status and roles

The browser-side pieces are features/auth/services/firebaseAuth.ts, authApi.ts, and context/AuthProvider.tsx.

AuthProvider exposes login, registerBuyer, registerDealerApplication, logout, and refreshUser. loadUserWithDealerStatus() obtains the backend account and, for a buyer, attempts to load any dealer application. switchAccount() clears React Query data when the account changes so another person does not inherit the previous account's cached data.

The frontend shared/services/apiClient.ts creates the Axios client. Its request interceptor obtains getIdToken() from the current Firebase user and sends Authorization: Bearer For FormData, it removes the JSON content-type header so the browser can provide the multipart boundary. Its response interceptor turns API errors into useful frontend error messages.

For protected backend routes, follow this sequence:

Function and file	Check performed
verifyFirebaseToken() — verifyFirebaseToken.ts	Parses bearer token and verifies Firebase identity. verifyWithCache() uses a bounded cache keyed by a token hash; validity is bounded by TTL and token expiry
loadLocalUser() — loadLocalUser.ts	Uses the verified identity to find/create the MotorX account
findOrCreateAuthUser() — authUser.repository.ts	Looks up Firebase UID, updates selected profile/login fields, defaults new users to active buyers
requireAuthenticated() — requireAuthenticated.ts	Requires both Firebase identity and local account; rejects inactive/suspended accounts
requireRole(...roles) — requireRole.ts	Returns middleware allowing only the listed local MotorX roles

For example, `requireRole('admin')` creates a function that Express invokes for each request. `next()` allows the following middleware/handler to run. `next(new AppError(...))` forwards the failure to the error handler.

The role comes from the local database account, not from a role submitted by the browser. Firebase authentication alone does not make someone an administrator.

RoleGuard.tsx handles frontend page access and redirects. Backend middleware independently protects the API even if someone bypasses the UI.

6. A complete example: dealer creates a listing

1. ListingForm.tsx collects vehicle fields and handles submit/edit behavior.
2. listingApi.ts sends the HTTP request through apiClient.
3. listing.routes.ts maps `POST /api/v1/listings` to the dealer-only middleware chain, `createListingBodySchema`, then `createListing`.
4. listing.controller.ts: `createListing(request, response)` uses `request.localUser._id` as the owner, calls `createDealerListing`, and responds with HTTP 201.
5. listing.service.ts: `createDealerListing` checks capacity, normalizes the registration number, checks for an active duplicate, attempts embedding generation, adds trusted ownership/timestamps, and calls `createListingRecord`.
6. listing.repository.ts: `createListingRecord` persists through the Mongoose model.
7. `serializeListing()` returns the public DTO shape: string IDs, formatted dates, ordered images, vehicle attributes, and other intended fields.

Related service functions:

Function	Responsibility
<code>assertDealerListingCapacity</code>	Enforce the draft/active listing limit
<code>assertRegistrationNotActivelyListed</code>	Early duplicate-registration check
<code>rethrowAsRegistrationConflict</code>	Turn database uniqueness failures into a consistent conflict error
<code>getDealerListings</code>	Paginated dealer inventory with optional filters
<code>getDealerListing</code>	Retrieve one owned listing
<code>getDealerListingStats</code>	Inventory/status/stale counts
<code>updateDealerListing</code>	Change editable fields
<code>changeDealerListingStatus</code>	Apply allowed status changes
<code>applyBulkListingAction</code>	Publish, mark sold, archive, confirm availability, reduce price, or delete within the dealer's selected scope
<code>deleteDealerListing</code>	Delete an owned listing and handle related cleanup

Listing lifecycle statuses are draft, active, sold, and archived. These differ from CSV job statuses.

7. Backend feature map

For every module in this table, the companion index breaks down each file and function.

Directory	Operations and files to start with
<code>modules/auth-users</code>	<code>authUser.routes.ts</code> : GET/PATCH /me; repository synchronizes identity; model stores account role/status/profile

Directory	Operations and files to start with
modules/dealers	dealer.service.ts: submitDealerApplication, getMyDealerApplication, updateMyDealerProfile, serializeReviewState; document middleware/content/storage files validate and store verification files
modules/marketplace	Dealer listing CRUD/status/bulk actions; listingImage.* handles photos and thumbnails; publicVisibility.ts filters public access to hidden dealers
modules/buyers	browseListingsAsBuyer, viewListingAsBuyer, similarListingsForBuyer, recommendedListingsForBuyer; buyer.similarity.ts scores recommendations
modules/inventory	Accept uploads, store files, create durable jobs, enqueue processing, report progress/rejections, retry failed processing
modules/search	Analyze text, apply structured filters, produce embeddings, combine lexical/vector search candidates
modules/notifications	Store and return account notifications; service helpers create notifications for business events
modules/admin	Users, dealer reviews, listing moderation, bulk job monitoring, audit history and system health

viewListingAsBuyer() only exposes the dealer's public business profile alongside a public listing. Full account information in the new admin detail endpoint is a separate admin-only view.

Administrator operations include getUsersForAdmin, changeUserStatusAsAdmin, getListingsForAdmin, removeListingAsAdmin, getUploadsForAdmin, getAuditLogsForAdmin, and reviewDealerApplicationAsAdmin. Changes such as dealer approval and account suspension use transactions to keep related database writes/audit records consistent. Dealer approval also checks the applicant's current Firebase email-verification state.

getDealerDocumentForAdmin() checks the application/document, observes document retention state, and records access. Its controller streams the protected file with restrictive caching/content headers.

The newly added collection/account path is:

```
UploadMonitoring -> UploadCollection -> adminApi.listUploadRecords
-> GET /api/v1/admin/uploads/:uploadId/records
-> getAdminUploadRecords -> getUploadRecordsForAdmin
-> listAdminUploadRecords -> ListingModel or RejectedRecordModel

UserManagement / ListingMonitoring / UploadCollection -> AccountDetails
-> adminApi.getUserDetails -> GET /api/v1/admin/users/:userId
-> getAdminUserDetails -> getUserDetailsForAdmin
-> findAdminUser + findAdminDealer
```

8. Bulk CSV upload, step by step

The HTTP request accepts the file and returns a job. The separate worker does the lengthy processing.

Stage	File and functions
Choose file/category	InventoryUpload.tsx
Send multipart file	inventoryApi.ts: uploadCsv(category, file, onProgress)
Authenticate/limit/parse/validate	inventory.routes.ts, inventory.middleware.ts, inventory.validation.ts; Multer runs before body validation
Handle HTTP upload	inventory.controller.ts: uploadDealerInventory
Accept durably	inventory.service.ts: createInventoryUpload, enqueueOrDefer

Stage	File and functions
Store source file	inventory.storage.ts: storeInventoryCsv
Record job	inventory.repository.ts: createUploadJob; uploadJob.model.ts defines fields
Queue work	inventory.queue.ts: enqueueInventoryUpload
Consume	worker.ts: startWorker, processInventoryJob
Dispatch CSV job	inventoryUpload.job.ts: processInventoryUploadJob
Process	uploadJob.service.ts: extractInventoryUpload, downloadInventoryStream, processInventoryBatch
Display results	UploadDetails.tsx; admin UploadMonitoring/UploadCollection

If the source file was stored but creating the durable Mongo job fails, the service attempts file cleanup. After the Mongo job exists, a queue outage does not discard that accepted upload: `enqueueOrDefer` allows reconciliation to queue it later.

The processing functions divide the work further:

File	Function / purpose
pipeline/extract.ts	extractCsvBatches: stream CSV in bounded batches
pipeline/transform.ts	prepareInventoryBatch: preserve source row numbers/data while separating valid and invalid rows
pipeline/normalize.ts	normalizeInventoryRow and helpers such as normalizeNumber, normalizeEnumValue, titleCase: standardize CSV values
pipeline/validate.ts	validateInventoryRow: validate normalized category-specific vehicle data
pipeline/detectDuplicates.ts	createListingDuplicateKey, detectExactDuplicates: identify duplicates
pipeline/persist.ts	persistValidRows: add trusted dealer/job/row references and save draft listings; persistRejectedRows: save correction details
pipeline/generateEmbedding.ts	generateEmbedding: configured hosted embedding generation or local fallback when no hosted key exists
repositories/listing.repository.ts	Listing persistence and imported-row lookup
repositories/rejectedRecord.repository.ts	Rejected-row persistence
repositories/uploadJob.repository.ts	Claim/renew lease, progress checkpoints, completion/failure/retry state

An imported row initially creates a **draft** listing. A completed import does not mean the vehicle is sold or publicly active. Rejected rows contain original values and error reasons. Completed job states are `completed` or `completedWithErrors`; pending, processing and failed jobs have separate meanings.

`extractInventoryUpload()` claims the job, holds a lease, streams batches, checkpoints progress and updates the final outcome. Imported-row references prevent retries from blindly recreating earlier rows. `jobLease.ts` detects loss of ownership; `activeLeases.ts` tracks work this process holds; `transientError.ts` distinguishes retryable failures. `runLeaseReaper()` recovers stuck or unqueued work.

9. Worker jobs beyond CSV

File	Operation
jobs/inventoryImages.job.ts	processInventoryImagesJob dispatches photo ZIP work
services/imageProcessing.service.ts	Image archive processing and matching photos to imported vehicles
services/imageReencode.ts	Image conversion helpers
jobs/reaper.job.ts	runLeaseReaper: job recovery/reconciliation and worker heartbeat
jobs/documentRetention.job.ts	runDocumentRetention: remove verification files after the configured retention period
jobs/staleListingReminder.job.ts	runStaleListingReminders: remind dealers about stale inventory
jobs/emailOutbox.job.ts	runEmailOutbox: deliver queued emails
services/notification.service.ts	Produce upload and related notification outcomes
services/emailTemplate.ts	Email rendering helpers
config/mailler.ts	Mail transport configuration
health.ts	startWorkerHealthServer: worker health endpoint

`startWorker()` starts maintenance timers as well as the BullMQ consumer. Its shutdown function clears those timers, drains work and releases remaining owned leases before dependency cleanup. These recurring jobs run business operations; they are not automated tests.

10. Search and recommendations

`search.queryAnalyzer.ts` interprets query text into intent, filters and corrections. `search.service.ts` contains `searchListings(query)`:

- Structured/non-relevance searches use `listStructuredListings`.
- Relevant free text can combine lexical candidates and vector candidates.
- Embedding/vector failures fall back to lexical results.
- Results are ranked and paginated, and include metadata describing search mode and interpretation.

`search.repository.ts` supplies `buildListingFilter`, `buildListingSort`, `listStructuredListings`, `listSearchCandidates`, and `listVectorCandidates`. Public queries select active listings and apply dealer visibility filtering. Vector queries use the configured Atlas vector index; ordinary MongoDB connection success alone does not establish that vector search is configured.

`search.embedding.ts` and `packages/shared-contracts/src/search/embedding.ts` define embedding generation/shared representation helpers. `backfill-search-embeddings.mjs` updates older data through an explicit maintenance script.

Buyer recommendations are separate. `similarListingsForBuyer()` scores candidate vehicles against one seed; `recommendedListingsForBuyer()` uses recently viewed IDs sent by the browser. The related UI files are `VehicleSuggestions.tsx` and `recentlyViewed.ts`.

11. Frontend startup and page organization

1. `index.html` provides the root element.
2. `src/main.tsx` imports global CSS, finds the root, and renders `<App />` under `React.StrictMode`.
3. `app/App.tsx` mounts `AppProviders`, `BrowserRouter`, lazy-loaded pages, `Suspense`, routes and role guards.
4. `app/providers.tsx`: `AppProviders` wraps theme, translations, query caching, auth, and vehicle comparison state.

Directory/files	Responsibility
portals/buyer/pages/Marketplace.tsx, VehicleDetails.tsx, ComparePage.tsx	Browse/search, detail/contact view, comparison
portals/dealer/pages/DealerDashboard.tsx, ListingManager.tsx, ListingForm.tsx	Dealer summary, inventory management, listing create/edit
portals/dealer/pages/InventoryUpload.tsx, UploadDetails.tsx, DealerProfile.tsx	CSV/ZIP submission, processing results, dealer profile
portals/admin/pages/AdminDashboard.tsx, UserManagement.tsx, DealerApprovals.tsx	Admin summary, account moderation, application review
portals/admin/pages/ListingMonitoring.tsx, UploadMonitoring.tsx, AuditLogs.tsx	Cross-dealer listings, upload jobs, audit history
portals/admin/pages/AccountDetails.tsx, UploadCollection.tsx	Newly added account and collection drill-downs
portals/*/layout/*Layout.tsx	Portal navigation and surrounding page layout
features/auth/pages/ and components/	Login, registration, pending application, guards and verification notices
features/*/services/*Api.ts	Feature-specific API calls
features/buyers/hooks/useBuyerListings.ts, useBuyerListing.ts	Buyer data-fetching hooks
features/listings/components/	Cards, galleries, photos, cropping, status and price-reduction UI
features/compare/	Comparison state, toggles and tray
features/notifications/	Notification API and UI
shared/components/	Responsive tables, pagination controls, shared portal layout
shared/services/queryClient.ts	Shared React Query client
shared/utils/	Formatting, phone helpers and safe browser-storage access
shared/i18n/	Translation provider/hook/switcher and English, Sinhala, Tamil messages
app/theme/	Theme state and toggle
index.css	Global visual styles
vite.config.ts, tsconfig.json	Build/dev tool and TypeScript configuration

A typical React page has state (useState), asynchronous loading (useEffect or a query hook), event handlers, and JSX describing the visible UI. A handler such as toggleStatus() updates one account; the component function renders the screen. The companion index lists component-local named handlers separately.

12. Shared contracts

File/directory	Shared responsibility
src/index.ts	Package exports
src/enums/index.ts	Shared allowed-value constants/types
src/dtos/index.ts	Data transfer shapes for API consumers
src/interfaces/index.ts	Common interfaces
src/vehicle/attributeSchemas.ts	Category-specific vehicle attribute validation
src/vehicle/listingSchemas.ts	Listing validation shared across applications
src/vehicle/csvTemplates.ts	CSV headers/examples and buildCsvTemplateContent
src/queue/inventoryJobs.ts	Inventory queue contracts
src/media/imageConstraints.ts	Shared image constraints
src/utls/registrationNumber.ts	Registration-number normalization
src/utls/mongoUri.ts	Database URI checks
src/search/embedding.ts	Search text/vector helpers

TypeScript types help the compiler check code; Zod schemas validate actual incoming values at runtime. A value having an interface name in code does not mean an untrusted HTTP request has been validated.

13. How test files are written

MotorX uses Vitest. Tests generally sit beside the implementation and end in `.test.ts` or `.test.tsx`. The configurations discover those filename patterns; merely adding a normally matching test file makes it eligible for the appropriate workspace test run.

File	Test setup
apps/backend/vitest.config.ts	Node environment; <code>src/**/*.test.ts</code> ; 15-second test/hook timeouts
apps/worker/vitest.config.ts	Node environment; <code>src/**/*.test.ts</code> ; 15-second test/hook timeouts
apps/frontend/vitest.config.ts	jsdom browser simulation, React plugin, @ alias, setup file, <code>.test.ts/.test.tsx</code> , up to two workers
apps/frontend/src/test/setup.ts	Registers DOM matchers and cleans rendered components after each test

The usual structure is arrange, act, assert: prepare inputs/dependencies, call the behavior, then check the result.

This is an existing unit test from `pagination.test.ts`:

```
import { describe, expect, it } from 'vitest';
import { buildPaginationMeta } from './pagination.js';

describe('buildPaginationMeta', () => {
  it('calculates the final partial page', () => {
    expect(buildPaginationMeta(2, 20, 41)).toEqual({
      page: 2,
      limit: 20,
      total: 41,
      totalPages: 3,
    });
  });
});
```

```
});
```

describe groups tests. it declares one expected behavior. expect checks the actual result. This test needs no running HTTP server or database.

For frontend behavior, RoleGuard.test.tsx mocks useAuth, renders a dealer route under MemoryRouter, and checks whether a visitor sees the page, login screen, restricted message, or application-status page. renderDealerPage() is the reusable test helper. vi.hoisted creates the mock available to vi.mock; render mounts React; screen.getByText queries visible output.

LoginForm.test.tsx, ListingForm.test.tsx, and the admin/dealer page test files check UI actions with mocked external calls. userEvent simulates typing/clicking; findBy... waits for asynchronous UI output. These are simulated-browser component tests, not a real browser opening the production site.

For real database behavior, admin.repository.test.ts uses:

```
beforeAll(connectTestDb);
afterEach(clearTestDb);
afterAll(disconnectTestDb);
```

It creates dealer records, calls the repository, and checks the returned order. See src/test/db.ts:

- validateTestMongoUri verifies an allowed host/scheme and a dedicated motorx_test database name.
- getSafeTestMongoUri requires TEST_MONGODB_URI; it refuses to substitute the application database URI.
- connectTestDb opens the test connection.
- clearTestDb deletes records between tests.
- disconnectTestDb closes the connection.

Those tests deliberately delete fixture data, which is why they must target disposable test data. Tests sharing that database run with --maxWorkers=1 in CI.

accessControl.journey.test.ts and other *.journey.test.ts files use Supertest to call the Express app, real MongoDB for persistence, and mocked Firebase/storage boundaries. This exercises routing, authorization, services and repositories together. It still does not exercise a real browser or real Firebase login.

Worker tests cover CSV extraction/normalization/validation, retry/lease behavior, image processing, retention, email outbox and recovery. See the function index for every test's declared behavior. A test file's existence does not prove its latest execution passed.

14. Commands to run tests locally

Run these from D:\MotorX\MotorX. On this Windows setup, use npm.cmd and npx.cmd because PowerShell may block the .ps1 wrappers.

```
# Build the package whose dist exports are imported by the applications.
npm.cmd run build --workspace @motorx/shared-contracts

# One pure unit test, without a running application stack.
npm.cmd test --workspace @motorx/backend -- src/shared/utils/pagination.test.ts

# One frontend component test.
npm.cmd test --workspace @motorx/frontend -- src/features/auth/components/RoleGuard.test.tsx

# All frontend tests.
npm.cmd test --workspace @motorx/frontend
```

Full backend/worker runs need their test dependencies. In particular, start a dedicated test MongoDB replica set and configure its URI before database tests; the URI assignment alone does not start MongoDB:

```
$env:TEST_MONGODB_URI = 'mongodb://127.0.0.1:27017/motorx_test?replicaSet=rs0'
npm.cmd test --workspace @motorx/backend -- --maxWorkers=1
npm.cmd test --workspace @motorx/worker -- --maxWorkers=1
```

Use `compose.test.yml` for the repository's isolated Docker test infrastructure, merged with `compose.yml`. Its backend and worker commands run their respective suites serially within each service; avoid running both suites against the same test database simultaneously. The frontend has a separate test profile and must be invoked with a `test` command if using it for component tests. The workflow in `ci.yml` is also an exact reference for a fresh test environment.

Root `npm.cmd test` delegates to workspace test scripts. The `shared-contracts test` script currently prints that it has no standalone tests; some shared behavior is exercised from application tests.

The backend provides an explicit watch script:

```
npm.cmd run test:watch --workspace @motorx/backend
```

For frontend watch mode, run the configured Vitest from its workspace directory:

```
Set-Location D:\MotorX\MotorX\apps\frontend  
npx.cmd vitest
```

`test:coverage` scripts generate coverage reports. TypeScript builds check types/produce output; they do not replace behavior tests. `tsx watch` restarts backend/worker application code after changes; it is not Vitest watch mode.

15. When tests run automatically

The current automation is in `.github/workflows/ci.yml`. It is configured for pushes to `main/dewni`, pull requests targeting those branches, manual dispatch, and Monday 03:00 UTC schedules. Actual executions depend on GitHub Actions being enabled and the workflow/environment being available.

Its build/test job:

1. Checks out code and starts a disposable MongoDB 7 replica set.
2. Sets up Node 24 and installs locked dependencies with `npm ci`.
3. Runs dependency audit and Compose configuration validation.
4. Builds shared contracts, backend, worker and frontend.
5. Runs backend tests with one worker.
6. Runs worker tests with one worker.
7. Runs frontend tests.

A separate job scans for leaked secrets. Docker image jobs depend on successful build/test, then build and scan backend/worker/frontend images. A failure can stop later dependent jobs.

`deploy.yml` is configured to react to a successful CI workflow on `main` while excluding scheduled CI runs; it also supports manual dispatch. Deployment has additional environment/commit checks. The existence of the file does not confirm production secrets, approvals or infrastructure are configured.

Tests do not automatically run just because the system is serving users.

Mechanism	Trigger	What it does
<code>npm test / vitest run</code>	Explicit command or CI step	Runs tests once
Vitest watch mode	You start the watcher; relevant files change	Reruns tests during development
GitHub CI	Configured push/PR/manual/schedule event	Builds/tests/scans code
Backend <code>/health/live</code>	Health probe	Checks process liveness
Backend <code>/health/ready</code>	Health probe	Checks dependency readiness; Mongo failure gives 503, Redis outage can report degraded operation
Docker <code>healthcheck.test</code>	Container health interval	Runs a health-check command; the YAML key <code>test</code> does not mean the Vitest suite
Worker maintenance timers	Process startup/configured intervals	Reap leases, enforce retention, send reminders/emails

Runtime request validation is also not a test suite: it validates each incoming request so invalid data is rejected while the system operates.

16. Navigation details that prevent confusion

- `apps/frontend/src/app/App.tsx` mounts real frontend routes. `app/router.tsx`, `portal/*.routes.tsx`, and `config/routes.ts` also contain route metadata/helpers; a path listed there alone does not mount a page.
- `SystemHealth.tsx` exists, and the backend has `/api/v1/admin/system-health`, but the inspected `App.tsx` does not mount a separate frontend system-health page.
- `apps/backend/src/app.ts` mounts real backend routes. `src/routes.ts` is a descriptive object; for example, its `/health` string alone is not an Express registration.
- Admin persistence imports `AdminAuditLogModel` from `admin.model.ts`. The separate `audit-log.model.ts` defines a different model; do not assume it is the model used by those repository operations.
- Worker `pipeline/categorize.ts` and `pipeline/enrich.ts` exist as helpers, but the inspected import-processing path uses normalization, validation, duplicate detection and persistence; those two filenames do not prove that they execute in that path.
- Backend `shared/middleware/notFound.ts` exists, but the inspected `app.ts` does not mount it. File existence and runtime wiring are different.
- `scripts/` and `migration/backfill` files run when explicitly invoked; they do not automatically execute on every request. Inspect their entry points before using them.

17. A practical reading order

Start with `backend/server.ts`, `config/env.ts`, `config/database.ts`, and `app.ts`. Then trace one feature through its route, controller, service, repository and model. Read the corresponding frontend API adapter and page next. Follow the bulk upload into the worker after that. Finally, read the feature's unit/component/integration tests and `ci.yml` so you know both what the code does and how that behavior is checked.

For a particular function, search its name in [the full file/function index](#), open the linked file, and use its listed line number. Direct calls help you choose the next implementation to inspect; they are a source-navigation aid rather than a complete runtime call graph.

Checks performed while preparing this guide

All local links in both documents resolved. The backend `pagination.test.ts` example passed 2 tests, and the frontend `RoleGuard.test.tsx` example passed 4 tests. The complete application suites, production deployment, and preceding admin feature changes were not verified by these two example runs.

MotorX file and function index

Start with [the walkthrough](#). This index covers 314 TypeScript/TSX/MJS source and configuration files under apps and packages, with 819 named functions, components, and methods, including local named helpers. Generated dependencies, build outputs, public binary assets, and private environment files are excluded.

Paths are relative to D:/MotorX/MotorX. Line numbers describe this working-tree snapshot and may change. Named callbacks assigned to variables are included; anonymous effect/event callbacks are part of their containing implementation, not separately named functions. Test cases are listed by their declared names. Direct calls are syntactic navigation aids, not a complete runtime call graph; calls inside nested anonymous callbacks are omitted from that list. Leading source comments are reproduced as explanations and are not independent verification.

Root configuration, infrastructure, non-JavaScript scripts, generated reports, and runtime flows are explained in the walkthrough. This includes the uncommitted admin collection/account changes from the preceding task; their presence does not establish deployment or a passing test run.

Regenerate with `node docs/generate-codebase-index.cjs`.

apps/backend/scripts/backfill-search-embeddings.mjs

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Function/component	Parameters	Explanation and direct calls
embed (line 16)	text	No leading explanation comment; read the linked implementation. Direct named calls: <code>createLocalSearchEmbedding</code> , <code>fetch</code> , <code>encodeURIComponent</code> , <code>JSON.stringify</code> , <code>text.slice</code> , <code>AbortSignal.timeout</code> , <code>normalizeEmbeddingResponse</code> , <code>response.json</code> .

apps/backend/scripts/migrate-vehicle-categories.mjs

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/app.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `app`.

Function/component	Parameters	Explanation and direct calls
checkWithin (line 59)	check, timeoutMs	Resolves false instead of waiting when a dependency does not answer quickly: a hung Redis or MongoDB must never make the health endpoint itself hang. Direct named calls: <code>Promise.race</code> , <code>check</code> , <code>clearTimeout</code> .

HTTP registrations (relative to the mount in `app.ts`):

- `app.use("/api")` — line 51
- `app.get("/health/live")` — line 53
- `app.get("/health/ready")` — line 72
- `app.use("/api/v1/auth")` — line 85
- `app.use("/api/v1/listings")` — line 88

- `app.use("/api/v1/listings")` — line 89
- `app.use("/api/v1/search")` — line 90
- `app.use("/api/v1/listing-images")` — line 91
- `app.use("/api/v1/dealers")` — line 92
- `app.use("/api/v1/dealer/uploads")` — line 93
- `app.use("/api/v1/admin")` — line 94
- `app.use("/api/v1/notifications")` — line 95

apps/backend/src/config/database.ts

Configuration, validated environment settings, or a shared external-service client.

Exported declarations: `connectDatabase`, `disconnectDatabase`.

Function/component	Parameters	Explanation and direct calls
<code>connectDatabase</code> (line 5)	None	Opens the shared Mongoose connection before the API starts accepting requests. Direct named calls: <code>mongoose.connect</code> .
<code>disconnectDatabase</code> (line 10)	None	Closes MongoDB cleanly during application shutdown. Direct named calls: <code>mongoose.disconnect</code> .

apps/backend/src/config/env.ts

Configuration, validated environment settings, or a shared external-service client.

Exported declarations: `env`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/config/firebase.ts

Configuration, validated environment settings, or a shared external-service client.

Exported declarations: `firebaseAuth`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/config/logger.ts

Configuration, validated environment settings, or a shared external-service client.

Exported declarations: `logger`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/config/mongoUri.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

Declared test groups and cases:

- describe: findProductionMongoUriProblems (line 4)
- it: accepts an Atlas SRV URI naming the production database (line 5)
- it: accepts a standard URI with TLS enabled (line 9)
- it: rejects unencrypted and local connections (line 13)
- it: rejects SRV URIs that explicitly disable TLS (line 19)
- it.each(['motorx_test', 'motorx-dev', 'motorx_development', 'local']): rejects the non-production database %s (line 23)
- it: requires an explicit database name (line 27)
- it: rejects strings that are not MongoDB URIs (line 31)

apps/backend/src/config/queue.ts

Configuration, validated environment settings, or a shared external-service client.

Exported declarations: inventoryQueue, closeInventoryQueue.

Function/component	Parameters	Explanation and direct calls
closeInventoryQueue (line 20)	None	Closes the Redis producer during graceful backend shutdown. Direct named calls: <code>inventoryQueue.close</code> .

apps/backend/src/config/redis.ts

Configuration, validated environment settings, or a shared external-service client.

Exported declarations: redisClient, disconnectRedis.

Function/component	Parameters	Explanation and direct calls
disconnectRedis (line 17)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>redisClient.quit</code> .

apps/backend/src/config/storage.ts

Configuration, validated environment settings, or a shared external-service client.

Exported declarations: storageClient, storageConfig.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/modules/admin/admin.controller.ts

HTTP boundary: reads request values, calls services, and sends responses.

Exported declarations: `getAdminUsers`, `patchAdminUser`, `getAdminListings`, `removeAdminListing`, `getAdminDashboardStats`, `getAdminAuditLogs`, `getAdminUploads`, `getAdminSystemHealth`, `getAdminDealerApplications`, `getAdminDealerDocument`, `approveAdminDealerApplication`, `rejectAdminDealerApplication`, `getAdminUserDetails`, `getAdminUploadRecords`.

Function/component	Parameters	Explanation and direct calls
<code>getAdminUsers</code> (line 9)	<code>request</code> , <code>response</code>	Sends the filtered user-management collection. Direct named calls: <code>getUsersForAdmin</code> , <code>sendSuccess</code> .
<code>patchAdminUser</code> (line 12)	<code>request</code> , <code>response</code>	Applies an administrator-requested account status change. Direct named calls: <code>sendSuccess</code> , <code>changeUserStatusAsAdmin</code> , <code>String</code> .
<code>getAdminListings</code> (line 15)	<code>request</code> , <code>response</code>	Sends the filtered cross-dealership listing collection. Direct named calls: <code>getListingsForAdmin</code> , <code>sendSuccess</code> .
<code>removeAdminListing</code> (line 18)	<code>request</code> , <code>response</code>	Archives one listing through the admin service. Direct named calls: <code>sendSuccess</code> , <code>removeListingAsAdmin</code> , <code>String</code> .
<code>getAdminDashboardStats</code> (line 21)	<code>_request</code> , <code>response</code>	Sends aggregate platform statistics. Direct named calls: <code>sendSuccess</code> , <code>getDashboardStatsForAdmin</code> .
<code>getAdminAuditLogs</code> (line 24)	<code>request</code> , <code>response</code>	Sends filtered persistent administrative events. Direct named calls: <code>getAuditLogsForAdmin</code> , <code>sendSuccess</code> .
<code>getAdminUploads</code> (line 27)	<code>request</code> , <code>response</code>	Sends platform-wide upload processing activity. Direct named calls: <code>getUploadsForAdmin</code> , <code>sendSuccess</code> .
<code>getAdminSystemHealth</code> (line 30)	<code>_request</code> , <code>response</code>	Sends the current operational status snapshot. Direct named calls: <code>sendSuccess</code> , <code>getSystemHealthForAdmin</code> .
<code>getAdminDealerApplications</code> (line 33)	<code>request</code> , <code>response</code>	Sends the pending dealer review queue. Direct named calls: <code>sendSuccess</code> , <code>getDealerApplicationsForAdmin</code> .
<code>getAdminDealerDocument</code> (line 36)	<code>request</code> , <code>response</code>	Streams one protected dealer verification document. Direct named calls: <code>getDealerDocumentForAdmin</code> , <code>String</code> , <code>Number</code> , <code>readDealerDocument</code> , <code>response.setHeader</code> , <code>document.originalName.replace</code> , <code>response.send</code> .
<code>approveAdminDealerApplication</code> (line 50)	<code>request</code> , <code>response</code>	Approves one pending dealer application. Direct named calls: <code>sendSuccess</code> , <code>reviewDealerApplicationAsAdmin</code> , <code>String</code> .
<code>rejectAdminDealerApplication</code> (line 53)	<code>request</code> , <code>response</code>	Rejects one pending dealer application with a reason. Direct named calls: <code>sendSuccess</code> , <code>reviewDealerApplicationAsAdmin</code> , <code>String</code> .
<code>getAdminUserDetails</code> (line 55)	<code>request</code> , <code>response</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>response.setHeader</code> , <code>sendSuccess</code> , <code>getUserDetailsForAdmin</code> , <code>String</code> .
<code>getAdminUploadRecords</code> (line 59)	<code>request</code> , <code>response</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>getUploadRecordsForAdmin</code> , <code>String</code> , <code>sendSuccess</code> .

apps/backend/src/modules/admin/admin.documentAccess.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

Declared test groups and cases:

- describe: admin access to dealer verification documents (line 36)
- it: records who viewed which document in the audit log (line 39)
- it: answers 410 Gone once documents were deleted by the retention policy, without logging a view (line 50)
- it: does not log a view for a document index that does not exist (line 57)

apps/backend/src/modules/admin/admin.model.ts

Database shapes, Mongoose models, and indexes. Models describe stored documents; they do not open the database connection.

Exported declarations: AdminAuditEvent, AdminAuditLog, AdminAuditLogModel.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/modules/admin/admin.repository.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

Declared test groups and cases:

- describe: admin repository (line 7)
- it: lists pending dealer applications oldest first (line 10)

apps/backend/src/modules/admin/admin.repository.ts

Persistence operations: reads or writes stored records for the owning feature.

Exported declarations: listAdminUsers, updateAdminUserStatus, listAdminListings, archiveListingByAdmin, getAdminStats, createAdminAuditLog, listAdminAuditLogs, listAdminUploads, listDealerApplications, listPendingDealerApplications, findDealerApplicationById, updateDealerApplicationReview, promoteApplicantToDealer, findAdminUser, findAdminDealer, findAdminUpload, listAdminUploadRecords.

Function/component	Parameters	Explanation and direct calls
escapeRegex (line 11)	value	No leading explanation comment; read the linked implementation. Direct named calls: value.replace.
listAdminUsers (line 14)	options	Builds a bounded user query without exposing raw search input to regex syntax. Direct named calls: escapeRegex, Promise.all, AuthUserModel.find, AuthUserModel.countDocuments.
updateAdminUserStatus (line 29)	userId, status, session	No leading explanation comment; read the linked implementation. Direct named calls: AuthUserModel.findByIdAndUpdate.
listAdminListings (line 34)	options	Includes dealer identity so administrators can moderate across dealerships. Direct named calls: escapeRegex, Promise.all, ListingModel.find, ListingModel.countDocuments.

Function/component	Parameters	Explanation and direct calls
archiveListingByAdmin (line 51)	listingId, session	Soft removal preserves the listing for later audit inspection. Direct named calls: ListingModel.findByIdAndUpdate.
getAdminStats (line 57)	None	Independent counts run together to minimize dashboard latency. Direct named calls: Promise.all, AuthUserModel.countDocuments, ListingModel.countDocuments, DealerModel.countDocuments.
createAdminAuditLog (line 68)	input, session	No leading explanation comment; read the linked implementation. Direct named calls: AdminAuditLogModel.create.
createdWithin (line 73)	from, to	createdAt condition for an inclusive YYYY-MM-DD date range (UTC days).
listAdminAuditLogs (line 78)	options	No leading explanation comment; read the linked implementation. Direct named calls: createdWithin, Promise.all, AdminAuditLogModel.find, AdminAuditLogModel.countDocuments.
listAdminUploads (line 87)	options	No leading explanation comment; read the linked implementation. Direct named calls: createdWithin, Promise.all, UploadJobModel.find, UploadJobModel.countDocuments.
listDealerApplications (line 99)	options	Loads pending applications in submission order for the admin review queue. Direct named calls: DealerModel.find.
listPendingDealerApplications (line 105)	None	Backward-compatible pending queue helper for repository callers and tests. Direct named calls: listDealerApplications.
findDealerApplicationById (line 110)	dealerId, session	Loads one application, optionally inside the review transaction. Direct named calls: DealerModel.findById.
updateDealerApplicationReview (line 115)	dealerId, status, adminId, rejectionReason, session	Applies a review decision only while the application remains pending. Direct named calls: DealerModel.findOneAndUpdate.
promoteApplicantToDealer (line 123)	userId, session	Grants dealer access in the same transaction as application approval. Direct named calls: AuthUserModel.findByIdAndUpdate.
findAdminUser (line 127)	userId	No leading explanation comment; read the linked implementation. Direct named calls: AuthUserModel.findById.
findAdminDealer (line 128)	userId	No leading explanation comment; read the linked implementation. Direct named calls: DealerModel.findOne.
findAdminUpload (line 131)	uploadId	No leading explanation comment; read the linked implementation. Direct named calls: UploadJobModel.findById.
listAdminUploadRecords (line 132)	uploadId, options	No leading explanation comment; read the linked implementation. Direct named calls: Promise.all, model.find, model.countDocuments.

apps/backend/src/modules/admin/admin.routes.ts

Route definitions: inspect the listed registrations and guards. Express routes are mounted by backend app.ts; frontend route arrays are metadata, while app/App.tsx mounts the actual React routes.

Exported declarations: adminRouter.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

HTTP registrations (relative to the mount in app.ts):

- adminRouter.get("/users") — line 15

- `adminRouter.patch("/users/:userId")` — line 16
- `adminRouter.get("/listings")` — line 17
- `adminRouter.delete("/listings/:listingId")` — line 18
- `adminRouter.get("/stats")` — line 19
- `adminRouter.get("/audit-logs")` — line 20
- `adminRouter.get("/uploads")` — line 21
- `adminRouter.get("/system-health")` — line 22
- `adminRouter.get("/dealer-applications")` — line 23
- `adminRouter.get("/dealer-applications/:dealerId/documents/:documentIndex")` — line 24
- `adminRouter.patch("/dealer-applications/:dealerId/approve")` — line 25
- `adminRouter.patch("/dealer-applications/:dealerId/reject")` — line 26
- `adminRouter.get("/users/:userId")` — line 28
- `adminRouter.get("/uploads/:uploadId/records")` — line 29

apps/backend/src/modules/admin/admin.service.ts

Business operations and orchestration; see each function and its direct calls below.

Exported declarations: `getUsersForAdmin`, `changeUserStatusAsAdmin`, `getListingsForAdmin`, `removeListingAsAdmin`, `getDashboardStatsForAdmin`, `getAuditLogsForAdmin`, `getUploadsForAdmin`, `getSystemHealthForAdmin`, `getDealerApplicationsForAdmin`, `getDealerDocumentForAdmin`, `reviewDealerApplicationAsAdmin`, `getUserDetailsForAdmin`, `getUploadRecordsForAdmin`.

Function/component	Parameters	Explanation and direct calls
<code>serializeUser</code> (line 17)	<code>user</code>	Converts user persistence fields into the admin API representation. Direct named calls: <code>String</code> .
<code>serializeListing</code> (line 20)	<code>record</code>	Converts populated listing fields into the admin API representation. Direct named calls: <code>String</code> .
<code>serializeDealer</code> (line 26)	<code>dealer</code>	Converts dealer persistence fields into the shared administration contract. Direct named calls: <code>dealer._id.toString</code> , <code>dealer.userId.toString</code> , <code>dealer.reviewedBy?.toString</code> , <code>dealer.reviewedAt?.toISOString</code> , <code>dealer.documentsDeletedAt?.toISOString</code> , <code>serializeReviewState</code> , <code>dealer.createdAt.toISOString</code> , <code>dealer.updatedAt.toISOString</code> .
<code>inTransaction</code> (line 42)	<code>fn</code>	Runs <code>fn</code> in one MongoDB transaction (retried by the driver on transient errors) and returns its result. Direct named calls: <code>mongoose.startSession</code> , <code>session.withTransaction</code> , <code>session.endSession</code> .
<code>getUsersForAdmin</code> (line 52)	<code>query</code>	Returns a paginated administrative view of user accounts. Direct named calls: <code>listAdminUsers</code> , <code>result.documents.map</code> , <code>buildPaginationMeta</code> .
<code>changeUserStatusAsAdmin</code> (line 55)	<code>userId</code> , <code>status</code> , <code>adminId</code>	Changes account access and records the significant operation. Direct named calls: <code>adminId.toString</code> , <code>inTransaction</code> , <code>invalidateHiddenDealerIds</code> , <code>notifyAccountSuspended</code> , <code>serializeUser</code> .
<code>getListingsForAdmin</code> (line 72)	<code>query</code>	Returns a paginated cross-dealership listing view. Direct named calls: <code>listAdminListings</code> , <code>result.documents.map</code> , <code>buildPaginationMeta</code> .

Function/component	Parameters	Explanation and direct calls
<code>removeListingAsAdmin</code> (line 75)	<code>listingId</code> , <code>adminId</code>	Archives a listing and records which administrator removed it. Direct named calls: <code>inTransaction</code> , <code>notifyListingRemoved</code> , <code>record._id.toString</code> , <code>record.createdAt.toISOString</code> , <code>serializeListing</code> .
<code>getDashboardStatsForAdmin</code> (line 98)	None	Returns database-backed dashboard totals. Direct named calls: <code>getAdminStats</code> .
<code>getAuditLogsForAdmin</code> (line 101)	<code>query</code>	Returns paginated audit history with populated administrator names. Direct named calls: <code>listAdminAuditLogs</code> , <code>result.documents.map</code> , <code>buildPaginationMeta</code> .
<code>getUploadsForAdmin</code> (line 108)	<code>query</code>	Returns paginated platform-wide inventory upload activity. Direct named calls: <code>listAdminUploads</code> , <code>result.documents.map</code> , <code>buildPaginationMeta</code> .
<code>getSystemHealthForAdmin</code> (line 121)	None	Reports truthful live state for configured backend dependencies. Direct named calls: <code>inventoryQueue.getJobCounts</code> , <code>Date.now</code> , <code>Math.floor</code> , <code>process.uptime</code> , <code>lastSeenAt?.toISOString</code> .
<code>getDealerApplicationsForAdmin</code> (line 147)	<code>query</code>	Returns the pending applications visible to administrators. Direct named calls: <code>listDealerApplications</code> , <code>records.map</code> .
<code>getDealerDocumentForAdmin</code> (line 151)	<code>dealerId</code> , <code>index</code> , <code>adminId</code>	Returns protected document metadata after validating its application and index, and records who opened which identity document in the audit log before any bytes are released. Direct named calls: <code>findDealerApplicationById</code> , <code>dealer.documentsDeletedAt.toISOString</code> , <code>createAdminAuditLog</code> .
<code>assertApplicantEmailVerified</code> (line 163)	<code>dealerId</code>	A dealer can publish to every buyer, so their email must be proven before approval. Checked with Firebase at approval time (not from a token), so it reflects the applicant's current state. Direct named calls: <code>findDealerApplicationById</code> , <code>AuthUserModel.findById</code> , <code>firebaseAuth.getUser</code> .
<code>reviewDealerApplicationAsAdmin</code> (line 173)	<code>dealerId</code> , <code>adminId</code> , <code>decision</code> , <code>reason</code>	Reviews an application, role assignment, and audit record as one transaction. Direct named calls: <code>assertApplicantEmailVerified</code> , <code>mongoose.startSession</code> , <code>session.withTransaction</code> , <code>session.endSession</code> , <code>notifyDealerApplicationDecision</code> .
<code>getUserDetailsForAdmin</code> (line 194)	<code>userId</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>findAdminUser</code> , <code>findAdminDealer</code> , <code>serializeUser</code> .
<code>getUploadRecordsForAdmin</code> (line 200)	<code>uploadId</code> , <code>query</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>findAdminUpload</code> , <code>listAdminUploadRecords</code> , <code>result.documents.map</code> , <code>buildPaginationMeta</code> .

apps/backend/src/modules/admin/admin.validation.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

Declared test groups and cases:

- describe: admin validation (line 4)
- it: applies bounded pagination defaults to admin collections (line 5)
- it: accepts supported user filters (line 10)
- it: accepts supported listing filters (line 16)

- it: rejects unsupported status values (line 22)

apps/backend/src/modules/admin/admin.validation.ts

Validation rules and related types; these reject or normalize unsupported input.

Exported declarations: `listAdminUsersQuerySchema`, `adminUserIdParamsSchema`, `updateAdminUserBodySchema`, `listAdminListingsQuerySchema`, `adminListingIdParamsSchema`, `adminDealerIdParamsSchema`, `adminDealerDocumentParamsSchema`, `rejectDealerApplicationBodySchema`, `listAdminAuditQuerySchema`, `listAdminUploadsQuerySchema`, `listAdminDealerApplicationsQuerySchema`, `ListAdminUsersQuery`, `ListAdminListingsQuery`, `ListAdminAuditQuery`, `ListAdminUploadsQuery`, `ListAdminDealerApplicationsQuery`, `adminUploadIdParamsSchema`, `adminUploadRecordsQuerySchema`, `AdminUploadRecordsQuery`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/modules/admin/audit-log.model.ts

Database shapes, Mongoose models, and indexes. Models describe stored documents; they do not open the database connection.

Exported declarations: `AuditLogDocument`, `AuditLogModel`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/modules/admin/index.ts

Module entry point or re-export barrel; inspect exported symbols/import targets.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/modules/auth-users/authUser.model.ts

Database shapes, Mongoose models, and indexes. Models describe stored documents; they do not open the database connection.

Exported declarations: `AuthUser`, `AuthUserDocument`, `AuthUserModel`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/modules/auth-users/authUser.repository.ts

Persistence operations: reads or writes stored records for the owning feature.

Exported declarations: `findOrCreateAuthUser`.

Function/component	Parameters	Explanation and direct calls
<code>findOrCreateAuthUser</code> (line 11)	<code>identity</code>	Synchronizes a verified Firebase identity with its local MotorX user. Reads on every request (role and suspension status must always be current) but writes only when the account is new, its email or name changed, or <code>lastLoginAt</code> is older than <code>LAST_LOGIN_REFRESH_MS</code> . Direct named calls:

Function/component	Parameters	Explanation and direct calls
		<code>identity.email?.trim</code> , <code>AuthUserModel.findOne</code> , <code>Date.now</code> , <code>AuthUserModel.findOneAndUpdate</code> .

apps/backend/src/modules/auth-users/authUser.routes.ts

Route definitions: inspect the listed registrations and guards. Express routes are mounted by backend `app.ts`; frontend route arrays are metadata, while `app/App.tsx` mounts the actual React routes.

Exported declarations: `authUserRouter`.

Function/component	Parameters	Explanation and direct calls
<code>serializeUser</code> (line 18)	<code>user</code>	No leading explanation comment; read the linked implementation.

HTTP registrations (relative to the mount in `app.ts`):

- `authUserRouter.get("/me")` — line 31
- `authUserRouter.patch("/me")` — line 42

apps/backend/src/modules/auth-users/index.ts

Module entry point or re-export barrel; inspect exported symbols/import targets.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/modules/buyers/buyer.controller.ts

HTTP boundary: reads request values, calls services, and sends responses.

Exported declarations: `getBuyerListings`, `getBuyerListing`, `getSimilarListings`, `getRecommendedListings`.

Function/component	Parameters	Explanation and direct calls
<code>getBuyerListings</code> (line 7)	<code>request</code> , <code>response</code>	Sends the public marketplace collection without using response metadata. Direct named calls: <code>sendSuccess</code> , <code>browseListingsAsBuyer</code> .
<code>getBuyerListing</code> (line 10)	<code>request</code> , <code>response</code>	Sends one public vehicle listing. Direct named calls: <code>sendSuccess</code> , <code>viewListingAsBuyer</code> , <code>String</code> .
<code>getSimilarListings</code> (line 13)	<code>request</code> , <code>response</code>	Sends the public listings most like one listing. Direct named calls: <code>sendSuccess</code> , <code>similarListingsForBuyer</code> , <code>String</code> .
<code>getRecommendedListings</code> (line 18)	<code>request</code> , <code>response</code>	Sends public listings matched to the vehicles this browser viewed recently. Direct named calls: <code>sendSuccess</code> , <code>recommendedListingsForBuyer</code> .

apps/backend/src/modules/buyers/buyer.repository.ts

Persistence operations: reads or writes stored records for the owning feature.

Exported declarations: `listBuyerListings`, `findBuyerListingById`, `findRecommendationCandidates`, `findViewedListings`.

Function/component	Parameters	Explanation and direct calls
<code>listBuyerListings</code> (line 8)	<code>options</code>	Returns a filtered page containing only publicly available listings. Direct named calls: <code>listStructuredListings</code> .
<code>findBuyerListingById</code> (line 11)	<code>listingId</code>	Returns one listing only when it is public: active, and its dealer is not suspended. Direct named calls: <code>ListingModel.findOne</code> , <code>publicDealerFilter</code> .
<code>findRecommendationCandidates</code> (line 15)	<code>options</code>	Most-recent public listings that could be recommended: bounded, index-friendly (status + category + price), never including the vehicles the buyer is already looking at. Direct named calls: <code>publicDealerFilter</code> , <code>ListingModel.find</code> .
<code>findViewedListings</code> (line 23)	<code>ids</code>	The vehicles a buyer recently viewed, used only to describe their taste. Sold listings still count (the buyer liked that kind of vehicle); drafts and archived listings never do. Direct named calls: <code>ListingModel.find</code> .

apps/backend/src/modules/buyers/buyer.routes.ts

Route definitions: inspect the listed registrations and guards. Express routes are mounted by backend `app.ts`; frontend route arrays are metadata, while `app/App.tsx` mounts the actual React routes.

Exported declarations: `buyerRouter`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

HTTP registrations (relative to the mount in `app.ts`):

- `buyerRouter.get("/")` — line 8
- `buyerRouter.get("/recommendations")` — line 10
- `buyerRouter.get("/:listingId/similar")` — line 11
- `buyerRouter.get("/:listingId")` — line 12

apps/backend/src/modules/buyers/buyer.service.ts

Business operations and orchestration; see each function and its direct calls below.

Exported declarations: `BuyerDealerInfo`, `BuyerListingDetailDto`, `browseListingsAsBuyer`, `viewListingAsBuyer`, `similarListingsForBuyer`, `recommendedListingsForBuyer`.

Function/component	Parameters	Explanation and direct calls
<code>browseListingsAsBuyer</code> (line 16)	<code>query</code>	Returns listings and pagination together inside data instead of response metadata. Direct named calls: <code>listBuyerListings</code> , <code>result.documents.map</code> , <code>buildPaginationMeta</code> .
<code>viewListingAsBuyer</code> (line 19)	<code>listingId</code>	Returns the requested active listing, with its dealer's public profile, or a public not-found response. Direct named calls: <code>findBuyerListingById</code> , <code>DealerModel.findOne</code> , <code>serializeListing</code> .
<code>candidatePool</code> (line 39)	<code>options</code> , <code>wanted</code>	Candidates in a similar price band first. When that band is too thin (a rare model, a small marketplace), other vehicles of the same types fill the gap, so the section is rarely empty. Direct named calls: <code>findRecommendationCandidates</code> , <code>nearby.map</code> , <code>wider.filter</code> .
<code>similarListingsForBuyer</code> (line 48)	<code>listingId</code> , <code>limit</code>	"Similar vehicles" on a listing page: the same kind of vehicle, closest in make, model, price and age. Direct named calls: <code>findBuyerListingById</code> , <code>candidatePool</code> , <code>rankByScore</code> .

Function/component	Parameters	Explanation and direct calls
<code>recommendedListingsForBuyer</code> (line 57)	<code>viewedIds</code> , <code>limit</code>	"Recommended for you": ranks public listings against the vehicles this browser viewed recently (sent by the page, newest first). Nothing about the buyer is stored on the server. Direct named calls: <code>findViewedListings</code> , <code>viewedIds.map</code> , <code>seeds.map</code> , <code>candidatePool</code> , <code>Math.min</code> , <code>Math.max</code> , <code>rankByScore</code> .

apps/backend/src/modules/buyers/buyer.similarity.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
<code>vary</code> (line 5)	<code>changes</code>	No leading explanation comment; read the linked implementation.

Declared test groups and cases:

- `describe`: vehicle similarity (line 7)
- `it`: scores the same model above the same make above another make (line 8)
- `it`: prefers a closer price and year (line 16)
- `it`: ignores letter case and spacing in makes, and compares the town part of a location (line 21)
- `it`: weights the most recently viewed vehicle more than older views (line 25)
- `it`: keeps the original (newest first) order for equal scores (line 32)

apps/backend/src/modules/buyers/buyer.similarity.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `ComparableVehicle`, `similarityScore`, `profileScore`, `rankByScore`.

Function/component	Parameters	Explanation and direct calls
<code>same</code> (line 14)	<code>a</code> , <code>b</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>a.trim</code> , <code>b.trim</code> .
<code>closeness</code> (line 15)	<code>difference</code> , <code>range</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>Math.max</code> , <code>Math.abs</code> .
<code>town</code> (line 16)	<code>location</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>location.split</code> .
<code>similarityScore</code> (line 18)	<code>seed</code> , <code>candidate</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>same</code> , <code>closeness</code> , <code>Math.max</code> , <code>town</code> .
<code>profileScore</code> (line 41)	<code>seeds</code> , <code>candidate</code>	Recently viewed vehicles count more than older ones: the latest has weight 1, then 0.85, 0.72... Direct named calls: <code>seeds.forEach</code> .
<code>rankByScore</code> (line 48)	<code>candidates</code> , <code>score</code> , <code>limit</code>	Highest score first; equal scores keep the newer listing first (candidates arrive newest first). Direct named calls: <code>candidates.map</code> .

apps/backend/src/modules/buyers/buyer.validation.ts

Validation rules and related types; these reject or normalize unsupported input.

Exported declarations: `similarListingsQuerySchema`, `recommendationsQuerySchema`, `SimilarListingsQuery`, `RecommendationsQuery`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/modules/buyers/index.ts

Module entry point or re-export barrel; inspect exported symbols/import targets.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/modules/dealers/dealer.controller.ts

HTTP boundary: reads request values, calls services, and sends responses.

Exported declarations: `createApplication`, `getMyApplication`, `updateMyProfile`.

Function/component	Parameters	Explanation and direct calls
<code>createApplication</code> (line 9)	<code>request</code> , <code>response</code>	Handles submission of a buyer's dealer application. Direct named calls: <code>storeDealerDocuments</code> , <code>request.localUser!._id.toString</code> , <code>submitDealerApplication</code> , <code>sendSuccess</code> , <code>deleteDealerDocuments</code> .
<code>getMyApplication</code> (line 24)	<code>request</code> , <code>response</code>	Returns the authenticated user's application. Direct named calls: <code>sendSuccess</code> , <code>getMyDealerApplication</code> .
<code>updateMyProfile</code> (line 29)	<code>request</code> , <code>response</code>	Updates the approved dealer's editable business details. Direct named calls: <code>sendSuccess</code> , <code>updateMyDealerProfile</code> .

apps/backend/src/modules/dealers/dealer.model.ts

Database shapes, Mongoose models, and indexes. Models describe stored documents; they do not open the database connection.

Exported declarations: `Dealer`, `DealerDocument`, `DealerModel`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/modules/dealers/dealer.repository.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
<code>application</code> (line 11)	<code>businessName</code> , <code>registrationNumber</code> , <code>phone</code> , <code>address</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>registrationNumber.toLowerCase</code> .

Declared test groups and cases:

- `describe`: dealer repository (line 6)
- `it`: creates a pending dealer application and finds it by user id (line 20)

apps/backend/src/modules/dealers/dealer.repository.ts

Persistence operations: reads or writes stored records for the owning feature.

Exported declarations: `findDealerByUserId`, `createDealer`, `resubmitRejectedApplication`, `updateApprovedDealerProfile`.

Function/component	Parameters	Explanation and direct calls
<code>findDealerByUserId</code> (line 8)	<code>userId</code>	Finds the single dealer application belonging to a user. Direct named calls: <code>DealerModel.findOne</code> .
<code>createDealer</code> (line 10)	<code>userId</code> , <code>input</code>	Creates a pending dealer application for a buyer. Direct named calls: <code>DealerModel.create</code> .
<code>resubmitRejectedApplication</code> (line 15)	<code>previous</code> , <code>input</code>	Replaces a rejected application with the corrected one and sends it back for review. The earlier decision moves to <code>reviewHistory</code> . Matching on status 'rejected' turns a concurrent double submission (or one after approval) into a no-op that returns null. Direct named calls: <code>DealerModel.findOneAndUpdate</code> .
<code>updateApprovedDealerProfile</code> (line 33)	<code>userId</code> , <code>input</code>	Updates an approved dealer's editable business details (website " removes the website). Direct named calls: <code>DealerModel.findOneAndUpdate</code> .

apps/backend/src/modules/dealers/dealer.routes.ts

Route definitions: inspect the listed registrations and guards. Express routes are mounted by backend `app.ts`; frontend route arrays are metadata, while `app/App.tsx` mounts the actual React routes.

Exported declarations: `dealerRouter`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

HTTP registrations (relative to the mount in `app.ts`):

- `dealerRouter.post("/applications")` — line 17
- `dealerRouter.get("/me")` — line 18
- `dealerRouter.patch("/me/profile")` — line 19

apps/backend/src/modules/dealers/dealer.service.ts

Business operations and orchestration; see each function and its direct calls below.

Exported declarations: `serializeReviewState`, `submitDealerApplication`, `getMyDealerApplication`, `updateMyDealerProfile`.

Function/component	Parameters	Explanation and direct calls
<code>serializeReviewState</code> (line 12)	<code>dealer</code>	Submission time and earlier rejections, shared by the dealer and admin representations.
<code>serializeDealer</code> (line 20)	<code>dealer</code>	Converts a MongoDB dealer record into the public API contract. Direct named calls: <code>dealer._id.toString</code> , <code>dealer.userId.toString</code> , <code>dealer.reviewedBy?.toString</code> , <code>dealer.reviewedAt?.toISOString</code> , <code>dealer.documentsDeletedAt?.toISOString</code> , <code>serializeReviewState</code> , <code>dealer.createdAt.toISOString</code> , <code>dealer.updatedAt.toISOString</code> .

Function/component	Parameters	Explanation and direct calls
submitDealerApplication (line 40)	userId, role, input, verificationDocuments	Creates a buyer's dealer application or, after a rejection, replaces it with the corrected version: the earlier decision is kept in reviewHistory and its documents are deleted. Direct named calls: findDealerById, resubmitRejectedApplication, deleteDealerDocuments, serializeDealer, resubmitted.toObject, notifyDealerApplicationSubmitted, createDealer.
getMyDealerApplication (line 66)	userId	Returns the current user's dealer application and review state. Direct named calls: findDealerById, serializeDealer.
updateMyDealerProfile (line 74)	userId, input	Lets an approved dealer keep their public business details current (FR-DEALER-03). Direct named calls: updateApprovedDealerProfile, serializeDealer.

apps/backend/src/modules/dealers/dealer.validation.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

Declared test groups and cases:

- describe: dealer application validation (line 21)
- it: normalizes multipart form fields into the dealer application shape (line 22)
- it: requires enough information for a meaningful review (line 32)
- it: requires a rejection reason (line 37)

apps/backend/src/modules/dealers/dealer.validation.ts

Validation rules and related types; these reject or normalize unsupported input.

Exported declarations: createDealerApplicationSchema, updateDealerProfileSchema, dealerIdParamsSchema, rejectionBodySchema, CreateDealerApplicationBody, UpdateDealerProfileBody.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/modules/dealers/dealerDocument.content.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
pdf (line 6)	body	No leading explanation comment; read the linked implementation. Direct named calls: Buffer.from.
file (line 7)	buffer, originalname	No leading explanation comment; read the linked implementation.

Declared test groups and cases:

- describe: detectDocumentType (line 9)

- it: identifies files by their bytes, not their name or declared type (line 10)
- describe: `sanitizeDealerDocument` (line 18)
- it: accepts a plain PDF unchanged (line 19)
- it.each([['JavaScript', '/OpenAction << /S /JavaScript /JS (app.alert(1)) >>'], ['a launch action', '/OpenAction << /S /Launch /F (cmd.exe) >>'], ['an embedded file', '/Names << /EmbeddedFiles 3 0 R >>'],]): rejects a PDF containing %s (line 27)
- it: does not mistake /JSON-like names for scripts (line 35)
- it: rejects an executable renamed to `.pdf` (line 39)
- it: re-encodes image documents and strips their metadata (line 45)
- it: rejects a file that only starts like a JPEG (line 59)

apps/backend/src/modules/dealers/dealerDocument.content.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `DealerDocumentContentType`, `detectDocumentType`, `sanitizeDealerDocument`.

Function/component	Parameters	Explanation and direct calls
<code>detectDocumentType</code> (line 17)	<code>buffer</code>	Identifies a file by its actual bytes; the browser-declared MIME type is never trusted. Direct named calls: <code>buffer.subarray</code> , <code>Buffer.from</code> .
<code>sanitizeDealerDocument</code> (line 28)	<code>file</code>	Validates one verification document and returns the bytes that may be stored. Images are rebuilt from their pixels (no metadata such as GPS location, no hidden data); PDFs are accepted only when they contain no scripts, launch actions, or embedded files. This is a content check, not an antivirus scan. Direct named calls: <code>detectDocumentType</code> , <code>PDF_ACTIVE_CONTENT.test</code> , <code>file.buffer.toString</code> , <code>reencodeImage</code> .

apps/backend/src/modules/dealers/dealerDocument.middleware.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `uploadDealerDocuments`.

Function/component	Parameters	Explanation and direct calls
<code>fileFilter</code> (line 10)	<code>_request</code> , <code>file</code> , <code>callback</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>callback</code> , <code>acceptedTypes.has</code> .
<code>uploadDealerDocuments</code> (line 17)	<code>request</code> , <code>response</code> , <code>next</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>documentUpload</code> .

apps/backend/src/modules/dealers/dealerDocument.storage.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `StoredDealerDocument`, `storeDealerDocuments`, `deleteDealerDocuments`, `readDealerDocument`.

Function/component	Parameters	Explanation and direct calls
<code>storeDealerDocuments</code> (line 20)	<code>userId</code> , <code>files</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>Promise.all</code> , <code>files.map</code> , <code>randomUUID</code> , <code>storageClient.send</code> , <code>stored.push</code> , <code>deleteDealerDocuments</code> .

Function/component	Parameters	Explanation and direct calls
<code>deleteDealerDocuments</code> (line 42)	<code>documents</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>Promise.allSettled</code> , <code>documents.map</code> .
<code>readDealerDocument</code> (line 49)	<code>key</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>storageClient.send</code> , <code>Buffer.from</code> , <code>object.Body.transformToByteArray</code> .

apps/backend/src/modules/dealers/index.ts

Module entry point or re-export barrel; inspect exported symbols/import targets.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/modules/inventory/index.ts

Module entry point or re-export barrel; inspect exported symbols/import targets.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/modules/inventory/inventory.controller.ts

HTTP boundary: reads request values, calls services, and sends responses.

Exported declarations: `uploadDealerInventory`, `uploadDealerInventoryImages`, `retryInventoryUpload`, `retryInventoryUploadImages`, `downloadInventoryCsvTemplate`, `listMyInventoryUploads`, `getMyInventoryUpload`, `listMyUploadRejectedRecords`.

Function/component	Parameters	Explanation and direct calls
<code>uploadDealerInventory</code> (line 9)	<code>request</code> , <code>response</code>	Accepts one CSV (for the dealer-selected category) and immediately returns its queued upload job. Direct named calls: <code>sendSuccess</code> , <code>createInventoryUpload</code> .
<code>uploadDealerInventoryImages</code> (line 15)	<code>request</code> , <code>response</code>	Accepts one vehicle-photos zip for an already-processed upload job. Direct named calls: <code>sendSuccess</code> , <code>attachInventoryImagesZip</code> , <code>String</code> .
<code>retryInventoryUpload</code> (line 20)	<code>request</code> , <code>response</code>	Re-queues this dealer's failed CSV import (resumes from its last checkpoint). Direct named calls: <code>sendSuccess</code> , <code>retryDealerUpload</code> , <code>String</code> .
<code>retryInventoryUploadImages</code> (line 25)	<code>request</code> , <code>response</code>	Re-queues this dealer's failed photo processing for the zip already stored with the upload. Direct named calls: <code>sendSuccess</code> , <code>retryDealerUploadImages</code> , <code>String</code> .
<code>downloadInventoryCsvTemplate</code> (line 30)	<code>request</code> , <code>response</code>	Streams a downloadable CSV template (headers + example rows) for the requested category. Direct named calls: <code>getInventoryCsvTemplate</code> , <code>response.status</code> .
<code>listMyInventoryUploads</code> (line 36)	<code>request</code> , <code>response</code>	Sends the authenticated dealer's upload history without response metadata. Direct named calls: <code>sendSuccess</code> , <code>getDealerUploads</code> .
<code>getMyInventoryUpload</code> (line 39)	<code>request</code> , <code>response</code>	Sends one authenticated dealer-owned upload job. Direct named calls: <code>sendSuccess</code> , <code>getDealerUpload</code> , <code>String</code> .

Function/component	Parameters	Explanation and direct calls
<code>listMyUploadRejectedRecords</code> (line 42)	<code>request, response</code>	Sends the rejected rows for one authenticated dealer-owned upload job. Direct named calls: <code>sendSuccess</code> , <code>getDealerUploadRejectedRecords</code> , <code>String</code> .

apps/backend/src/modules/inventory/inventory.middleware.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `uploadSingleInventoryCsv`, `uploadSingleImagesZip`.

Function/component	Parameters	Explanation and direct calls
<code>fileFilter</code> (line 7)	<code>_request, file, callback</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>callback</code> , <code>storageConfig.allowedInventoryTypes.has</code> .
<code>uploadSingleInventoryCsv</code> (line 10)	<code>request, response, next</code>	Parses one CSV and converts Multer errors into stable validation responses. Direct named calls: <code>csvUpload</code> .
<code>fileFilter</code> (line 21)	<code>_request, file, callback</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>callback</code> , <code>zipMimeTypes.has</code> .
<code>uploadSingleImagesZip</code> (line 24)	<code>request, response, next</code>	Parses one vehicle-photos zip and converts Multer errors into stable validation responses. Direct named calls: <code>imagesZipUpload</code> .

apps/backend/src/modules/inventory/inventory.queue.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
<code>jobIn</code> (line 8)	<code>state</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>vi.fn</code> .

Declared test groups and cases:

- describe: inventory queue publishing (line 10)
- it: queues a new upload under its fixed job ID (line 13)
- it: processes photos attached a second time: the finished run is removed, then queued again (line 19)
- it: does the same after a failed run (line 30)
- it: leaves a job that is still waiting or running alone, so it is never processed twice at once (line 38)

apps/backend/src/modules/inventory/inventory.queue.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `enqueueInventoryUpload`, `enqueueInventoryImages`.

Function/component	Parameters	Explanation and direct calls
<code>addUnderFixedId</code> (line 11)	<code>name, uploadJobId, bullJobId</code>	Adds a job under its fixed ID. BullMQ keeps finished jobs for a while and silently ignores a new job with the same ID, so a dealer attaching photos to the same upload a second time (or retrying) would never be processed. A

Function/component	Parameters	Explanation and direct calls
		finished job is removed first; a job still waiting or running is left alone. The ID stays fixed because the worker's reaper finds jobs by it (same rule as there). Direct named calls: <code>inventoryQueue.getJob</code> , <code>ALIVE_STATES.has</code> , <code>existing.getState</code> , <code>existing.remove</code> , <code>inventoryQueue.add</code> .
<code>enqueueInventoryUpload</code> (line 21)	<code>uploadJobId</code>	Publishes only the durable MongoDB job ID, keeping CSV content out of Redis. Direct named calls: <code>addUnderFixedId</code> , <code>inventoryBullJobId.csv</code> .
<code>enqueueInventoryImages</code> (line 24)	<code>uploadJobId</code>	The photo stage of the same upload, under its own suffixed ID so it never collides with the CSV job. Direct named calls: <code>addUnderFixedId</code> , <code>inventoryBullJobId.images</code> .

apps/backend/src/modules/inventory/inventory.repository.ts

Persistence operations: reads or writes stored records for the owning feature.

Exported declarations: `createUploadJob`, `listDealerUploadJobs`, `findDealerUploadJob`, `markImageProcessingPending`, `resetFailedUploadJob`, `resetFailedImageProcessing`, `listRejectedRecordsForUpload`.

Function/component	Parameters	Explanation and direct calls
<code>createUploadJob</code> (line 7)	<code>input</code>	Creates the durable pending job before it is published to Redis. Direct named calls: <code>UploadJobModel.create</code> .
<code>listDealerUploadJobs</code> (line 10)	<code>dealerId</code> , <code>page</code> , <code>limit</code>	Lists only upload jobs owned by the authenticated dealer. Direct named calls: <code>Promise.all</code> , <code>UploadJobModel.find</code> , <code>UploadJobModel.countDocuments</code> .
<code>findDealerUploadJob</code> (line 13)	<code>uploadId</code> , <code>dealerId</code>	Finds one upload only when it belongs to the authenticated dealer. Direct named calls: <code>UploadJobModel.findOne</code> .
<code>markImageProcessingPending</code> (line 19)	<code>uploadId</code> , <code>dealerId</code> , <code>input</code>	Removes a pending record during compensation after enqueueing failure. Marks a finished CSV upload job as having a photos zip queued for processing. Only allowed once the CSV itself has finished, so the zip only ever matches listings that actually got created, and never while an earlier zip is still queued or running. A new zip gets a fresh attempt budget. Direct named calls: <code>UploadJobModel.findOneAndUpdate</code> .
<code>resetFailedUploadJob</code> (line 29)	<code>uploadId</code> , <code>dealerId</code>	Controlled retry of a failed CSV import: back to pending with a fresh attempt budget. Safe because processing resumes from its last checkpoint and never imports the same row twice. Direct named calls: <code>UploadJobModel.findOneAndUpdate</code> .
<code>resetFailedImageProcessing</code> (line 38)	<code>uploadId</code> , <code>dealerId</code>	Controlled retry of failed photo processing for the zip already stored with this upload. Direct named calls: <code>UploadJobModel.findOneAndUpdate</code> .
<code>listRejectedRecordsForUpload</code> (line 47)	<code>uploadJobId</code> , <code>page</code> , <code>limit</code>	Lists rejected rows for one upload job, most recent CSV row first. Direct named calls: <code>Promise.all</code> , <code>RejectedRecordModel.find</code> , <code>RejectedRecordModel.countDocuments</code> .

apps/backend/src/modules/inventory/inventory.routes.ts

Route definitions: inspect the listed registrations and guards. Express routes are mounted by backend `app.ts`; frontend route arrays are metadata, while `app/App.tsx` mounts the actual React routes.

Exported declarations: `inventoryRouter`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

HTTP registrations (relative to the mount in `app.ts`):

- `inventoryRouter.get("/template/:category")` — line 15
- `inventoryRouter.post("/")` — line 18
- `inventoryRouter.get("/")` — line 19
- `inventoryRouter.get("/:uploadId")` — line 20
- `inventoryRouter.get("/:uploadId/rejected-records")` — line 21
- `inventoryRouter.post("/:uploadId/images")` — line 22
- `inventoryRouter.post("/:uploadId/retry")` — line 23
- `inventoryRouter.post("/:uploadId/images/retry")` — line 24

apps/backend/src/modules/inventory/inventory.service.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
<code>jobDocument</code> (line 25)	overrides	No leading explanation comment; read the linked implementation.
<code>jobDocument.toObject</code> (line 25)	None	No leading explanation comment; read the linked implementation.

Declared test groups and cases:

- `describe: inventory upload acceptance` (line 27)
- `it: queues the job straight away when Redis is available` (line 35)
- `it: keeps the accepted upload (file and job) when Redis is down, for the worker to queue later` (line 45)
- `it: does not make the dealer wait when Redis hangs instead of refusing` (line 55)
- `it: tells a dealer at the listing limit before storing anything` (line 66)
- `it: still removes the stored file if the MongoDB job itself cannot be created` (line 73)
- `describe: controlled retry of failed jobs` (line 82)
- `it: re-queues a failed CSV import owned by the dealer` (line 85)
- `it: refuses to retry an upload that is not failed (or belongs to another dealer)` (line 93)
- `it: re-queues failed photo processing under the images job` (line 100)

apps/backend/src/modules/inventory/inventory.service.ts

Business operations and orchestration; see each function and its direct calls below.

Exported declarations: `createInventoryUpload`, `getInventoryCsvTemplate`, `attachInventoryImagesZip`, `retryDealerUpload`, `retryDealerUploadImages`, `getDealerUploads`, `getDealerUpload`, `getDealerUploadRejectedRecords`.

Function/component	Parameters	Explanation and direct calls
serializeUpload (line 16)	record	Converts an upload document into the dealer-safe API representation. Direct named calls: String.
enqueueOrDefer (line 30)	enqueue, uploadJobId	Enqueues promptly when Redis is reachable. A slow or failed enqueue never fails the request: the MongoDB job is already durable, and the worker re-queues any job left pending too long. Direct named calls: Promise.race, enqueue, logger.warn, String, clearTimeout.
serializeRejectedRecord (line 40)	record	Converts a rejected-record document into the dealer-safe API representation. Direct named calls: String.
createInventoryUpload (line 45)	dealerId, category, file	Stores, records, and enqueues one accepted CSV. Once the MongoDB job exists the upload is accepted: if Redis is unavailable the job simply stays pending and the worker's reconciler queues it later, instead of the upload being thrown away. Direct named calls: validateInventoryCsv, assertDealerListingCapacity, file.originalname.replace, randomUUID, storeInventoryCsv, createUploadJob, Promise.allSettled, deleteInventoryCsv, enqueueOrDefer, job._id.toString, serializeUpload, job.toObject.
getInventoryCsvTemplate (line 62)	category	Renders a downloadable CSV template (headers + example rows) for the given category. Direct named calls: buildCsvTemplateContent.
attachInventoryImagesZip (line 71)	dealerId, uploadId, file	Stores a vehicle-photos zip and queues it for matching against this upload job's listings. Only allowed once the CSV itself has finished (completed/completedWithErrors) — the zip is matched only against listings this exact job actually created. Direct named calls: validateInventoryImagesZip, file.originalname.replace, randomUUID, storeInventoryImagesZip, markImageProcessingPending, Promise.allSettled, deleteInventoryImagesZip, enqueueOrDefer, serializeUpload, job.toObject.
retryDealerUpload (line 87)	dealerId, uploadId	Controlled retry of a failed CSV import, owned by this dealer. Direct named calls: resetFailedUploadJob, enqueueOrDefer, serializeUpload, job.toObject.
retryDealerUploadImages (line 95)	dealerId, uploadId	Controlled retry of failed photo processing for this dealer's already uploaded zip. Direct named calls: resetFailedImageProcessing, enqueueOrDefer, serializeUpload, job.toObject.
getDealerUploads (line 103)	dealerId, query	Returns the authenticated dealer's upload history with pagination inside data. Direct named calls: listDealerUploadJobs, result.documents.map, buildPaginationMeta.
getDealerUpload (line 106)	dealerId, uploadId	Returns one dealer-owned upload or a non-disclosing not-found response. Direct named calls: findDealerUploadJob, serializeUpload.
getDealerUploadRejectedRecords (line 109)	dealerId, uploadId, query	Returns the rejected rows for one dealer-owned upload, verifying ownership first. Direct named calls: findDealerUploadJob, listRejectedRecordsForUpload, result.documents.map, buildPaginationMeta.

apps/backend/src/modules/inventory/inventory.storage.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: storeInventoryCsv, deleteInventoryCsv, storeInventoryImagesZip, deleteInventoryImagesZip.

Function/component	Parameters	Explanation and direct calls
storeInventoryCsv (line 5)	key, file	Stores the original private inventory file for worker processing and traceability. Direct named calls: <code>storageClient.send</code> .
deleteInventoryCsv (line 8)	key	Removes an uploaded object when job creation or enqueueing cannot complete. Direct named calls: <code>storageClient.send</code> .
storeInventoryImagesZip (line 11)	key, file	Stores the original vehicle-photos zip for worker processing. Direct named calls: <code>storageClient.send</code> .
deleteInventoryImagesZip (line 14)	key	Removes an uploaded zip when job creation or enqueueing cannot complete. Direct named calls: <code>storageClient.send</code> .

apps/backend/src/modules/inventory/inventory.validation.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

Declared test groups and cases:

- describe: inventory upload validation (line 6)
- it: accepts a car CSV containing every required header (line 7)
- it: reports missing required headers for the selected category (line 11)
- it: requires different headers for a different category (motorcycle needs bikeType, not bodyType) (line 17)
- it: rejects non-CSV extensions and binary content (line 23)
- it: applies bounded history pagination (line 28)
- it: applies bounded rejected-records pagination (line 33)

apps/backend/src/modules/inventory/inventory.validation.ts

Validation rules and related types; these reject or normalize unsupported input.

Exported declarations: `inventoryUploadIdParamsSchema`, `listInventoryUploadsQuerySchema`, `ListInventoryUploadsQuery`, `listRejectedRecordsQuerySchema`, `ListRejectedRecordsQuery`, `uploadCategoryBodySchema`, `csvTemplateParamsSchema`, `validateInventoryCsv`, `validateInventoryImagesZip`.

Function/component	Parameters	Explanation and direct calls
validateInventoryCsv (line 30)	file, category	Performs inexpensive structural checks before storing and enqueueing a CSV. Direct named calls: <code>file.originalname.toLowerCase</code> , <code>file.buffer.includes</code> , <code>file.buffer.subarray</code> , <code>Math.min</code> , <code>missing.join</code> .
hasZipSignature (line 41)	buffer	Zip's local-file-header signature ("PK\x03\x04") — also matches an empty archive's end-of-central-directory signature ("PK\x05\x06") so an intentionally empty zip isn't rejected.
validateInventoryImagesZip (line 47)	file	Performs inexpensive structural checks before storing and enqueueing a vehicle-photos zip. Direct named calls: <code>file.originalname.toLowerCase</code> , <code>hasZipSignature</code> .

apps/backend/src/modules/inventory/rejectedRecord.model.ts

Database shapes, Mongoose models, and indexes. Models describe stored documents; they do not open the database connection.

Exported declarations: `RejectedInventoryRecord`, `RejectedRecordModel`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/modules/inventory/uploadJob.model.ts

Database shapes, Mongoose models, and indexes. Models describe stored documents; they do not open the database connection.

Exported declarations: `ImageProcessingStatus`, `UploadJob`, `UploadJobModel`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/modules/marketplace/index.ts

Module entry point or re-export barrel; inspect exported symbols/import targets.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/modules/marketplace/listing.controller.ts

HTTP boundary: reads request values, calls services, and sends responses.

Exported declarations: `createListing`, `listMyListings`, `getMyListing`, `getMyListingStats`, `updateListing`, `updateListingStatus`, `bulkUpdateListings`, `deleteListing`.

Function/component	Parameters	Explanation and direct calls
<code>createListing</code> (line 9)	<code>request</code> , <code>response</code>	Creates a listing owned by the authenticated dealer. Direct named calls: <code>createDealerListing</code> , <code>sendSuccess</code> .
<code>listMyListings</code> (line 15)	<code>request</code> , <code>response</code>	Sends all listings owned by the authenticated dealer. Direct named calls: <code>getDealerListings</code> , <code>sendSuccess</code> .
<code>getMyListing</code> (line 21)	<code>request</code> , <code>response</code>	Sends one listing owned by the authenticated dealer for preview or editing. Direct named calls: <code>sendSuccess</code> , <code>getDealerListing</code> , <code>String</code> .
<code>getMyListingStats</code> (line 25)	<code>request</code> , <code>response</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>sendSuccess</code> , <code>getDealerListingStats</code> .
<code>updateListing</code> (line 30)	<code>request</code> , <code>response</code>	Updates editable fields on an owned listing. Direct named calls: <code>sendSuccess</code> , <code>updateDealerListing</code> , <code>String</code> .
<code>updateListingStatus</code> (line 35)	<code>request</code> , <code>response</code>	Moves an owned listing through its allowed lifecycle. Direct named calls: <code>sendSuccess</code> , <code>changeDealerListingStatus</code> , <code>String</code> .
<code>bulkUpdateListings</code> (line 40)	<code>request</code> , <code>response</code>	Publishes, marks sold, archives, re-confirms, re-prices, or deletes many owned listings at once. Direct named calls: <code>sendSuccess</code> , <code>applyBulkListingAction</code> .
<code>deleteListing</code> (line 45)	<code>request</code> , <code>response</code>	Permanently deletes an owned listing. Direct named calls: <code>deleteDealerListing</code> , <code>String</code> , <code>sendSuccess</code> .

apps/backend/src/modules/marketplace/listing.model.ts

Database shapes, Mongoose models, and indexes. Models describe stored documents; they do not open the database connection.

Exported declarations: `ListingImage`, `Listing`, `ListingDocument`, `ListingModel`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/modules/marketplace/listing.repository.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
<code>carListing</code> (line 20)	overrides	No leading explanation comment; read the linked implementation. Direct named calls: <code>normalizeRegistrationNumber</code> .

Declared test groups and cases:

- describe: listing repository — registration duplicate detection (line 8)
- it: finds an active listing sharing a normalized registration number (line 28)
- it: does not treat an archived listing as a blocking duplicate (line 34)
- it: excludes the listing being edited from its own duplicate check (line 40)
- it: does not match a different registration number (line 46)
- it: rejects a second draft/active listing for the same registration number at the DB level (line 52)
- it: allows relisting a registration number once the prior listing is archived (line 63)

apps/backend/src/modules/marketplace/listing.repository.ts

Persistence operations: reads or writes stored records for the owning feature.

Exported declarations: `ListingRecord`, `createListingRecord`, `findActiveListingByRegistration`, `staleListingFilter`, `listDealerListings`, `countOpenDealerListings`, `countDealerListings`, `updateOwnedListing`, `findOwnedListing`, `deleteOwnedListing`, `transitionOwnedListingStatus`, `addOwnedListingImage`, `removeOwnedListingImage`, `restoreOwnedListingImage`, `reorderOwnedListingImages`, `BulkListingScope`, `countListingsInScope`, `updateListingsInScope`, `deleteArchivedListingsInScope`.

Function/component	Parameters	Explanation and direct calls
<code>createListingRecord</code> (line 7)	input	Inserts a dealer-owned listing document. Direct named calls: <code>ListingModel.create</code> .
<code>findActiveListingByRegistration</code> (line 13)	normalizedRegistrationNumber, excludeListingId	Finds a currently listed (draft/active) vehicle sharing the given normalized registration number, excluding one listing (used when editing/reactivating that same listing). Direct named calls: <code>ListingModel.findOne</code> .
<code>staleListingFilter</code> (line 23)	dealerId, cutoff	Active listings the dealer has not confirmed since cutoff. Listings saved before <code>lastConfirmedAt</code> existed fall back to their last update time.

Function/component	Parameters	Explanation and direct calls
<code>listDealerListings</code> (line 28)	<code>dealerId</code> , <code>page</code> , <code>limit</code> , <code>options</code>	Returns all statuses of listings owned by one dealer. Direct named calls: <code>staleListingFilter</code> , <code>options.search.replace</code> , <code>Promise.all</code> , <code>ListingModel.find</code> , <code>ListingModel.countDocuments</code> .
<code>countOpenDealerListings</code> (line 44)	<code>dealerId</code>	Listings that count toward the per-dealer limit: drafts and active listings (sold and archived do not). Direct named calls: <code>ListingModel.countDocuments</code> .
<code>countDealerListings</code> (line 48)	<code>dealerId</code> , <code>staleBefore</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>Promise.all</code> , <code>ListingModel.countDocuments</code> , <code>staleListingFilter</code> .
<code>updateOwnedListing</code> (line 58)	<code>listingId</code> , <code>dealerId</code> , <code>update</code> , <code>unsetDescription</code> , <code>unsetEmbedding</code>	Updates a listing only when it belongs to the authenticated dealer. Direct named calls: <code>ListingModel.findOneAndUpdate</code> , <code>Object.keys</code> .
<code>findOwnedListing</code> (line 68)	<code>listingId</code> , <code>dealerId</code>	Loads a listing scoped to its owner for business-rule checks. Direct named calls: <code>ListingModel.findOne</code> .
<code>deleteOwnedListing</code> (line 73)	<code>listingId</code> , <code>dealerId</code>	Permanently removes a listing only when it belongs to the authenticated dealer. Direct named calls: <code>ListingModel.findOneAndDelete</code> .
<code>transitionOwnedListingStatus</code> (line 78)	<code>listingId</code> , <code>dealerId</code> , <code>currentStatus</code> , <code>update</code>	Applies a status change only if the expected current status still matches. Direct named calls: <code>ListingModel.findOneAndUpdate</code> .
<code>addOwnedListingImage</code> (line 83)	<code>listingId</code> , <code>dealerId</code> , <code>image</code> , <code>maximum</code>	Adds image metadata only when the listing is owned and below its image limit. Direct named calls: <code>ListingModel.findOneAndUpdate</code> .
<code>removeOwnedListingImage</code> (line 92)	<code>listingId</code> , <code>dealerId</code> , <code>imageKey</code>	Removes image metadata from a dealer-owned listing. Direct named calls: <code>ListingModel.findOneAndUpdate</code> .
<code>restoreOwnedListingImage</code> (line 101)	<code>listingId</code> , <code>dealerId</code> , <code>image</code>	Restores image metadata when deleting the storage object fails. Direct named calls: <code>ListingModel.findOneAndUpdate</code> .
<code>reorderOwnedListingImages</code> (line 106)	<code>listingId</code> , <code>dealerId</code> , <code>images</code>	Replaces image metadata with a validated dealer-defined order. Direct named calls: <code>ListingModel.findOneAndUpdate</code> .
<code>countListingsInScope</code> (line 113)	<code>scope</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>ListingModel.countDocuments</code> .
<code>updateListingsInScope</code> (line 119)	<code>scope</code> , <code>eligible</code> , <code>update</code>	Applies one update to every listing in scope whose status allows it. <code>update</code> may be an aggregation pipeline, so values can depend on each listing (its price, its first publish date). Direct named calls: <code>ListingModel.updateMany</code> , <code>Array.isArray</code> .
<code>deleteArchivedListingsInScope</code> (line 127)	<code>scope</code>	Deletes the archived listings in scope and returns the image keys they held, for storage cleanup. Direct named calls: <code>ListingModel.find</code> , <code>ListingModel.deleteMany</code> , <code>listings.map</code> , <code>listings.flatMap</code> .

apps/backend/src/modules/marketplace/listing.routes.ts

Route definitions: inspect the listed registrations and guards. Express routes are mounted by backend `app.ts`; frontend route arrays are metadata, while `app/App.tsx` mounts the actual React routes.

Exported declarations: `listingRouter`, `listingImageRouter`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

HTTP registrations (relative to the mount in `app.ts`):

- `listingRouter.get("/mine")` — line 19
- `listingRouter.post("/mine/bulk")` — line 20
- `listingRouter.get("/mine/stats")` — line 21
- `listingRouter.get("/mine/:listingId")` — line 22
- `listingRouter.post("/")` — line 23
- `listingRouter.patch("/:listingId")` — line 24
- `listingRouter.patch("/:listingId/status")` — line 25
- `listingRouter.delete("/:listingId")` — line 26
- `listingRouter.post("/:listingId/images")` — line 27
- `listingRouter.delete("/:listingId/images/:imageKey")` — line 29
- `listingRouter.patch("/:listingId/images/reorder")` — line 31
- `listingImageRouter.get("/thumbs/:imageKey")` — line 34
- `listingImageRouter.get("/:imageKey")` — line 35

apps/backend/src/modules/marketplace/listing.service.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
<code>createTestListing</code> (line 37)	overrides	No leading explanation comment; read the linked implementation. Direct named calls: <code>createDealerListing</code> .

Declared test groups and cases:

- `describe`: listing service — `updateDealerListing` (line 26)
- `it`: clears the description when updated to null (line 50)
- `it`: still updates the description to a new value (line 59)

apps/backend/src/modules/marketplace/listing.service.ts

Business operations and orchestration; see each function and its direct calls below.

Exported declarations: `serializeListing`, `assertDealerListingCapacity`, `createDealerListing`, `staleCutoff`, `getDealerListings`, `getDealerListingStats`, `applyBulkListingAction`, `getDealerListing`, `updateDealerListing`, `deleteDealerListing`, `changeDealerListingStatus`.

Function/component	Parameters	Explanation and direct calls
<code>serializeListing</code> (line 32)	listing	Converts a listing record into the shared API DTO. Direct named calls: <code>listing._id.toString</code> , <code>listing.dealerId.toString</code> , <code>listing.images.slice</code> , <code>listing.publishedAt?.toISOString</code> .
<code>assertRegistrationNotActively Listed</code> (line 46)	normalizedRegistrationNumber, excludeListingId	Rejects a registration number that already belongs to a currently listed (draft/active) vehicle. Archived/sold listings don't block — the same physical vehicle may legitimately be relisted. Direct named calls: <code>findActiveListingByRegistration</code> .

Function/component	Parameters	Explanation and direct calls
rethrowAsRegistrationConflict (line 55)	error	Rethrows a MongoDB duplicate-key error as the same conflict <code>assertRegistrationNotActivelyListed</code> throws, and anything else unchanged. The partial unique index on <code>normalizedRegistrationNumber</code> is the actual race-safety backstop; the pre-check above is just a cheap early rejection for the common non-racing case, so a concurrent request that slips past it still gets the same 409.
assertDealerListingCapacity (line 62)	dealerId	Refuses new listings once the dealer holds <code>MAX_LISTINGS_PER_DEALER</code> drafts and active listings. Direct named calls: <code>countOpenDealerListings</code> .
createDealerListing (line 68)	dealerId, input	Creates a draft or immediately published listing for a dealer. Direct named calls: <code>assertDealerListingCapacity</code> , <code>normalizeRegistrationNumber</code> , <code>assertRegistrationNotActivelyListed</code> , <code>generateSearchEmbedding</code> , <code>composeListingSearchText</code> , <code>createListingRecord</code> , <code>input.registrationNumber.trim</code> , <code>serializeListing</code> , <code>document.toObject</code> , <code>rethrowAsRegistrationConflict</code> .
staleCutoff (line 87)	now	Active listings last confirmed before this moment are stale stock. Direct named calls: <code>now.getTime</code> .
getDealerListings (line 90)	dealerId, query	Returns every listing owned by the authenticated dealer (optionally only stale stock, or one upload's listings). Direct named calls: <code>listDealerListings</code> , <code>staleCutoff</code> , <code>documents.map</code> , <code>buildPaginationMeta</code> .
getDealerListingStats (line 96)	dealerId	No leading explanation comment; read the linked implementation. Direct named calls: <code>countDealerListings</code> , <code>staleCutoff</code> .
applyBulkListingAction (line 103)	dealerId, input	Applies one action to many of the dealer's own listings at once. Each listing is only changed when the action applies to its current status (the same lifecycle rules as one-by-one changes); the others are counted as skipped. Another dealer's listing IDs simply never match. Direct named calls: <code>countListingsInScope</code> , <code>updateListingsInScope</code> , <code>deleteArchivedListingsInScope</code> , <code>Promise.allSettled</code> , <code>imageKeys.map</code> , <code>console.error</code> .
getDealerListing (line 140)	listingId, dealerId	Returns one listing for its owner, including drafts and archived records for dealer preview/editing. Direct named calls: <code>findOwnedListing</code> , <code>serializeListing</code> , <code>listing.toObject</code> .
updateDealerListing (line 147)	listingId, dealerId, input	Updates editable fields while preserving listing ownership, category, and status. Direct named calls: <code>findOwnedListing</code> , <code>Object.entries</code> , <code>normalizeRegistrationNumber</code> , <code>assertRegistrationNotActivelyListed</code> , <code>input.registrationNumber.trim</code> , <code>validateAttributesForCategory</code> , <code>result.error.issues.map</code> , <code>searchableFields.some</code> , <code>existing.toObject</code> , <code>generateSearchEmbedding</code> , <code>composeListingSearchText</code> , <code>updateOwnedListing</code> , <code>serializeListing</code> , <code>listing.toObject</code> .
deleteDealerListing (line 191)	listingId, dealerId	Permanently removes an owned listing and best-effort cleans up its stored images. Direct named calls: <code>deleteOwnedListing</code> , <code>Promise.allSettled</code> , <code>listing.images.map</code> , <code>console.error</code> .
changeDealerListingStatus (line 203)	listingId, dealerId, nextStatus	Enforces the listing lifecycle before applying an atomic status transition. Direct named calls: <code>findOwnedListing</code> , <code>assertRegistrationNotActivelyListed</code> , <code>transitionOwnedListingStatus</code> , <code>serializeListing</code> , <code>listing.toObject</code> , <code>rethrowAsRegistrationConflict</code> .

apps/backend/src/modules/marketplace/listing.validation.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

Declared test groups and cases:

- describe: registration number normalization (line 5)
- it: normalizes hyphens, spaces, and casing to the same identity (line 6)
- describe: listing request validation (line 15)
- it: applies safe pagination defaults (line 16)
- it: rejects pagination outside its allowed range (line 20)
- it: parses location and mileage filters and rejects inverted ranges (line 25)
- it: requires at least one field when updating a listing (line 31)
- it: only accepts statuses controlled by the dealer workflow (line 36)
- it: requires at least one image key when reordering (line 41)
- describe: create listing category validation (line 47)
- it: accepts a petrol car with engine capacity (line 48)
- it: rejects an electric car missing battery capacity/range (line 53)
- it: rejects a petrol/diesel car missing engine capacity (line 58)
- it: rejects a listing whose attributes belong to the wrong category (line 63)
- it: requires a registration number (line 68)

apps/backend/src/modules/marketplace/listing.validation.ts

Validation rules and related types; these reject or normalize unsupported input.

Exported declarations: `listListingsQuerySchema`, `listMyListingsQuerySchema`, `bulkListingActionBodySchema`, `createListingBodySchema`, `listingIdParamsSchema`, `updateListingBodySchema`, `updateListingStatusBodySchema`, `listingImageKeyParamsSchema`, `publicListingImageKeyParamsSchema`, `listingImageMetadataSchema`, `reorderListingImagesBodySchema`, `validateAttributesForCategory`, `ListListingsQuery`, `ListMyListingsQuery`, `BulkListingActionBody`, `CreateListingBody`, `UpdateListingBody`.

Function/component	Parameters	Explanation and direct calls
<code>rangesInOrder</code> (line 46)	<code>query</code> , <code>context</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>context.addIssue</code> .
<code>validateAttributesForCategory</code> (line 120)	<code>category</code> , <code>mergedAttributes</code>	No leading explanation comment; read the linked implementation.

apps/backend/src/modules/marketplace/listingImage.controller.ts

HTTP boundary: reads request values, calls services, and sends responses.

Exported declarations: `addListingImage`, `deleteListingImage`, `reorderListingImages`, `getListingImageFile`, `getListingThumbFile`.

Function/component	Parameters	Explanation and direct calls
<code>addListingImage</code> (line 11)	<code>request</code> , <code>response</code>	Uploads one image to an authenticated dealer's listing. Direct named calls: <code>addDealerListingImage</code> , <code>String</code> , <code>sendSuccess</code> .
<code>deleteListingImage</code> (line 17)	<code>request</code> , <code>response</code>	Deletes one owned listing image from MongoDB and object storage. Direct named calls: <code>sendSuccess</code> , <code>deleteDealerListingImage</code> , <code>String</code> .
<code>reorderListingImages</code> (line 22)	<code>request</code> , <code>response</code>	Saves a complete reordered list of image keys for an owned listing. Direct named calls: <code>sendSuccess</code> , <code>reorderDealerListingImages</code> , <code>String</code> .
<code>getListingImageFile</code> (line 29)	<code>request</code> , <code>response</code>	Serves a listing photo from private storage through a public, read-only endpoint. Used locally and as the fallback when no CDN is configured. The file is streamed straight from storage (not held in memory), and a browser that already has it gets 304 Not Modified with no body. Direct named calls: <code>sendListingImage</code> .
<code>getListingThumbFile</code> (line 32)	<code>request</code> , <code>response</code>	Same as above for the small copy used by cards and phones. Direct named calls: <code>sendListingImage</code> .
<code>sendListingImage</code> (line 34)	<code>request</code> , <code>response</code> , <code>variant</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>getListingImageObject</code> , <code>String</code> , <code>request.header</code> , <code>response.status</code> , <code>response.setHeader</code> , <code>pipeline</code> .

apps/backend/src/modules/marketplace/listingImage.middleware.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `uploadSingleListingImage`.

Function/component	Parameters	Explanation and direct calls
<code>fileFilter</code> (line 10)	<code>_request</code> , <code>file</code> , <code>callback</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>callback</code> , <code>storageConfig.allowedImageTypes.has</code> .
<code>uploadSingleListingImage</code> (line 16)	<code>request</code> , <code>response</code> , <code>next</code>	Parses one in-memory image and converts Multer failures into validation errors. Direct named calls: <code>imageUpload</code> .

apps/backend/src/modules/marketplace/listingImage.service.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

Declared test groups and cases:

- describe: listing image signatures (line 4)
- it: accepts supported image headers (line 5)
- it: rejects spoofed and unsupported files (line 11)

apps/backend/src/modules/marketplace/listingImage.service.ts

Business operations and orchestration; see each function and its direct calls below.

Exported declarations: `addDealerListingImage`, `deleteDealerListingImage`, `reorderDealerListingImages`.

Function/component	Parameters	Explanation and direct calls
<code>atPosition</code> (line 15)	<code>image</code> , <code>order</code>	Copies one image's stored fields with a new position (keeps the small copy's URL).
<code>addDealerListingImage</code> (line 20)	<code>listingId</code> , <code>dealerId</code> , <code>file</code> , <code>alt</code>	Uploads an image and atomically attaches its metadata to an owned listing. Direct named calls: <code>hasValidImageSignature</code> , <code>findOwnedListing</code> , <code>reencodeImage</code> , <code>randomUUID</code> , <code>Promise.all</code> , <code>uploadListingImage</code> , <code>deleteListingImageObject</code> , <code>addOwnedListingImage</code> , <code>serializeListing</code> , <code>updated.toObject</code> .
<code>deleteDealerListingImage</code> (line 56)	<code>listingId</code> , <code>dealerId</code> , <code>imageKey</code>	Deletes image metadata and storage, restoring metadata if storage fails. Direct named calls: <code>findOwnedListing</code> , <code>listing.images.find</code> , <code>removeOwnedListingImage</code> , <code>deleteListingImageObject</code> , <code>restoreOwnedListingImage</code> , <code>updated.images.map</code> , <code>reorderOwnedListingImages</code> , <code>serializeListing</code> .
<code>reorderDealerListingImages</code> (line 71)	<code>listingId</code> , <code>dealerId</code> , <code>imageKeys</code>	Validates a complete key set before saving a new image order. Direct named calls: <code>findOwnedListing</code> , <code>listing.images.map</code> , <code>imageKeys.some</code> , <code>imageKeys.map</code> , <code>reorderOwnedListingImages</code> , <code>serializeListing</code> , <code>updated.toObject</code> .

`apps/backend/src/modules/marketplace/listingImage.signature.ts`

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `hasValidImageSignature`.

Function/component	Parameters	Explanation and direct calls
<code>hasValidImageSignature</code> (line 2)	<code>file</code>	Confirms that file bytes match the declared supported image type. Direct named calls: <code>bytes.subarray</code> , <code>Buffer.from</code> , <code>bytes.toString</code> .

`apps/backend/src/modules/marketplace/listingImage.storage.ts`

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `listingImageObjectKey`, `listingThumbObjectKey`, `ListingImageVariant`, `uploadListingImage`, `ListingImageObject`, `getListingImageObject`, `deleteListingImageObject`.

Function/component	Parameters	Explanation and direct calls
<code>listingImageObjectKey</code> (line 9)	<code>key</code>	Listing photos live under their own prefix, separate from private objects (dealer documents, inventory CSVs and zips). That lets a CDN be granted read access to this prefix only. The public image key (used in URLs and on the listing) stays the bare file name.
<code>listingThumbObjectKey</code> (line 11)	<code>key</code>	The small copy of the same photo (same file name, in the thumbs/ folder).
<code>uploadListingImage</code> (line 17)	<code>key</code> , <code>body</code> , <code>contentType</code> , <code>variant</code>	Uploads one re-encoded image and returns its public URL. Behind a CDN, <code>S3_PUBLIC_URL</code> is the CDN address (origin path = the prefix); locally it is this API's <code>/listing-images</code> route. Direct named calls: <code>storageClient.send</code> , <code>listingThumbObjectKey</code> , <code>listingImageObjectKey</code> , <code>encodeURIComponent</code> .
<code>isNoSuchKey</code> (line 28)	<code>error</code>	No leading explanation comment; read the linked implementation.
<code>isNotModified</code> (line 29)	<code>error</code>	No leading explanation comment; read the linked implementation.

Function/component	Parameters	Explanation and direct calls
getListingImageObject (line 38)	key, ifNoneMatch, variant	Opens one listing image as a stream (never buffered in full). Photos stored before the prefix existed are read from their old location until the migration script has moved them. Passing the browser's If-None-Match lets storage answer "not modified" without sending bytes. Direct named calls: listingThumbObjectKey, listingImageObjectKey, storageClient.send, isNotModified, isNoSuchKey, Object.assign.
deleteListingImageObject (line 62)	key	Removes one listing image and its small copy, wherever stored (current prefix or the pre-migration location). Direct named calls: Promise.all, listingImageObjectKey, listingThumbObjectKey.

apps/backend/src/modules/marketplace/publicVisibility.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: getHiddenDealerIds, invalidateHiddenDealerIds, publicDealerFilter.

Function/component	Parameters	Explanation and direct calls
getHiddenDealerIds (line 11)	None	No leading explanation comment; read the linked implementation. Direct named calls: Date.now, AuthUserModel.find, suspended.map.
invalidateHiddenDealerIds (line 18)	None	No leading explanation comment; read the linked implementation.
publicDealerFilter (line 21)	None	Filter fragment for listing queries: excludes listings of suspended dealers. Direct named calls: getHiddenDealerIds.

apps/backend/src/modules/notifications/index.ts

Module entry point or re-export barrel; inspect exported symbols/import targets.

Exported declarations: notificationsModule.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/modules/notifications/notification.controller.ts

HTTP boundary: reads request values, calls services, and sends responses.

Exported declarations: getMyNotifications, getMyUnreadNotificationCount, patchMyNotificationRead, patchMyNotificationsReadAll.

Function/component	Parameters	Explanation and direct calls
getMyNotifications (line 10)	request, response	Sends the current user's own notification inbox, newest first. Direct named calls: getNotificationsForUser, sendSuccess.
getMyUnreadNotificationCount (line 17)	request, response	Sends the badge count used to render the unread bell indicator. Direct named calls: sendSuccess, getUnreadNotificationCount.
patchMyNotificationRead (line 22)	request, response	Marks one notification the requesting user owns as read. Direct named calls: markOneNotificationRead, String, sendSuccess.

Function/component	Parameters	Explanation and direct calls
patchMyNotificationsReadAll (line 29)	request, response	Marks every unread notification for the requesting user as read. Direct named calls: markAllNotificationsAsRead, sendSuccess.

apps/backend/src/modules/notifications/notification.model.ts

Database shapes, Mongoose models, and indexes. Models describe stored documents; they do not open the database connection.

Exported declarations: Notification, NotificationDocument, NotificationModel.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/modules/notifications/notification.repository.ts

Persistence operations: reads or writes stored records for the owning feature.

Exported declarations: createNotification, setNotificationEmailStatus, listNotificationsForUser, countUnreadNotifications, markNotificationRead, markAllNotificationsRead, findAdminUserIds, findUserContact.

Function/component	Parameters	Explanation and direct calls
createNotification (line 6)	input	Creates a notification, marking the email leg 'pending' only when the email channel applies. Direct named calls: NotificationModel.create, input.channels.includes.
setNotificationEmailStatus (line 10)	notificationId, emailStatus	No leading explanation comment; read the linked implementation. Direct named calls: NotificationModel.updateOne.
listNotificationsForUser (line 14)	userId, page, limit	No leading explanation comment; read the linked implementation. Direct named calls: Promise.all, NotificationModel.find, NotificationModel.countDocuments.
countUnreadNotifications (line 22)	userId	No leading explanation comment; read the linked implementation. Direct named calls: NotificationModel.countDocuments.
markNotificationRead (line 26)	notificationId, userId	No leading explanation comment; read the linked implementation. Direct named calls: NotificationModel.findOneAndUpdate.
markAllNotificationsRead (line 30)	userId	No leading explanation comment; read the linked implementation. Direct named calls: NotificationModel.updateMany.
findAdminUserIds (line 36)	None	Cross-module read: admin-only triggers (new dealer application, high rejection rate) fan out to every active administrator rather than one fixed recipient. Direct named calls: AuthUserModel.find, admins.map.
findUserContact (line 41)	userId	No leading explanation comment; read the linked implementation. Direct named calls: AuthUserModel.findById.

apps/backend/src/modules/notifications/notification.routes.ts

Route definitions: inspect the listed registrations and guards. Express routes are mounted by backend app.ts; frontend route arrays are metadata, while app/App.tsx mounts the actual React routes.

Exported declarations: notificationRouter.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

HTTP registrations (relative to the mount in `app.ts`):

- `notificationRouter.get("/")` — line 14
- `notificationRouter.get("/unread-count")` — line 15
- `notificationRouter.patch("/:notificationId/read")` — line 16
- `notificationRouter.patch("/read-all")` — line 17

apps/backend/src/modules/notifications/notification.service.ts

Business operations and orchestration; see each function and its direct calls below.

Exported declarations: `notifyUploadJobClean`, `notifyImageProcessingClean`, `notifyDealerApplicationSubmitted`, `notifyUploadHighRejectionRate`, `notifyUploadJobFailed`, `notifyUploadJobCompletedWithErrors`, `notifyImageProcessingFailed`, `notifyImageProcessingCompletedWithErrors`, `notifyDealerApplicationDecision`, `notifyListingRemoved`, `notifyAccountSuspended`, `getNotificationsForUser`, `getUnreadNotificationCount`, `markOneNotificationRead`, `markAllNotificationsAsRead`.

Function/component	Parameters	Explanation and direct calls
<code>serializeNotification</code> (line 15)	<code>record</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>String</code> .
<code>notify</code> (line 34)	<code>userId</code> , <code>type</code> , <code>title</code> , <code>message</code> , <code>channels</code> , <code>details</code>	Records one notification. Emails are only queued here (<code>emailStatus</code> 'pending'); the worker's outbox job sends them with retries. A failure to record a notification is logged, never thrown, so it cannot make an already committed action (approval, suspension, removal) look failed. Direct named calls: <code>createNotification</code> , <code>logger.error</code> , <code>String</code> .
<code>notifyMany</code> (line 42)	<code>userIds</code> , <code>type</code> , <code>title</code> , <code>message</code> , <code>channels</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>Promise.all</code> , <code>userIds.map</code> .
<code>notifyUploadJobClean</code> (line 48)	<code>dealerUserId</code>	-- In-app only ----- Direct named calls: <code>notify</code> .
<code>notifyImageProcessingClean</code> (line 52)	<code>dealerUserId</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>notify</code> .
<code>notifyDealerApplicationSubmitted</code> (line 56)	<code>businessName</code> , <code>resubmitted</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>findAdminUserIds</code> , <code>notifyMany</code> .
<code>notifyUploadHighRejectionRate</code> (line 63)	<code>uploadJobId</code> , <code>rejectionRate</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>findAdminUserIds</code> , <code>notifyMany</code> , <code>Math.round</code> .
<code>notifyUploadJobFailed</code> (line 70)	<code>dealerUserId</code> , <code>reason</code>	-- In-app + email ----- Direct named calls: <code>notify</code> .
<code>notifyUploadJobCompletedWithErrors</code> (line 74)	<code>dealerUserId</code> , <code>rejectedRecords</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>notify</code> .
<code>notifyImageProcessingFailed</code> (line 78)	<code>dealerUserId</code> , <code>reason</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>notify</code> .
<code>notifyImageProcessingCompletedWithErrors</code> (line 82)	<code>dealerUserId</code> , <code>unmatchedFolders</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>notify</code> .
<code>notifyDealerApplicationDecision</code> (line 86)	<code>dealerUserId</code> , <code>decision</code> , <code>reason</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>notify</code> .

Function/component	Parameters	Explanation and direct calls
notifyListingRemoved (line 92)	dealerUserId, listingTitle, details	No leading explanation comment; read the linked implementation. Direct named calls: notify.
notifyAccountSuspended (line 98)	userId	-- Email-primary ----- Direct named calls: notify.
getNotificationsForUser (line 104)	userId, page, limit	-- Reads for the notifications API ----- Direct named calls: listNotificationsForUser, result.documents.map, buildPaginationMeta.
getUnreadNotificationCount (line 109)	userId	No leading explanation comment; read the linked implementation. Direct named calls: countUnreadNotifications.
markOneNotificationRead (line 113)	notificationId, userId	No leading explanation comment; read the linked implementation. Direct named calls: markNotificationRead.
markAllNotificationsAsRead (line 117)	userId	No leading explanation comment; read the linked implementation. Direct named calls: markAllNotificationsRead.

apps/backend/src/modules/notifications/notification.validation.ts

Validation rules and related types; these reject or normalize unsupported input.

Exported declarations: listNotificationsQuerySchema, ListNotificationsQuery, notificationIdParamsSchema.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/modules/search/index.ts

Module entry point or re-export barrel; inspect exported symbols/import targets.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/modules/search/search.controller.ts

HTTP boundary: reads request values, calls services, and sends responses.

Exported declarations: getSearchResults.

Function/component	Parameters	Explanation and direct calls
getSearchResults (line 6)	request, response	No leading explanation comment; read the linked implementation. Direct named calls: sendSuccess, searchListings.

apps/backend/src/modules/search/search.embedding.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: generateSearchEmbedding.

Function/component	Parameters	Explanation and direct calls
generateSearchEmbedding (line 5)	text	Query and listing vectors always use the same configured model and dimensions. Direct named calls: <code>text.trim</code> , <code>createLocalSearchEmbedding</code> , <code>fetch</code> , <code>encodeURIComponent</code> , <code>JSON.stringify</code> , <code>AbortSignal.timeout</code> , <code>normalizeEmbeddingResponse</code> , <code>response.json</code> .

apps/backend/src/modules/search/search.queryAnalyzer.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

Declared test groups and cases:

- describe: natural-language search analyzer (line 6)
- it: extracts the SRS-style structured query (line 7)
- it: corrects common make and model typos (line 13)
- it: distinguishes mileage from price and extracts year ranges (line 19)
- describe: search request validation (line 24)
- it: bounds query and pagination work (line 25)
- describe: embedding contract (line 33)
- it: keeps local and provider vectors normalized (line 34)

apps/backend/src/modules/search/search.queryAnalyzer.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `AnalyzedSearchQuery`, `analyzeSearchQuery`.

Function/component	Parameters	Explanation and direct calls
editDistance (line 17)	left, right	No leading explanation comment; read the linked implementation. Direct named calls: <code>Array.from</code> , <code>Math.min</code> .
fuzzyVocabularyMatch (line 30)	token, vocabulary	No leading explanation comment; read the linked implementation. Direct named calls: <code>vocabulary.map</code> .
numericAmount (line 36)	value, unit	No leading explanation comment; read the linked implementation. Direct named calls: <code>Number</code> , <code>value.replace</code> .
analyzeSearchQuery (line 45)	rawQuery	Converts a bounded natural-language query into structured constraints plus residual intent. Direct named calls: <code>rawQuery.toLocaleLowerCase</code> , <code>consume</code> , <code>working.split</code> , <code>makes.includes</code> , <code>fuzzyVocabularyMatch</code> , <code>models.includes</code> , <code>make.replace</code> , <code>model.replace</code> , <code>tokens.filter</code> .
analyzeSearchQuery.consume (line 49)	pattern, action	No leading explanation comment; read the linked implementation. Direct named calls: <code>working.match</code> , <code>action</code> , <code>working.replace</code> .

apps/backend/src/modules/search/search.repository.ts

Persistence operations: reads or writes stored records for the owning feature.

Exported declarations: SearchFilters, escapeSearchRegex, buildListingFilter, buildListingSort, listStructuredListings, listSearchCandidates, listVectorCandidates.

Function/component	Parameters	Explanation and direct calls
escapeSearchRegex (line 28)	value	Buyer input is always escaped before it enters a regular expression. Direct named calls: <code>value.replace</code> .
buildListingFilter (line 31)	options	Produces the single structured-filter definition shared by browse and intelligent search. Direct named calls: <code>escapeSearchRegex</code> .
buildListingSort (line 51)	sortBy	No leading explanation comment; read the linked implementation.
listStructuredListings (line 60)	options	Keeps result-set work bounded through validated pagination and indexed filters. Direct named calls: <code>buildListingFilter</code> , <code>publicDealerFilter</code> , <code>Promise.all</code> , <code>ListingModel.find</code> , <code>buildListingSort</code> , <code>ListingModel.countDocuments</code> .
listSearchCandidates (line 70)	options, candidateLimit	Retrieves a bounded lexical pool for application-side hybrid scoring. Direct named calls: <code>buildListingFilter</code> , <code>publicDealerFilter</code> , <code>Promise.all</code> , <code>ListingModel.find</code> , <code>ListingModel.countDocuments</code> .
buildVectorFilter (line 79)	options	No leading explanation comment; read the linked implementation.
listVectorCandidates (line 93)	embedding, options, candidateLimit	Atlas Vector Search is optional at runtime; callers merge these candidates with lexical ones. Direct named calls: <code>publicDealerFilter</code> , <code>ListingModel.aggregate</code> , <code>Math.min</code> , <code>buildVectorFilter</code> , <code>Object.keys</code> .

apps/backend/src/modules/search/search.routes.ts

Route definitions: inspect the listed registrations and guards. Express routes are mounted by backend app.ts; frontend route arrays are metadata, while app/App.tsx mounts the actual React routes.

Exported declarations: searchRouter.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

HTTP registrations (relative to the mount in app.ts):

- `searchRouter.get("/")` — line 9

apps/backend/src/modules/search/search.service.ts

Business operations and orchestration; see each function and its direct calls below.

Exported declarations: searchListings.

Function/component	Parameters	Explanation and direct calls
lexicalScore (line 10)	query, listing	No leading explanation comment; read the linked implementation. Direct named calls: <code>query.toLocaleLowerCase</code> , <code>composeListingSearchText</code> , <code>tokens.filter</code> , <code>content.includes</code> , <code>Math.min</code> .
asRecordId (line 19)	value	No leading explanation comment; read the linked implementation. Direct named calls: <code>String</code> .

Function/component	Parameters	Explanation and direct calls
<code>matchesFullFilter</code> (line 21)	document, filters	No leading explanation comment; read the linked implementation. Direct named calls: <code>equals</code> , <code>includes</code> , <code>within</code> .
<code>matchesFullFilter.text</code> (line 22)	value	No leading explanation comment; read the linked implementation. Direct named calls: <code>String</code> .
<code>matchesFullFilter.includes</code> (line 24)	actual, expected	No leading explanation comment; read the linked implementation. Direct named calls: <code>text</code> .
<code>matchesFullFilter.equals</code> (line 25)	actual, expected	No leading explanation comment; read the linked implementation. Direct named calls: <code>text</code> .
<code>matchesFullFilter.within</code> (line 26)	actual, minimum, maximum	No leading explanation comment; read the linked implementation. Direct named calls: <code>Number</code> , <code>Number.isFinite</code> .
<code>searchListings</code> (line 39)	query	Executes structured-only search directly, or bounded hybrid ranking with safe degradation. Direct named calls: <code>performance.now</code> , <code>analyzeSearchQuery</code> , <code>Object.values</code> , <code>listStructuredListings</code> , <code>structured.documents.map</code> , <code>buildPaginationMeta</code> , <code>Math.round</code> , <code>listSearchCandidates</code> , <code>Promise.all</code> , <code>merged.set</code> , <code>asRecordId</code> , <code>matchesFullFilter</code> , <code>merged.get</code> , <code>merged.values</code> , <code>ranked.slice</code> , <code>Math.min</code> , <code>page.map</code> .

apps/backend/src/modules/search/search.validation.ts

Validation rules and related types; these reject or normalize unsupported input.

Exported declarations: `searchQuerySchema`, `SearchQuery`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/routes.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `routes`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/scripts/listingImageMigration.db.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
<code>listing</code> (line 18)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>ListingModel.create</code> .

Declared test groups and cases:

- describe: listing image URL updates (real MongoDB) (line 10)
- it: sets the small-copy URL only on the listed photos, even though their keys contain dots (line 27)
- it: rewrites photo URLs and keeps every other image field (line 37)

apps/backend/src/scripts/listingImageMigration.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
<code>fakeWorld</code> (line 6)	objects, listings	In-memory bucket and listings standing in for S3 and MongoDB. Direct named calls: <code>Object.entries</code> .
<code>fakeWorld.listings</code> (line 11)	None	No leading explanation comment; read the linked implementation.
<code>fakeWorld.exists</code> (line 12)	key	No leading explanation comment; read the linked implementation. Direct named calls: <code>bucket.has</code> .
<code>fakeWorld.read</code> (line 13)	key	No leading explanation comment; read the linked implementation. Direct named calls: <code>bucket.get</code> .
<code>fakeWorld.write</code> (line 14)	key, body	No leading explanation comment; read the linked implementation. Direct named calls: <code>bucket.set</code> .
<code>fakeWorld.remove</code> (line 15)	key	No leading explanation comment; read the linked implementation. Direct named calls: <code>bucket.delete</code> .
<code>fakeWorld.setImageUrls</code> (line 16)	id, urls	No leading explanation comment; read the linked implementation. Direct named calls: <code>urlUpdates.push</code> .
<code>fakeWorld.setThumbUrls</code> (line 17)	id, urls	No leading explanation comment; read the linked implementation. Direct named calls: <code>thumbUpdates.push</code> .
<code>fakeWorld.log</code> (line 18)	None	No leading explanation comment; read the linked implementation.
<code>phonePhoto</code> (line 23)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>sharp</code> .
<code>listings</code> (line 78)	None	No leading explanation comment; read the linked implementation.

Declared test groups and cases:

- describe: listing image migration (line 26)
- it: moves old photos under listing-images/ as metadata-free WebP and removes the originals (line 27)
- it: is safe to run again: already migrated photos are skipped (line 39)
- it: changes nothing in a dry run (line 48)
- it: points image URLs at the CDN when asked (line 59)
- it: creates a small copy for photos that have none, at most 800 px, and records its URL (line 67)
- it: reports missing and undecodable objects and leaves them in place (line 82)

apps/backend/src/scripts/listingImageMigration.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `MigrationDeps`, `MigrationOptions`, `MigrationSummary`, `migrateListingImages`, `imageFieldUpdate`.

Function/component	Parameters	Explanation and direct calls
<code>isCleanWebp</code> (line 25)	<code>bytes</code>	True when the stored bytes are already a metadata-free WebP (photos uploaded after re-encoding was introduced), so they can be moved without another lossy re-encode. Direct named calls: <code>sharp</code> .
<code>migrateListingImages</code> (line 35)	<code>deps, options</code>	Moves every listing photo still stored at the bucket root to the listing-images/ prefix, re-encoding it on the way (strips EXIF/GPS, hidden data; caps size). Idempotent: photos already under the prefix are skipped, so the script can be stopped and run again at any time. With <code>publicUrl</code> , also points each stored image URL at that base (e.g. the CDN). Every photo without a small copy (thumbs/) gets one, and its <code>thumbUrl</code> is recorded. Direct named calls: <code>deps.listings</code> , <code>options.publicUrl.replace</code> , <code>image.url.slice</code> , <code>image.url.lastIndexOf</code> , <code>encodeURIComponent</code> , <code>deps.exists</code> , <code>deps.read</code> , <code>deps.log</code> , <code>isCleanWebp</code> , <code>reencodeImage</code> , <code>String</code> , <code>deps.write</code> , <code>deps.remove</code> , <code>Object.keys</code> , <code>deps.setImageUrls</code> , <code>deps.setThumbUrls</code> .
<code>imageFieldUpdate</code> (line 101)	<code>field, values</code>	Update pipeline that sets <code>field</code> (url or <code>thumbUrl</code>) on each of a listing's images whose key is in <code>values</code> , leaving the other images untouched. The values travel as a literal list of {k, v} pairs matched with <code>\$filter</code> : photo keys contain dots ("...jpg"), which MongoDB would otherwise read as field paths and refuse. Direct named calls: <code>Object.entries</code> .

apps/backend/src/scripts/migrateListingImages.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Function/component	Parameters	Explanation and direct calls
<code>notFound</code> (line 20)	<code>error</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>String</code> .
<code>listings</code> (line 25)	<code>None</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>ListingModel.find</code> , <code>String</code> .
<code>exists</code> (line 30)	<code>key</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>storageClient.send</code> , <code>notFound</code> .
<code>read</code> (line 34)	<code>key</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>storageClient.send</code> , <code>Buffer.from</code> , <code>object.Body.transformToByteArray</code> , <code>notFound</code> .
<code>write</code> (line 38)	<code>key, body, contentType</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>storageClient.send</code> .
<code>remove</code> (line 41)	<code>key</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>storageClient.send</code> .
<code>setImageUrls</code> (line 42)	<code>listingId, urls</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>ListingModel.updateOne</code> , <code>imageFieldUpdate</code> .
<code>setThumbUrls</code> (line 43)	<code>listingId, urls</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>ListingModel.updateOne</code> , <code>imageFieldUpdate</code> .
<code>log</code> (line 44)	<code>message</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>console.warn</code> .

apps/backend/src/server.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Function/component	Parameters	Explanation and direct calls
shutdown (line 19)	signal	Stops accepting connections, lets in-flight requests finish, then releases MongoDB and Redis, all within SHUTDOWN_TIMEOUT_MS so the orchestrator never has to kill the process mid-cleanup. Direct named calls: <code>console.log</code> , <code>setTimeout</code> , <code>hardExit.unref</code> , <code>server.closeIdleConnections</code> , <code>Promise.race</code> , <code>drained.then</code> , <code>clearTimeout</code> , <code>console.warn</code> , <code>server.closeAllConnections</code> , <code>Promise.allSettled</code> , <code>disconnectDatabase</code> , <code>closeInventoryQueue</code> , <code>disconnectRedis</code> , <code>results.some</code> , <code>console.error</code> , <code>process.exit</code> .

apps/backend/src/shared/errors/AppError.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `AppError`.

Function/component	Parameters	Explanation and direct calls
constructor (line 2)	<code>statusCode</code> , <code>code</code> , <code>message</code> , <code>fields</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>super</code> .

apps/backend/src/shared/errors/errorCodes.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `errorCodes`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/shared/middleware/errorHandler.ts

Express request middleware: authentication, authorization, validation, limits, or error handling.

Exported declarations: `errorHandler`.

Function/component	Parameters	Explanation and direct calls
<code>errorHandler</code> (line 8)	<code>error</code> , <code>request</code> , <code>response</code> , <code>_next</code>	Converts application and unexpected errors into the standard error envelope. Direct named calls: <code>logger.error</code> , <code>request.header</code> , <code>response.status</code> .

apps/backend/src/shared/middleware/loadLocalUser.ts

Express request middleware: authentication, authorization, validation, limits, or error handling.

Exported declarations: `loadLocalUser`.

Function/component	Parameters	Explanation and direct calls
<code>loadLocalUser</code> (line 8)	<code>request</code> , <code>_response</code> , <code>next</code>	Loads or creates the local user associated with the verified Firebase identity. Direct named calls: <code>next</code> , <code>findOrCreateAuthUser</code> .

apps/backend/src/shared/middleware/notFound.ts

Express request middleware: authentication, authorization, validation, limits, or error handling.

Exported declarations: notFound.

Function/component	Parameters	Explanation and direct calls
notFound (line 1)	None	No leading explanation comment; read the linked implementation.

apps/backend/src/shared/middleware/rateLimits.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
appWith (line 13)	middleware	No leading explanation comment; read the linked implementation. Direct named calls: <code>express</code> , <code>app.get</code> , <code>app.use</code> .
increment (line 34)	None	No leading explanation comment; read the linked implementation.
decrement (line 35)	None	No leading explanation comment; read the linked implementation.
resetKey (line 36)	None	No leading explanation comment; read the linked implementation.

HTTP registrations (relative to the mount in `app.ts`):

- `app.get("/test")` — line 15
- `app.get("/test")` — line 47
- `app.post("/upload")` — line 73

Declared test groups and cases:

- `describe: rate limits` (line 20)
- `it: answers 429 in the standard error format once the budget is used up` (line 21)
- `it: lets requests through when the shared store (Redis) is down, instead of blocking the site` (line 32)
- `it: can count only successful requests, so rejected uploads do not use up a quota` (line 44)
- `describe.skipIf(!redisAvailable): rate limits backed by real Redis` (line 59)
- `it: enforces one shared budget (the default store, as used in production)` (line 60)
- `describe: upload concurrency gate` (line 69)
- `it: refuses uploads beyond the in-flight maximum with 503 and Retry-After, then admits again` (line 70)

apps/backend/src/shared/middleware/rateLimits.ts

Express request middleware: authentication, authorization, validation, limits, or error handling.

Exported declarations: `createLimiter`, `apiLimiter`, `searchLimiter`, `uploadBurstLimiter`, `csvDailyQuota`, `photoZipDailyQuota`, `listingPhotoLimiter`, `dealerApplicationLimiter`, `createUploadConcurrencyGate`, `uploadConcurrencyGate`.

Function/component	Parameters	Explanation and direct calls
<code>redisStore</code> (line 14)	<code>prefix</code>	Counters live in Redis so every backend instance enforces one shared budget.
<code>redisStore.sendCommand</code> (line 15)	<code>command</code> , <code>args</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>redisClient.call</code> .
<code>userOrIpKey</code> (line 20)	<code>request</code>	The authenticated user when known (per-account limits), otherwise the client IP (IPv6 addresses are grouped by /56 subnet so one host cannot dodge the limit by rotating addresses). Direct named calls: <code>String</code> , <code>ipKeyGenerator</code> .
<code>createLimiter</code> (line 29)	<code>{ name, windowMs, limit, message, code = errorCodes.rateLimited, byUser = false, countSuccessOnly = false, store }</code>	Builds one limiter. On a Redis outage the limiter lets requests through (and logs), because blocking every request would turn a Redis problem into a full outage. Direct named calls: <code>redisStore</code> , <code>rateLimit</code> .
<code>createLimiter.handler</code> (line 40)	<code>_request</code> , <code>_response</code> , <code>next</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>next</code> .
<code>createUploadConcurrencyGate</code> (line 67)	<code>maximum</code>	Caps how many file uploads this instance buffers in memory at once. Runs before the body is read, so an extra upload is refused immediately (503 + Retry-After) instead of using memory.
<code>createUploadConcurrencyGate.release</code> (line 77)	None	No leading explanation comment; read the linked implementation.

apps/backend/src/shared/middleware/requireAuthenticated.ts

Express request middleware: authentication, authorization, validation, limits, or error handling.

Exported declarations: `requireAuthenticated`.

Function/component	Parameters	Explanation and direct calls
<code>requireAuthenticated</code> (line 7)	<code>request</code> , <code>_response</code> , <code>next</code>	Allows only authenticated users whose local account is active. Direct named calls: <code>next</code> .

apps/backend/src/shared/middleware/requireOwnerOrAdmin.ts

Express request middleware: authentication, authorization, validation, limits, or error handling.

Exported declarations: `requireOwnerOrAdmin`.

Function/component	Parameters	Explanation and direct calls
<code>requireOwnerOrAdmin</code> (line 7)	<code>ownerParameter</code>	Allows resource access to its owner or an administrator.

apps/backend/src/shared/middleware/requireRole.ts

Express request middleware: authentication, authorization, validation, limits, or error handling.

Exported declarations: `requireRole`.

Function/component	Parameters	Explanation and direct calls
<code>requireRole</code> (line 8)	<code>roles</code>	Restricts a route to one or more MotorX roles.

apps/backend/src/shared/middleware/validateRequest.ts

Express request middleware: authentication, authorization, validation, limits, or error handling.

Exported declarations: `validateRequest`.

Function/component	Parameters	Explanation and direct calls
<code>validateRequest</code> (line 13)	<code>schemas</code>	Parses selected request sections and rejects invalid input with HTTP 400.

apps/backend/src/shared/middleware/verifyFirebaseToken.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
<code>inOneHour</code> (line 11)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>Math.floor</code> , <code>Date.now</code> .
<code>authenticate</code> (line 12)	<code>token</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>vi.fn</code> , <code>verifyFirebaseToken</code> .

Declared test groups and cases:

- describe: `verifyFirebaseToken` (line 19)
- it: checks a token (including revocation) with Firebase once, then reuses the result (line 22)
- it: checks with Firebase again after the cache period, so a revoked token stops working (line 34)
- it: never trusts a cached token beyond its own expiry (line 46)
- it: rejects requests without a bearer token and does not cache failures (line 57)

apps/backend/src/shared/middleware/verifyFirebaseToken.ts

Express request middleware: authentication, authorization, validation, limits, or error handling.

Exported declarations: `clearVerifiedTokenCache`, `verifyFirebaseToken`.

Function/component	Parameters	Explanation and direct calls
<code>cacheKey</code> (line 18)	<code>token</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>createHash</code> .
<code>clearVerifiedTokenCache</code> (line 20)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>verifiedTokens.clear</code> .
<code>verifyWithCache</code> (line 22)	<code>token</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>cacheKey</code> , <code>Date.now</code> , <code>verifiedTokens.get</code> , <code>verifiedTokens.delete</code> , <code>firebaseAuth.verifyIdToken</code> , <code>Math.min</code> , <code>verifiedTokens.keys</code> , <code>verifiedTokens.set</code> .

Function/component	Parameters	Explanation and direct calls
verifyFirebaseToken (line 40)	request, _response, next	Verifies the bearer token and attaches its Firebase identity to the request. Direct named calls: authorization?.match, next, verifyWithCache.

apps/backend/src/shared/responses/apiResponse.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: sendSuccess.

Function/component	Parameters	Explanation and direct calls
sendSuccess (line 5)	response, data, options	Sends the standard MotorX success envelope with optional status and metadata. Direct named calls: response.status.

apps/backend/src/shared/types/authenticatedRequest.ts

Type contracts; these describe values for TypeScript and are not database writes or runtime checks by themselves.

Exported declarations: AuthenticatedRequest.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/shared/utis/asyncHandler.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: asyncHandler.

Function/component	Parameters	Explanation and direct calls
asyncHandler (line 6)	handler	Forwards rejected controller promises to Express error middleware.

apps/backend/src/shared/utis/imageReencode.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

Declared test groups and cases:

- describe: reencodeImage (line 7)
- it: rebuilds a JPEG as WebP and removes EXIF metadata such as GPS location (line 8)
- it: applies EXIF orientation before stripping it, so photos stay upright (line 21)
- it: drops data hidden after the image (line 30)
- it: refuses an image over the 40 megapixel cap even though the file is small (line 39)
- it: refuses content that only starts like an image (line 46)
- it: refuses decodable formats that are not allowed, such as SVG and GIF (line 52)

- it: keeps PNG documents as PNG and shrinks them to the requested size (line 60)

apps/backend/src/shared/utils/imageReencode.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: ReencodeOutput, InvalidImageError, reencodeImage.

Function/component	Parameters	Explanation and direct calls
reencodeImage (line 14)	input, options	Fully decodes an image and saves a new file from its pixels only. Hidden trailing data, malformed structures, and all metadata (EXIF GPS, camera details) are dropped; the EXIF orientation is applied first so photos stay upright. Images above the pixel cap are refused before decoding, so a small file cannot expand into gigabytes of memory. Direct named calls: sharp, image.metadata, decodableFormats.has, resized.webp, resized.jpeg, resized.png.

apps/backend/src/shared/utils/object.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: objectUtils.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/shared/utils/pagination.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

Declared test groups and cases:

- describe: buildPaginationMeta (line 4)
- it: calculates the final partial page (line 5)
- it: returns zero pages for an empty collection (line 9)

apps/backend/src/shared/utils/pagination.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: buildPaginationMeta.

Function/component	Parameters	Explanation and direct calls
buildPaginationMeta (line 4)	page, limit, total	Builds consistent pagination metadata for collection responses. Direct named calls: Math.ceil.

apps/backend/src/test/accessControl.journey.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
bearer (line 23)	name	No leading explanation comment; read the linked implementation.
person (line 26)	name, role	Creates a signed-in person: Firebase accepts their token, and their local account has this role. Direct named calls: request, bearer, AuthUserModel.findOneAndUpdate.
listingFor (line 32)	dealerId	No leading explanation comment; read the linked implementation. Direct named calls: ListingModel.create.

Declared test groups and cases:

- describe: access control across dealers, roles, and admin actions (HTTP level, no browser) (line 42)
- describe: dealer B cannot reach dealer A's data by changing IDs (line 78)
- it: cannot view, edit, change status of, or delete the listing (line 81)
- it: cannot add, delete, or reorder the listing's photos (line 91)
- it: cannot see, retry, or attach photos to the upload (line 104)
- it: does not get the listing or upload in their own lists (line 113)
- it: cannot open the dealer's verification documents (line 121)
- describe: public and role boundaries (line 128)
- it: does not show a dealer's draft listing publicly (line 129)
- it: refuses to serve private storage objects through the public image route (line 134)
- it: requires sign-in, and the right role, for dealer and admin areas (line 140)
- it: blocks a suspended account even with a valid token (line 149)
- describe: dealer approval requires a verified email (line 155)
- it: refuses approval until the applicant has verified their email, then approves (line 156)

apps/backend/src/test/db.safety.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

Declared test groups and cases:

- describe: test database URI safety (line 4)
- it: accepts the local disposable database (line 18)
- it: accepts the Docker disposable database host (line 22)
- it: rejects a non-test database before any connection is attempted (line 26)
- it: rejects a remote Atlas host unless explicitly enabled (line 30)
- it: allows an explicitly approved remote disposable test database (line 34)

- it: rejects unsupported URI schemes (line 38)
- it: requires TEST_MONGODB_URI and never falls back to MONGODB_URI (line 42)
- it: validates the configured test URI before returning it (line 48)

apps/backend/src/test/db.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `validateTestMongoUri`, `getSafeTestMongoUri`, `connectTestDb`, `clearTestDb`, `disconnectTestDb`.

Function/component	Parameters	Explanation and direct calls
<code>validateTestMongoUri</code> (line 6)	<code>uri</code> , <code>allowRemote</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>allowedLocalHosts.has</code> , <code>parsed.hostname.toLowerCase</code> , <code>parsed.pathname.replace</code> .
<code>getSafeTestMongoUri</code> (line 26)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>validateTestMongoUri</code> .
<code>connectTestDb</code> (line 32)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>mongoose.connect</code> , <code>getSafeTestMongoUri</code> .
<code>clearTestDb</code> (line 36)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>Promise.all</code> , <code>Object.values</code> .
<code>disconnectTestDb</code> (line 42)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>mongoose.disconnect</code> .

apps/backend/src/test/dealerLifecycle.journey.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
<code>bearer</code> (line 22)	<code>name</code>	No leading explanation comment; read the linked implementation.
<code>person</code> (line 23)	<code>name</code> , <code>role</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>request</code> , <code>bearer</code> , <code>AuthUserModel.findOneAndUpdate</code> .
<code>sentStorageKeys</code> (line 35)	<code>commandName</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>mocks.storageSend.mock.calls.filter</code> .
<code>applicationRequest</code> (line 37)	<code>name</code> , <code>overrides</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>request</code> , <code>bearer</code> , <code>Object.entries</code> , <code>call.field</code> , <code>call.attach</code> .
<code>publishedListing</code> (line 61)	<code>dealerId</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>ListingModel.create</code> .
<code>visibleIn</code> (line 73)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>JSON.stringify</code> , <code>request</code> , <code>String</code> .
<code>upload</code> (line 150)	<code>fileName</code> , <code>createdAt</code>	No leading explanation comment; read the linked implementation.

Declared test groups and cases:

- describe: dealer lifecycle (HTTP level) (line 44)

- describe: suspended dealers disappear from public pages (line 60)
- it: hides the listing from browse, detail, and search while suspended, and shows it again after reactivation (line 69)
- describe: dealer profile editing (line 89)
- it: lets an approved dealer update contact details but not the verified business name or registration number (line 90)
- it: validates changes and is only available to approved dealers (line 101)
- describe: applying from an existing account and resubmitting after rejection (line 111)
- it: lets a signed-in buyer apply without creating a new account (line 112)
- it: lets a rejected applicant correct and resubmit, keeping the earlier decision and replacing the old documents (line 119)
- it: does not let an approved dealer submit another application (line 139)
- describe: admin monitoring filters (line 146)
- it: opens one exact upload and filters uploads and audit events by date (line 147)

apps/backend/src/test/env.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/backend/src/test/listingTools.journey.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
bearer (line 20)	name	No leading explanation comment; read the linked implementation.
person (line 21)	name, role	No leading explanation comment; read the linked implementation. Direct named calls: request, bearer, AuthUserModel.findOneAndUpdate.
vehicle (line 30)	dealerId, overrides	No leading explanation comment; read the linked implementation. Direct named calls: ListingModel.create.
bulk (line 41)	name, body	No leading explanation comment; read the linked implementation. Direct named calls: request, bearer.
statusOf (line 42)	id	No leading explanation comment; read the linked implementation. Direct named calls: ListingModel.findById.

Declared test groups and cases:

- describe: dealer listing tools and buyer discovery (HTTP level) (line 44)
- describe: bulk actions (line 60)
- it: publishes every chosen draft in one request, and skips listings the action does not apply to (line 61)
- it: publishes all drafts from one CSV upload without listing their IDs (line 79)
- it: never touches another dealer's listings, even when their IDs are sent (line 91)
- it: marks sold, archives, and reduces prices by a percentage (line 102)
- it: deletes only archived listings, and removes their photos and small copies from storage (line 115)

- it: rejects a request mixing chosen IDs with an upload, and a price cut without a percentage (line 129)
- it: is for dealers only (line 137)
- describe: stale stock (line 143)
- it: counts and lists active listings not confirmed for 60 days, oldest first (line 144)
- it: treats listings saved before lastConfirmedAt existed by their last update time (line 158)
- it: "still available", a price cut, or an edit makes a listing fresh again (line 168)
- describe: similar vehicles and recommendations (line 182)
- it: ranks the closest vehicles first and never includes the vehicle itself or hidden listings (line 183)
- it: recommends vehicles like the ones this browser viewed, leaving out the viewed ones (line 201)
- it: returns nothing (not an error) when no viewed vehicle can be found (line 218)
- it: rejects a malformed viewed list (line 224)
- it: serves the small copy of a photo from the thumbs/ folder (line 229)

apps/backend/vitest.config.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/frontend/src/app/App.tsx

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: App.

Function/component	Parameters	Explanation and direct calls
App (line 33)	None	No leading explanation comment; read the linked implementation.

apps/frontend/src/app/LandingPage.tsx

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: LandingPage.

Function/component	Parameters	Explanation and direct calls
LandingPage (line 8)	None	No leading explanation comment; read the linked implementation. Direct named calls: useState, useNavigate, useI18n, useBuyerListings, t, featuredListings.map.
LandingPage.handleSearchSubmit (line 14)	e	No leading explanation comment; read the linked implementation. Direct named calls: e.preventDefault, searchQuery.trim, navigate, encodeURIComponent.

apps/frontend/src/app/providers.tsx

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: AppProviders.

Function/component	Parameters	Explanation and direct calls
AppProviders (line 10)	{ children }	No leading explanation comment; read the linked implementation.

apps/frontend/src/app/router.tsx

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: router.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/frontend/src/app/theme/ThemeProvider.tsx

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: ThemeMode, ThemeContext, ThemeProvider.

Function/component	Parameters	Explanation and direct calls
getInitialTheme (line 15)	None	No leading explanation comment; read the linked implementation. Direct named calls: window.localStorage.getItem, window.matchMedia.
ThemeProvider (line 28)	{ children }	No leading explanation comment; read the linked implementation. Direct named calls: useState, useEffect, useMemo.
ThemeProvider.toggleTheme (line 39)	None	No leading explanation comment; read the linked implementation. Direct named calls: setThemeState.

apps/frontend/src/app/theme/ThemeToggle.tsx

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: ThemeToggle.

Function/component	Parameters	Explanation and direct calls
SunIcon (line 4)	None	No leading explanation comment; read the linked implementation.
MoonIcon (line 10)	None	No leading explanation comment; read the linked implementation.
ThemeToggle (line 16)	None	No leading explanation comment; read the linked implementation. Direct named calls: useTheme.

apps/frontend/src/app/theme/useTheme.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: useTheme.

Function/component	Parameters	Explanation and direct calls
useTheme (line 4)	None	No leading explanation comment; read the linked implementation. Direct named calls: useContext.

Function/component	Parameters	Explanation and direct calls

apps/frontend/src/config/env.ts

Configuration, validated environment settings, or a shared external-service client.

Exported declarations: env.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/frontend/src/config/firebase.ts

Configuration, validated environment settings, or a shared external-service client.

Exported declarations: firebaseConfig, firebaseApp, firebaseAuthClient.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/frontend/src/config/routes.ts

Configuration, validated environment settings, or a shared external-service client.

Exported declarations: routes.

Function/component	Parameters	Explanation and direct calls
vehicleDetails (line 6)	id	No leading explanation comment; read the linked implementation.
dealerUploadDetails (line 15)	id	No leading explanation comment; read the linked implementation.

apps/frontend/src/features/admin/services/adminApi.ts

Frontend API adapter: translates UI requests into HTTP calls and extracts response data.

Exported declarations: AdminUser, AdminUserDetails, AdminUploadRecord, AdminListing, AdminStats, AdminAuditEvent, AdminAuditLog, AdminUpload, AdminSystemHealth, AdminUserFilters, AdminListingFilters, AdminUploadFilters, AdminAuditFilters, adminApi.

Function/component	Parameters	Explanation and direct calls
getUserDetails (line 36)	userId	No leading explanation comment; read the linked implementation. Direct named calls: apiClient.get.
listUploadRecords (line 39)	uploadId, outcome, page	No leading explanation comment; read the linked implementation. Direct named calls: apiClient.get.
listUsers (line 43)	filters	No leading explanation comment; read the linked implementation. Direct named calls: apiClient.get.
setUserStatus (line 47)	userId, status	No leading explanation comment; read the linked implementation. Direct named calls: apiClient.patch.
listListings (line 51)	filters	No leading explanation comment; read the linked implementation. Direct named calls: apiClient.get.

Function/component	Parameters	Explanation and direct calls
<code>removeListing</code> (line 55)	<code>listingId</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>ApiClient.delete</code> .
<code>getStats</code> (line 59)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>ApiClient.get</code> .
<code>listAuditLogs</code> (line 63)	<code>filters</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>ApiClient.get</code> .
<code>listUploads</code> (line 67)	<code>filters</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>ApiClient.get</code> .
<code>getSystemHealth</code> (line 71)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>ApiClient.get</code> .

apps/frontend/src/features/auth/components/EmailVerificationBanner.test.tsx

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
<code>renderFor</code> (line 11)	<code>user</code>	Renders the banner as if Firebase reported this signed-in user (or nobody). Direct named calls: <code>firebaseAuth.onUserChanged.mockImplementation</code> , <code>render</code> .

Declared test groups and cases:

- describe: `EmailVerificationBanner` (line 16)
- it: is hidden when nobody is signed in or the email is verified (line 19)
- it: asks an unverified user to verify, and can resend the email (line 26)
- it: disappears once the user has verified (line 37)
- it: explains when the link has not been opened yet (line 46)

apps/frontend/src/features/auth/components/EmailVerificationBanner.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: `EmailVerificationBanner`.

Function/component	Parameters	Explanation and direct calls
<code>EmailVerificationBanner</code> (line 6)	None	Shown to signed-in users whose email address is not verified yet. Buyers can use MotorX without it, but a dealer application cannot be approved until the applicant has verified their email. Direct named calls: <code>useState</code> , <code>useEffect</code> .
<code>EmailVerificationBanner.run</code> (line 19)	<code>action</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>setBusy</code> , <code>setMessage</code> , <code>action</code> .

apps/frontend/src/features/auth/components/LoginForm.test.tsx

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
useAuth (line 7)	None	No leading explanation comment; read the linked implementation.
renderLogin (line 12)	None	No leading explanation comment; read the linked implementation. Direct named calls: render.
signIn (line 22)	email, password	No leading explanation comment; read the linked implementation. Direct named calls: <code>userEvent.type</code> , <code>screen.getByPlaceholderText</code> , <code>screen.getByLabelText</code> , <code>userEvent.click</code> , <code>screen.getByRole</code> .

Declared test groups and cases:

- describe: sign-in form (line 28)
- it.each([[{ role: 'dealer' }, '/dealer'], [{ role: 'admin' }, '/admin'], [{ role: 'buyer' }, '/marketplace'], [{ role: 'buyer', dealerStatus: 'pending' }, '/dealer/application-status'], [{ role: 'buyer', dealerStatus: 'rejected' }, '/dealer/application-status'],]): sends %o to the right place after signing in (line 31)
- it: explains a wrong password and a lockout in plain words (line 45)
- it: shows and hides the password (line 60)
- it: asks for the email before sending a reset link, then confirms it was sent (line 70)

apps/frontend/src/features/auth/components/LoginForm.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: LoginForm.

Function/component	Parameters	Explanation and direct calls
friendlyMessage (line 9)	error, t	No leading explanation comment; read the linked implementation. Direct named calls: <code>String</code> , <code>code.includes</code> , <code>t</code> .
LoginForm (line 16)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>useAuth</code> , <code>useNavigate</code> , <code>useI18n</code> , <code>useState</code> , <code>t</code> .
LoginForm.handleSignIn (line 26)	event	No leading explanation comment; read the linked implementation. Direct named calls: <code>event.preventDefault</code> , <code>setMessage</code> , <code>setIsSubmitting</code> , <code>login</code> , <code>email.trim</code> , <code>navigate</code> , <code>friendlyMessage</code> .
LoginForm.handlePasswordReset (line 48)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>email.trim</code> , <code>setMessage</code> , <code>t</code> , <code>firebaseAuth.sendPasswordReset</code> , <code>friendlyMessage</code> .

apps/frontend/src/features/auth/components/PortalRedirect.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: PortalRedirect.

Function/component	Parameters	Explanation and direct calls
PortalRedirect (line 5)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>useAuth</code> .

apps/frontend/src/features/auth/components/ProtectedRoute.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: ProtectedRoute.

Function/component	Parameters	Explanation and direct calls
ProtectedRoute (line 5)	None	No leading explanation comment; read the linked implementation. Direct named calls: useAuth.

apps/frontend/src/features/auth/components/RoleGuard.test.tsx

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
renderDealerPage (line 11)	None	Renders a dealer-only page behind the guard, plus the pages it may redirect to. Direct named calls: render.

Declared test groups and cases:

- describe: RoleGuard (line 25)
- it: sends signed-out visitors to the login page (line 26)
- it: shows the page to a user with an allowed role (line 32)
- it: shows "Access Restricted" to a signed-in user without the role (line 38)
- it: sends an applicant whose dealer application is still pending to the status page (line 45)

apps/frontend/src/features/auth/components/RoleGuard.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: RoleGuard.

Function/component	Parameters	Explanation and direct calls
RoleGuard (line 10)	{ allowedRoles }	No leading explanation comment; read the linked implementation. Direct named calls: useAuth, allowedRoles.includes.

apps/frontend/src/features/auth/context/AuthProvider.test.tsx

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
onAuthStateChanged (line 6)	listener	No leading explanation comment; read the linked implementation.
LogoutButton (line 13)	None	No leading explanation comment; read the linked implementation. Direct named calls: useAuth.

Declared test groups and cases:

- describe: AuthProvider cached data (line 15)

- it: clears the previous account's cached data on sign-out, so the next user never sees it (line 18)
- it: clears cached data when a different account signs in on the same browser (line 32)

apps/frontend/src/features/auth/context/AuthProvider.tsx

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: AuthContext, AuthProvider.

Function/component	Parameters	Explanation and direct calls
loadUserWithDealerStatus (line 12)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>authApi.getCurrentUser</code> , <code>getMyDealerApplication</code> .
AuthProvider (line 24)	{ children }	No leading explanation comment; read the linked implementation. Direct named calls: <code>useState</code> , <code>useQueryClient</code> , <code>useRef</code> , <code>useEffect</code> .
AuthProvider.switchAccount (line 33)	next	Cached API data (uploads, listings, notifications...) belongs to one account. Drop it whenever the signed-in account changes, including sign-out, so the next person on this browser never sees the previous user's data, even briefly. Direct named calls: <code>queryClient.clear</code> , <code>setUser</code> .
AuthProvider.login (line 56)	email, password	No leading explanation comment; read the linked implementation. Direct named calls: <code>firebaseAuth.signInWithEmail</code> , <code>loadUserWithDealerStatus</code> , <code>switchAccount</code> , <code>firebaseAuth.signOut</code> .
AuthProvider.registerBuyer (line 69)	data	No leading explanation comment; read the linked implementation. Direct named calls: <code>firebaseAuth.registerWithEmail</code> , <code>authApi.getCurrentUser</code> , <code>switchAccount</code> , <code>authApi.updateProfile</code> .
AuthProvider.registerDealerApplication (line 75)	data	No leading explanation comment; read the linked implementation. Direct named calls: <code>firebaseAuth.registerWithEmail</code> , <code>authApi.getCurrentUser</code> , <code>authApi.updateProfile</code> , <code>submitDealerApplication</code> , <code>toDealerApplicationPayload</code> , <code>firebaseAuth.signOut</code> , <code>switchAccount</code> , <code>firebaseAuth.deleteCurrentUser</code> .
AuthProvider.logout (line 97)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>firebaseAuth.signOut</code> , <code>switchAccount</code> .
AuthProvider.refreshUser (line 103)	None	Reloads the signed-in user (e.g. after submitting a dealer application from an existing account). Direct named calls: <code>loadUserWithDealerStatus</code> , <code>switchAccount</code> .

apps/frontend/src/features/auth/hooks/useAuth.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: useAuth.

Function/component	Parameters	Explanation and direct calls
useAuth (line 5)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>useContext</code> .

apps/frontend/src/features/auth/pages/DealerPendingPage.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: DealerPendingPage.

Function/component	Parameters	Explanation and direct calls
DealerPendingPage (line 7)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>useLocation</code> , <code>useAuth</code> , <code>useState</code> , <code>useEffect</code> , <code>status.replace</code> .

apps/frontend/src/features/auth/pages/LoginPage.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: LoginPage.

Function/component	Parameters	Explanation and direct calls
LoginPage (line 5)	None	No leading explanation comment; read the linked implementation.

apps/frontend/src/features/auth/pages/RegisterPage.test.tsx

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
<code>renderForSignedInBuyer</code> (line 19)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>mocks.useAuth.mockReturnValue</code> , <code>vi.fn</code> , <code>render</code> .

Declared test groups and cases:

- describe: RegisterPage for a signed-in buyer (line 24)
- it: shows the rejection reason, pre-fills the previous answers, and asks for no password (line 27)
- it: resubmits from the existing account with the new documents (line 38)
- it: sends an applicant whose application is still pending to the status page (line 56)

apps/frontend/src/features/auth/pages/RegisterPage.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: RegisterPage.

Function/component	Parameters	Explanation and direct calls
RegisterPage (line 12)	<code>{ mode = 'buyer' }</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>useAuth</code> , <code>useNavigate</code> , <code>useState</code> , <code>useEffect</code> .
<code>RegisterPage.handleBuyerChange</code> (line 81)	<code>field</code> , <code>value</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>setBuyerForm</code> .
<code>RegisterPage.handleDealerChange</code> (line 85)	<code>field</code> , <code>value</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>setDealerForm</code> .
<code>RegisterPage.handleBuyerSubmit</code> (line 89)	<code>event</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>event.preventDefault</code> , <code>setError</code> , <code>setIsSubmitting</code> , <code>registerBuyer</code> , <code>navigate</code> .

Function/component	Parameters	Explanation and direct calls
<code>RegisterPage.submitAsSignedInBuyer</code> (line 122)	None	Submits (or resubmits) the application from the signed-in buyer's existing account. Direct named calls: <code>setError</code> , <code>setIsSubmitting</code> , <code>submitDealerApplication</code> , <code>toDealerApplicationPayload</code> , <code>refreshUser</code> , <code>navigate</code> .
<code>RegisterPage.handleDealerSubmit</code> (line 142)	event	No leading explanation comment; read the linked implementation. Direct named calls: <code>event.preventDefault</code> , <code>setError</code> , <code>submitAsSignedInBuyer</code> , <code>setIsSubmitting</code> , <code>registerDealerApplication</code> , <code>navigate</code> .

apps/frontend/src/features/auth/services/authApi.ts

Frontend API adapter: translates UI requests into HTTP calls and extracts response data.

Exported declarations: `authApi`.

Function/component	Parameters	Explanation and direct calls
<code>toFrontendUser</code> (line 5)	user	No leading explanation comment; read the linked implementation. Direct named calls: <code>user.email.split</code> .
<code>getCurrentUser</code> (line 20)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>apiClient.get</code> , <code>toFrontendUser</code> .
<code>updateProfile</code> (line 24)	displayName, phone	No leading explanation comment; read the linked implementation. Direct named calls: <code>apiClient.patch</code> , <code>toFrontendUser</code> .

apps/frontend/src/features/auth/services/firebaseAuth.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `firebaseAuth`.

Function/component	Parameters	Explanation and direct calls
<code>signInWithEmail</code> (line 5)	email, password	No leading explanation comment; read the linked implementation. Direct named calls: <code>signInWithEmailAndPassword</code> .
<code>registerWithEmail</code> (line 8)	email, password, displayName	No leading explanation comment; read the linked implementation. Direct named calls: <code>createUserWithEmailAndPassword</code> , <code>updateProfile</code> , <code>sendEmailVerification</code> , <code>credential.user.getIdToken</code> .
<code>getIdToken</code> (line 18)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>firebaseAuthClient.currentUser?.getIdToken</code> .
<code>signOut</code> (line 21)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>signOut</code> .
<code>deleteCurrentUser</code> (line 23)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>deleteUser</code> .
<code>sendPasswordReset</code> (line 27)	email	No leading explanation comment; read the linked implementation. Direct named calls: <code>sendPasswordResetEmail</code> .
<code>resendEmailVerification</code> (line 29)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>sendEmailVerification</code> .

Function/component	Parameters	Explanation and direct calls
refreshEmailVerification (line 35)	None	Reloads the account after the user clicked the link in the email, and refreshes the ID token so the backend sees the new email_verified claim. Returns whether the email is now verified. Direct named calls: user.reload, user.getIdToken.
onUserChanged (line 43)	callback	No leading explanation comment; read the linked implementation. Direct named calls: onIdTokenChanged.
onAuthStateChanged (line 45)	callback	No leading explanation comment; read the linked implementation. Direct named calls: onAuthStateChanged.

apps/frontend/src/features/auth/types/auth.types.ts

Type contracts; these describe values for TypeScript and are not database writes or runtime checks by themselves.

Exported declarations: UserRole, DealerStatus, BuyerRegistrationInput, DealerApplicationInput, DealerApplication, User, DealerProfile, AuthState, AuthContextValue.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/frontend/src/features/buyers/buyer.constants.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: buyerVehicleMakes, buyerFuelTypes, buyerVehicleCategories.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/frontend/src/features/buyers/hooks/useBuyerListing.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: useBuyerListing.

Function/component	Parameters	Explanation and direct calls
useBuyerListing (line 5)	id	Loads one buyer-visible listing only when an ID is available. Direct named calls: useQuery, Boolean.
useBuyerListing.queryFn (line 5)	None	No leading explanation comment; read the linked implementation. Direct named calls: buyerApi.getVehicle.

apps/frontend/src/features/buyers/hooks/useBuyerListings.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: useBuyerListings.

Function/component	Parameters	Explanation and direct calls
useBuyerListings (line 9)	initialFilters, pageSize	Groups marketplace filtering and pagination state for buyer screens. Direct named calls: useState, useQuery, useCallback.

Function/component	Parameters	Explanation and direct calls
<code>useBuyerListings.queryFn</code> (line 11)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>buyerApi.searchVehicles</code> , <code>buyerApi.listVehicles</code> .
<code>useBuyerListings.placeholderData</code> (line 11)	previous	No leading explanation comment; read the linked implementation.

apps/frontend/src/features/buyers/services/buyerApi.ts

Frontend API adapter: translates UI requests into HTTP calls and extracts response data.

Exported declarations: `buyerApi`.

Function/component	Parameters	Explanation and direct calls
<code>toBuyerListing</code> (line 11)	<code>dto</code> , <code>dealer</code>	Converts the shared API contract into the buyer UI model. Direct named calls: <code>dto.images.map</code> .
<code>listVehicles</code> (line 22)	<code>filters</code> , <code>page</code> , <code>limit</code>	Loads active marketplace vehicles with pagination inside response data. Direct named calls: <code>apiClient.get</code> , <code>listings.map</code> .
<code>searchVehicles</code> (line 27)	<code>filters</code> , <code>page</code> , <code>limit</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>apiClient.get</code> , <code>listings.map</code> .
<code>getSimilarVehicles</code> (line 33)	<code>listingId</code> , <code>limit</code>	Public vehicles most like one vehicle (same kind, closest make, model, price and age). Direct named calls: <code>apiClient.get</code> , <code>response.data.data.listings.map</code> .
<code>getRecommendedVehicles</code> (line 38)	<code>viewedIds</code> , <code>limit</code>	Vehicles matched to the ones this browser viewed recently (newest first). Nothing is stored server-side. Direct named calls: <code>apiClient.get</code> , <code>viewedIds.join</code> , <code>response.data.data.listings.map</code> .
<code>getVehicle</code> (line 44)	<code>listingId</code>	Loads one active vehicle, with its dealer's public profile, for the details page. Direct named calls: <code>apiClient.get</code> , <code>toBuyerListing</code> .

apps/frontend/src/features/compare/CompareProvider.tsx

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `MAX_COMPARE`, `CompareProvider`, `useCompare`.

Function/component	Parameters	Explanation and direct calls
<code>CompareProvider</code> (line 20)	<code>{ children }</code>	The vehicles a buyer has picked to compare. Kept in this browser only, so the list survives page changes and reloads without an account. Direct named calls: <code>useState</code> , <code>useCallback</code> , <code>useMemo</code> .
<code>CompareProvider.has</code> (line 27)	<code>id</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>ids.includes</code> .
<code>CompareProvider.toggle</code> (line 28)	<code>id</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>ids.includes</code> , <code>save</code> , <code>ids.filter</code> .
<code>CompareProvider.remove</code> (line 34)	<code>id</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>save</code> , <code>ids.filter</code> .
<code>CompareProvider.clear</code> (line 35)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>save</code> .
<code>has</code> (line 41)	None	No leading explanation comment; read the linked implementation.

Function/component	Parameters	Explanation and direct calls
toggle (line 41)	None	No leading explanation comment; read the linked implementation.
remove (line 41)	None	No leading explanation comment; read the linked implementation.
clear (line 41)	None	No leading explanation comment; read the linked implementation.
useCompare (line 44)	None	Outside a provider (isolated component tests) comparing is simply unavailable. Direct named calls: useContext.

apps/frontend/src/features/compare/CompareToggle.tsx

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: CompareToggle.

Function/component	Parameters	Explanation and direct calls
CompareToggle (line 7)	{ listingId, title, className }	Adds a vehicle to, or removes it from, the compare list. When the list is full it says so instead of silently doing nothing. Direct named calls: useCompare, useId, useState, has, t.
CompareToggle.onClick (line 13)	event	No leading explanation comment; read the linked implementation. Direct named calls: event.preventDefault, event.stopPropagation, toggle, setFull, window.setTimeout.

apps/frontend/src/features/compare/CompareTray.tsx

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: CompareTray.

Function/component	Parameters	Explanation and direct calls
CompareTray (line 9)	None	A bar along the bottom of the buyer pages while vehicles are picked for comparison. Hidden on the compare page itself, which already shows them. Direct named calls: useCompare, useId, useLocation, useBodyClass, t, ids.join.

apps/frontend/src/features/compare/compare.test.tsx

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
id (line 15)	n	No leading explanation comment; read the linked implementation. Direct named calls: String.
vehicle (line 16)	n, overrides	No leading explanation comment; read the linked implementation. Direct named calls: id.
renderWith (line 22)	ui, path	No leading explanation comment; read the linked implementation. Direct named calls: render.

Declared test groups and cases:

- describe: compare vehicles (line 38)
- it: collects up to three vehicles, says when the list is full, and opens the comparison (line 41)
- it: shows vehicles side by side and highlights the best price, year and mileage (line 56)
- it: says so when a vehicle is no longer available (line 70)
- it: picks the best value only when the vehicles differ (line 76)

apps/frontend/src/features/dealers/schemas/dealer.schema.ts

Validation rules and related types; these reject or normalize unsupported input.

Exported declarations: dealerApplicationSchema, dealerResponseSchema, dealerListResponseSchema.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/frontend/src/features/dealers/services/dealerApi.ts

Frontend API adapter: translates UI requests into HTTP calls and extracts response data.

Exported declarations: submitDealerApplication, getDealerDocumentUrl, openDealerDocument, getMyDealerApplication, getPendingDealerApplications, approveDealerApplication, rejectDealerApplication, DealerProfileUpdate, updateMyDealerProfile.

Function/component	Parameters	Explanation and direct calls
submitDealerApplication (line 5)	input, documents	No leading explanation comment; read the linked implementation. Direct named calls: <code>Object.entries</code> , <code>form.append</code> , <code>dealerResponseSchema.parse</code> , <code>apiClient.post</code> .
getDealerDocumentUrl (line 19)	dealerId, documentIndex	No leading explanation comment; read the linked implementation.
openDealerDocument (line 21)	dealerId, documentIndex	No leading explanation comment; read the linked implementation. Direct named calls: <code>window.open</code> , <code>apiClient.get</code> , <code>getDealerDocumentUrl</code> , <code>URL.createObjectURL</code> , <code>window.setTimeout</code> , <code>tab?.close</code> .
getMyDealerApplication (line 34)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>dealerResponseSchema.parse</code> , <code>apiClient.get</code> .
getPendingDealerApplications (line 37)	status	No leading explanation comment; read the linked implementation. Direct named calls: <code>dealerListResponseSchema.parse</code> , <code>apiClient.get</code> .
approveDealerApplication (line 40)	dealerId	No leading explanation comment; read the linked implementation. Direct named calls: <code>dealerResponseSchema.parse</code> , <code>apiClient.patch</code> .
rejectDealerApplication (line 43)	dealerId, reason	No leading explanation comment; read the linked implementation. Direct named calls: <code>dealerResponseSchema.parse</code> , <code>apiClient.patch</code> .
updateMyDealerProfile (line 48)	input	No leading explanation comment; read the linked implementation. Direct named calls: <code>dealerResponseSchema.parse</code> , <code>apiClient.patch</code> .

apps/frontend/src/features/dealers/types/dealer.types.ts

Type contracts; these describe values for TypeScript and are not database writes or runtime checks by themselves.

Exported declarations: DealerApplicationStatus, DealerApplication, CreateDealerApplicationInput.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/frontend/src/features/dealers/utils/applicationPayload.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `toDealerApplicationPayload`.

Function/component	Parameters	Explanation and direct calls
<code>toDealerApplicationPayload</code> (line 5)	<code>data</code>	Maps the dealer application form to the API payload. Shared by the new-account flow and by signed-in buyers applying (or correcting a rejected application) from their existing account. Direct named calls: <code>data.brandFocus.split</code> , <code>Number</code> .

apps/frontend/src/features/inventory/services/inventoryApi.ts

Frontend API adapter: translates UI requests into HTTP calls and extracts response data.

Exported declarations: `inventoryApi`.

Function/component	Parameters	Explanation and direct calls
<code>toPaginated</code> (line 6)	<code>items</code> , <code>pagination</code>	No leading explanation comment; read the linked implementation.
<code>uploadCsv</code> (line 11)	<code>category</code> , <code>file</code> , <code>onProgress</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>formData.append</code> , <code>apiClient.post</code> .
<code>uploadCsv.onUploadProgress</code> (line 16)	<code>event</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>onProgress</code> , <code>Math.round</code> .
<code>downloadTemplate</code> (line 24)	<code>category</code>	Downloads the category CSV template and returns it as a browser-savable Blob. Direct named calls: <code>apiClient.get</code> .
<code>uploadImagesZip</code> (line 30)	<code>uploadId</code> , <code>file</code> , <code>onProgress</code>	Attaches a vehicle-photos zip (one folder per registration number) to a finished CSV upload. Direct named calls: <code>formData.append</code> , <code>apiClient.post</code> .
<code>uploadImagesZip.onUploadProgress</code> (line 34)	<code>event</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>onProgress</code> , <code>Math.round</code> .
<code>listUploads</code> (line 41)	<code>page</code> , <code>limit</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>apiClient.get</code> , <code>toPaginated</code> .
<code>getUpload</code> (line 48)	<code>uploadId</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>apiClient.get</code> .
<code>retryUpload</code> (line 54)	<code>uploadId</code>	Re-queues a failed CSV import; it resumes from where it stopped without duplicating rows. Direct named calls: <code>apiClient.post</code> .
<code>retryImages</code> (line 60)	<code>uploadId</code>	Re-queues failed photo processing for the zip already uploaded with this job. Direct named calls: <code>apiClient.post</code> .
<code>getRejectedRecords</code> (line 65)	<code>uploadId</code> , <code>page</code> , <code>limit</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>apiClient.get</code> , <code>toPaginated</code> .

apps/frontend/src/features/inventory/types/inventory.types.ts

Type contracts; these describe values for TypeScript and are not database writes or runtime checks by themselves.

Exported declarations: UploadJobStatus, ImageProcessingStatus, UploadJob, RejectedRecordReason, RejectedRecord.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/frontend/src/features/listings/components/ImageCropModal.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: ImageCropModal.

Function/component	Parameters	Explanation and direct calls
ImageCropModal (line 17)	{ file, onCancel, onConfirm }	Fixed-ratio pan & zoom cropper: the image always fills the frame ("cover"), the dealer can only drag/zoom within it, so the exported crop can never show letterboxing or an off-ratio result. Direct named calls: useState, useRef, useCallback.
ImageCropModal.handleImageLoad (line 34)	None	No leading explanation comment; read the linked implementation. Direct named calls: Math.max, setNaturalSize, setMinScale, setScale, setOffset.
ImageCropModal.onPointerDown (line 44)	event	No leading explanation comment; read the linked implementation. Direct named calls: event.currentTarget.setPointerCapture.
ImageCropModal.onPointerMove (line 49)	event	No leading explanation comment; read the linked implementation. Direct named calls: setOffset, clampOffset.
ImageCropModal.onPointerUp (line 56)	None	No leading explanation comment; read the linked implementation.
ImageCropModal.onScaleChange (line 58)	nextScale	No leading explanation comment; read the linked implementation. Direct named calls: setScale, setOffset.
ImageCropModal.handleConfirm (line 63)	None	No leading explanation comment; read the linked implementation. Direct named calls: document.createElement, canvas.getContext, ctx.drawImage, canvas.toBlob.
ImageCropModal.handleCancel (line 86)	None	No leading explanation comment; read the linked implementation. Direct named calls: URL.revokeObjectURL, onCancel.

apps/frontend/src/features/listings/components/ListingCard.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: ListingCard.

Function/component	Parameters	Explanation and direct calls
ListingCard (line 21)	{ listing, showStatus = false, comparable = true }	No leading explanation comment; read the linked implementation. Direct named calls: useI18n, listing.images.find, tEnum, getFuelType, formatPrice, t, formatMileage, getMileageKm, getTransmission.

apps/frontend/src/features/listings/components/ListingGallery.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: ListingGallery.

Function/component	Parameters	Explanation and direct calls
<code>ListingGallery</code> (line 19)	<code>{ images, title }</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>useI18n</code> , <code>useState</code> , <code>useRef</code> , <code>t</code> , <code>Math.min</code> , <code>images.map</code> .
<code>ListingGallery.show</code> (line 35)	<code>next</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>setSelectedIndex</code> .
<code>ListingGallery.onTouchStart</code> (line 37)	<code>event</code>	No leading explanation comment; read the linked implementation.
<code>ListingGallery.onTouchEnd</code> (line 38)	<code>event</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>Math.abs</code> , <code>show</code> .
<code>ListingGallery.onKeyDown</code> (line 47)	<code>event</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>event.preventDefault</code> , <code>show</code> .

apps/frontend/src/features/listings/components/ListingPhoto.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: `ListingPhoto`.

Function/component	Parameters	Explanation and direct calls
<code>ListingPhoto</code> (line 16)	<code>{ image, alt, sizes, priority = false, ...imgProps }</code>	A listing photo that lets the browser choose the right size: the 800 px copy on phones and in cards, the full photo only where it is drawn large on a wide or high-density screen.

apps/frontend/src/features/listings/components/ListingStatusBadge.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: `ListingStatusBadge`.

Function/component	Parameters	Explanation and direct calls
<code>ListingStatusBadge</code> (line 8)	<code>{ status }</code>	No leading explanation comment; read the linked implementation.

apps/frontend/src/features/listings/components/ReducePriceDialog.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: `ReducePriceDialog`.

Function/component	Parameters	Explanation and direct calls
<code>ReducePriceDialog</code> (line 15)	<code>{ listings, onConfirm, onClose }</code>	Asks how much to cut the price of one or many listings, and previews the new prices first. Direct named calls: <code>useState</code> , <code>useRef</code> , <code>Number.isInteger</code> , <code>useEffect</code> , <code>listings.slice</code> , <code>PRESETS.map</code> , <code>preview.map</code> .
<code>ReducePriceDialog.onKeyDown</code> (line 27)	<code>event</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>onCloseRef.current</code> .
<code>ReducePriceDialog.confirm</code> (line 32)	<code>None</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>setIsSaving</code> , <code>setError</code> , <code>onConfirm</code> , <code>onClose</code> .

apps/frontend/src/features/listings/schemas/listing.schema.ts

Validation rules and related types; these reject or normalize unsupported input.

Exported declarations: `createListingSchema`, `CreateListingInput`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/frontend/src/features/listings/services/listingApi.ts

Frontend API adapter: translates UI requests into HTTP calls and extracts response data.

Exported declarations: `listingApi`.

Function/component	Parameters	Explanation and direct calls
<code>toListing</code> (line 15)	<code>dto</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>dto.images.map</code> .
<code>toPaginatedResponse</code> (line 43)	<code>response</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>response.data.map</code> .
<code>getMyListingStats</code> (line 57)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>apiClient.get</code> .
<code>bulkAction</code> (line 62)	<code>input</code>	One action on many listings: chosen IDs, or every listing from one CSV upload. Direct named calls: <code>apiClient.post</code> .
<code>getMyListing</code> (line 66)	<code>id</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>apiClient.get</code> , <code>toListing</code> .
<code>getMyListings</code> (line 70)	<code>page</code> , <code>limit</code> , <code>filters</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>apiClient.get</code> , <code>toPaginatedResponse</code> .
<code>createListing</code> (line 77)	<code>input</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>apiClient.post</code> , <code>toListing</code> .
<code>updateListing</code> (line 82)	<code>id</code> , <code>input</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>apiClient.patch</code> , <code>toListing</code> .
<code>updateListingStatus</code> (line 87)	<code>id</code> , <code>input</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>apiClient.patch</code> , <code>toListing</code> .
<code>uploadImage</code> (line 92)	<code>id</code> , <code>file</code> , <code>alt</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>formData.append</code> , <code>apiClient.post</code> , <code>toListing</code> .
<code>deleteImage</code> (line 100)	<code>id</code> , <code>imageKey</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>apiClient.delete</code> , <code>encodeURIComponent</code> , <code>toListing</code> .
<code>reorderImages</code> (line 105)	<code>id</code> , <code>imageKeys</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>apiClient.patch</code> , <code>toListing</code> .
<code>deleteListing</code> (line 110)	<code>id</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>apiClient.delete</code> .

apps/frontend/src/features/listings/types/listing.types.ts

Type contracts; these describe values for TypeScript and are not database writes or runtime checks by themselves.

Exported declarations: `ListingStatus`, `VehicleImage`, `ListingDealer`, `Listing`, `ListingFilters`, `PaginatedResponse`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/frontend/src/features/listings/utils/vehicleAttributes.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `getMileageKm`, `getFuelType`, `getTransmission`, `getCondition`, `getEdition`, `getEngineCapacityCc`, `getBatteryCapacityKwh`, `getBatteryRangeKm`, `getBodyType`, `formatEnumLabel`.

Function/component	Parameters	Explanation and direct calls
<code>getMileageKm</code> (line 8)	listing	No leading explanation comment; read the linked implementation.
<code>getFuelType</code> (line 12)	listing	No leading explanation comment; read the linked implementation.
<code>getTransmission</code> (line 16)	listing	No leading explanation comment; read the linked implementation.
<code>getCondition</code> (line 20)	listing	No leading explanation comment; read the linked implementation.
<code>getEdition</code> (line 24)	listing	No leading explanation comment; read the linked implementation.
<code>getEngineCapacityCc</code> (line 28)	listing	No leading explanation comment; read the linked implementation.
<code>getBatteryCapacityKwh</code> (line 32)	listing	No leading explanation comment; read the linked implementation.
<code>getBatteryRangeKm</code> (line 36)	listing	No leading explanation comment; read the linked implementation.
<code>getBodyType</code> (line 40)	listing	No leading explanation comment; read the linked implementation.
<code>formatEnumLabel</code> (line 45)	value	Human-readable label for a snake_case enum value, e.g. "plug_in_hybrid" -> "Plug in hybrid". Direct named calls: <code>value.replace</code> , <code>spaced.charAt</code> , <code>spaced.slice</code> .

apps/frontend/src/features/notifications/components/NotificationCenter.test.tsx

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
<code>minutesAgo</code> (line 11)	m	No leading explanation comment; read the linked implementation. Direct named calls: <code>Date.now</code> .
<code>note</code> (line 12)	id, overrides	No leading explanation comment; read the linked implementation. Direct named calls: <code>minutesAgo</code> .
<code>renderCentre</code> (line 16)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>render</code> .

Declared test groups and cases:

- describe: notification centre (line 20)
- it: shows the unread count and the list, with details, times and e-mail status (line 23)
- it: marks one notification read, then all of them (line 45)
- it: opens the stale listings from a stale-stock reminder (line 64)
- it: says when there is nothing new, caps a large count, and stays quiet if loading fails (line 73)

- it: does not break the page when the notification service is down (line 84)

apps/frontend/src/features/notifications/components/NotificationCenter.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: NotificationCenter.

Function/component	Parameters	Explanation and direct calls
timeAgo (line 22)	value	No leading explanation comment; read the linked implementation. Direct named calls: Math.max, Math.floor, Date.now.
NotificationCenter (line 32)	None	No leading explanation comment; read the linked implementation. Direct named calls: useState, useNavigate, useEffect, notifications.map.
NotificationCenter.refresh (line 39)	None	No leading explanation comment; read the linked implementation. Direct named calls: Promise.all, notificationApi.unreadCount, notificationApi.list, setUnreadCount, setNotifications.
NotificationCenter.markRead (line 55)	notification	No leading explanation comment; read the linked implementation. Direct named calls: setOpen, navigate, setNotifications, setUnreadCount, notificationApi.markRead.
NotificationCenter.markAllRead (line 64)	None	No leading explanation comment; read the linked implementation. Direct named calls: setLoading, notificationApi.markAllRead, setNotifications, setUnreadCount.

apps/frontend/src/features/notifications/services/notificationApi.ts

Frontend API adapter: translates UI requests into HTTP calls and extracts response data.

Exported declarations: notificationApi.

Function/component	Parameters	Explanation and direct calls
list (line 5)	page, limit	No leading explanation comment; read the linked implementation. Direct named calls: apiClient.get.
unreadCount (line 9)	None	No leading explanation comment; read the linked implementation. Direct named calls: apiClient.get.
markRead (line 13)	id	No leading explanation comment; read the linked implementation. Direct named calls: apiClient.patch.
markAllRead (line 16)	None	No leading explanation comment; read the linked implementation. Direct named calls: apiClient.patch.

apps/frontend/src/features/recommendations/VehicleSuggestions.tsx

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: RecommendedVehicles, SimilarVehicles.

Function/component	Parameters	Explanation and direct calls
SuggestionRow (line 11)	{ id, title, subtitle, listings, action }	A titled row of vehicle cards. On phones the row scrolls sideways instead of stacking, so it never pushes the main results far down the page. Direct named calls: listings.map.

Function/component	Parameters	Explanation and direct calls
RecommendedVehicles (line 28)	{ limit = 8 }	"Recommended for you": shown only once this browser has viewed some vehicles, and only when the server found matches. Failures hide the section; it is a bonus, never a blocker. Direct named calls: useI18n, useState, useQuery, t.
RecommendedVehicles.queryFn (line 31)	None	No leading explanation comment; read the linked implementation. Direct named calls: buyerApi.getRecommendedVehicles.
RecommendedVehicles.clear (line 33)	None	No leading explanation comment; read the linked implementation. Direct named calls: clearRecentlyViewed, setViewed.
SimilarVehicles (line 39)	{ listingId, limit = 6 }	"Similar vehicles" under a listing. Direct named calls: useI18n, useQuery, t.
SimilarVehicles.queryFn (line 41)	None	No leading explanation comment; read the linked implementation. Direct named calls: buyerApi.getSimilarVehicles.

apps/frontend/src/features/recommendations/recentlyViewed.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: MAX_RECENTLY_VIEWED, getRecentlyViewed, recordRecentlyViewed, clearRecentlyViewed.

Function/component	Parameters	Explanation and direct calls
getRecentlyViewed (line 8)	None	No leading explanation comment; read the linked implementation. Direct named calls: readStored.
recordRecentlyViewed (line 12)	listingId	No leading explanation comment; read the linked implementation. Direct named calls: writeStored, getRecentlyViewed.
clearRecentlyViewed (line 16)	None	No leading explanation comment; read the linked implementation. Direct named calls: removeStored.

apps/frontend/src/main.tsx

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/frontend/src/portals/admin/admin.routes.tsx

Route definitions: inspect the listed registrations and guards. Express routes are mounted by backend app.ts; frontend route arrays are metadata, while app/App.tsx mounts the actual React routes.

Exported declarations: adminRoutes.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/frontend/src/portals/admin/layout/AdminLayout.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: AdminLayout.

Function/component	Parameters	Explanation and direct calls
icon (line 5)	path	No leading explanation comment; read the linked implementation.
AdminLayout (line 33)	None	No leading explanation comment; read the linked implementation. Direct named calls: useAuth.

apps/frontend/src/portals/admin/pages/AccountDetails.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: AccountDetails.

Function/component	Parameters	Explanation and direct calls
AccountDetails (line 5)	{ userId, onClose }	No leading explanation comment; read the linked implementation. Direct named calls: useState, useEffect, formatDate, user.dealer.brands.join, Object.entries.

apps/frontend/src/portals/admin/pages/AdminDashboard.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: AdminDashboard.

Function/component	Parameters	Explanation and direct calls
AdminDashboard (line 9)	None	No leading explanation comment; read the linked implementation. Direct named calls: useState, uploads.filter, applications.map, attentionUploads.map, attentionItems.slice, uploads.slice, activity.slice, useEffect, unavailable.has, visibleAttention.map, visibleUploads.map, visibleActivity.map.
AdminDashboard.toggleExpanded (line 28)	section	No leading explanation comment; read the linked implementation. Direct named calls: setExpanded.
formatWaitingTime (line 64)	createdAt	No leading explanation comment; read the linked implementation. Direct named calls: Math.floor, Date.now, Math.max.
formatUploadStatus (line 65)	status	No leading explanation comment; read the linked implementation.
formatAuditEvent (line 66)	event, target	No leading explanation comment; read the linked implementation.
SectionHeader (line 67)	{ title, count, expanded, onToggle }	No leading explanation comment; read the linked implementation.
EmptyMessage (line 68)	{ text }	No leading explanation comment; read the linked implementation.
UnavailableMessage (line 69)	{ text }	No leading explanation comment; read the linked implementation.

apps/frontend/src/portals/admin/pages/AuditLogs.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: AuditLogs.

Function/component	Parameters	Explanation and direct calls
AuditLogs (line 15)	None	Filters (event type, date range, page) live in the URL so a filtered view can be shared or reloaded. Direct named calls: useSearchParams, params.get, Number, useState, useEffect, Boolean, Object.keys, logs.map.
AuditLogs.update (line 27)	changes	No leading explanation comment; read the linked implementation. Direct named calls: Object.entries, next.set, next.delete, setParams.

apps/frontend/src/portals/admin/pages/DealerApprovals.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: DealerApprovals.

Function/component	Parameters	Explanation and direct calls
DealerApprovals (line 7)	None	No leading explanation comment; read the linked implementation. Direct named calls: useState, useSearchParams, params.get, useEffect, dealers.map.
DealerApprovals.setStatus (line 20)	value	No leading explanation comment; read the linked implementation. Direct named calls: setParams.
DealerApprovals.loadApplications (line 22)	None	No leading explanation comment; read the linked implementation. Direct named calls: setError, setDealers, getPendingDealerApplications, setIsLoading.
DealerApprovals.decide (line 33)	dealerId, decision	No leading explanation comment; read the linked implementation. Direct named calls: rejectionReason.trim, setError, setProcessingId, approveDealerApplication, rejectDealerApplication, setRejectingId, setRejectionReason, loadApplications.

apps/frontend/src/portals/admin/pages/ListingMonitoring.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: ListingMonitoring.

Function/component	Parameters	Explanation and direct calls
ListingMonitoring (line 12)	None	No leading explanation comment; read the linked implementation. Direct named calls: useState, useCallback, useEffect, filterableCategories.map, listings.map.
ListingMonitoring.removeListing (line 36)	listing	No leading explanation comment; read the linked implementation. Direct named calls: window.confirm, setRemovingId, setError, adminApi.removeListing, setListings.

apps/frontend/src/portals/admin/pages/SystemHealth.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: SystemHealth.

Function/component	Parameters	Explanation and direct calls
SystemHealth (line 5)	None	No leading explanation comment; read the linked implementation. Direct named calls: useState, useEffect, services.map, formatDateTime.

apps/frontend/src/portals/admin/pages/UploadCollection.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: UploadCollection.

Function/component	Parameters	Explanation and direct calls
UploadCollection (line 8)	{ job }	No leading explanation comment; read the linked implementation. Direct named calls: useState, useEffect, records.map.

apps/frontend/src/portals/admin/pages/UploadMonitoring.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: UploadMonitoring.

Function/component	Parameters	Explanation and direct calls
statusBadge (line 11)	status	No leading explanation comment; read the linked implementation.
UploadMonitoring (line 15)	None	Filters live in the URL, so dashboard links (?uploadId=...) open the exact record and a filtered view can be shared or reloaded. Direct named calls: useSearchParams, params.get, Number, useState, useEffect, dealers.map, uploads.map.
UploadMonitoring.update (line 31)	changes	No leading explanation comment; read the linked implementation. Direct named calls: Object.entries, next.set, next.delete, setParams.

apps/frontend/src/portals/admin/pages/UserManagement.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: UserManagement.

Function/component	Parameters	Explanation and direct calls
UserManagement (line 9)	None	No leading explanation comment; read the linked implementation. Direct named calls: useState, useCallback, useEffect, users.map.
UserManagement.toggleStatus (line 33)	user	Replace only the changed row so current filters remain intact. Direct named calls: setUpdatingId, setError, adminApi.setUserStatus, setUsers.

apps/frontend/src/portals/admin/pages/adminDetails.test.tsx

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

Declared test groups and cases:

- it: opens a bulk collection, shows current listing status, and switches to rejected rows (line 12)
- it: opens the listing owner account with dealership contact details (line 22)

- it: reports collection load failures (line 31)

apps/frontend/src/portals/admin/pages/adminManagement.test.tsx

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
renderAt (line 16)	path, route, element	No leading explanation comment; read the linked implementation. Direct named calls: render.
user (line 18)	id, overrides	No leading explanation comment; read the linked implementation.
listing (line 19)	id, overrides	No leading explanation comment; read the linked implementation.

Declared test groups and cases:

- describe: User management (line 24)
- it: lists users, filters by role and search, and suspends one without reloading the rest (line 27)
- it: shows an empty result and load or update errors (line 48)
- it: reports a failed status change (line 59)
- describe: Listing monitoring (line 68)
- it: filters listings and archives one after confirmation (line 71)
- it: drops an archived listing from an "active" view and keeps it when the dealer cancels (line 93)
- it: shows load and archive errors (line 109)
- describe: Audit logs (line 123)
- it: reads filters from the address, shows events and pages through them (line 126)
- it: distinguishes "nothing recorded" from "nothing matches" and from an outage (line 144)
- describe: System health (line 159)
- it: shows each service status, or the error when the check fails (line 162)

apps/frontend/src/portals/admin/pages/adminPages.test.tsx

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
application (line 15)	id, overrides	No leading explanation comment; read the linked implementation.
renderAt (line 20)	path, route, element	No leading explanation comment; read the linked implementation. Direct named calls: render.

Declared test groups and cases:

- describe: Dealer Approvals (line 22)
- it: opens the tab from the link (?status=approved) instead of always showing pending (line 25)
- it: highlights and scrolls to the exact application from a dashboard link, and shows earlier rejections (line 31)

- describe: Admin dashboard (line 43)
- it: says a section is unavailable instead of showing zero or "none yet" when it fails to load (line 46)
- it: links a waiting application to that exact application (line 59)
- describe: Upload monitoring (line 70)
- it: loads only the linked upload when opened with ?uploadId (line 73)
- it: passes the date range and page to the server (line 82)

apps/frontend/src/portals/buyer/buyer.routes.tsx

Route definitions: inspect the listed registrations and guards. Express routes are mounted by backend app.ts; frontend route arrays are metadata, while app/App.tsx mounts the actual React routes.

Exported declarations: buyerRoutes.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/frontend/src/portals/buyer/layout/BuyerLayout.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: BuyerLayout.

Function/component	Parameters	Explanation and direct calls
BuyerLayout (line 9)	None	No leading explanation comment; read the linked implementation. Direct named calls: useAuth, useCompare, useI18n, useLocation, useState, useRef, useEffect, t.
BuyerLayout.onKeyDown (line 21)	event	No leading explanation comment; read the linked implementation. Direct named calls: setMenuOpen, menuButtonRef.current?.focus.

apps/frontend/src/portals/buyer/pages/ComparePage.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: bestIndexes, ComparePage.

Function/component	Parameters	Explanation and direct calls
number (line 14)	value	No leading explanation comment; read the linked implementation.
bestIndexes (line 25)	values, better	Indexes of the vehicles holding the best value in a row (ties all count); none when all are equal. Direct named calls: values.filter, Math.min, Math.max, known.every, values.flatMap.
ComparePage (line 33)	None	No leading explanation comment; read the linked implementation. Direct named calls: useI18n, useSearchParams, useCompare, searchParams.get, useQueries, ids.map, useEffect, t, results.map, loaded.filter, rows.map.
ComparePage.removeVehicle (line 42)	id	Removes the vehicle from the picked list and, when the page came from a link, from the link too. Direct named calls: remove, ids.filter, setSearchParams, rest.join.

Function/component	Parameters	Explanation and direct calls
<code>ComparePage.queryFn</code> (line 49)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>buyerApi.getVehicle</code> .
<code>ComparePage.value</code> (line 54)	I	No leading explanation comment; read the linked implementation. Direct named calls: <code>formatPrice</code> .
<code>ComparePage.score</code> (line 54)	I	No leading explanation comment; read the linked implementation.
<code>ComparePage.value</code> (line 55)	I	No leading explanation comment; read the linked implementation. Direct named calls: <code>String</code> .
<code>ComparePage.score</code> (line 55)	I	No leading explanation comment; read the linked implementation.
<code>ComparePage.value</code> (line 56)	I	No leading explanation comment; read the linked implementation. Direct named calls: <code>getMileageKm</code> , <code>number</code> .
<code>ComparePage.value</code> (line 57)	I	No leading explanation comment; read the linked implementation. Direct named calls: <code>tEnum</code> .
<code>ComparePage.value</code> (line 58)	I	No leading explanation comment; read the linked implementation. Direct named calls: <code>getBodyType</code> , <code>tEnum</code> .
<code>ComparePage.value</code> (line 59)	I	No leading explanation comment; read the linked implementation. Direct named calls: <code>getCondition</code> , <code>tEnum</code> .
<code>ComparePage.value</code> (line 60)	I	No leading explanation comment; read the linked implementation. Direct named calls: <code>getFuelType</code> , <code>tEnum</code> .
<code>ComparePage.value</code> (line 61)	I	No leading explanation comment; read the linked implementation. Direct named calls: <code>getTransmission</code> , <code>tEnum</code> .
<code>ComparePage.value</code> (line 62)	I	No leading explanation comment; read the linked implementation. Direct named calls: <code>getEngineCapacityCc</code> , <code>number</code> .
<code>ComparePage.value</code> (line 63)	I	No leading explanation comment; read the linked implementation.
<code>ComparePage.value</code> (line 64)	I	No leading explanation comment; read the linked implementation.

apps/frontend/src/portals/buyer/pages/Marketplace.test.tsx

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
<code>car</code> (line 13)	<code>n</code> , overrides	No leading explanation comment; read the linked implementation.
<code>results</code> (line 18)	<code>cars</code> , <code>total</code> , <code>totalPages</code>	No leading explanation comment; read the linked implementation.
<code>renderMarketplace</code> (line 20)	<code>path</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>render</code> .

Declared test groups and cases:

- describe: buyer marketplace (line 25)
- it: lists vehicles and filters them, counting the active filters (line 28)
- it: searches in everyday language after the buyer stops typing, sorted by relevance (line 45)
- it: starts from a search in the address, e.g. from the home page (line 58)

- it: pages through long result lists (line 66)
- it: shows an empty result with a way out, and an error with a retry (line 80)
- it: opens the filter sheet on small screens and closes it with Escape, returning focus (line 93)

apps/frontend/src/portals/buyer/pages/Marketplace.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: Marketplace.

Function/component	Parameters	Explanation and direct calls
countActiveFilters (line 12)	filters	Filters that narrow the results (search text and sort order are shown elsewhere). Direct named calls: <code>Object.entries</code> .
toNumber (line 15)	value	No leading explanation comment; read the linked implementation. Direct named calls: <code>Number</code> .
Marketplace (line 17)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>useI18n</code> , <code>useSearchParams</code> , <code>searchParams.get</code> , <code>useBuyerListings</code> , <code>useState</code> , <code>useRef</code> , <code>countActiveFilters</code> , <code>useEffect</code> , <code>t</code> , <code>buyerVehicleCategories.map</code> , <code>buyerVehicleMakes.map</code> , <code>buyerFuelTypes.map</code> , <code>transmissionTypes.map</code> , <code>vehicleConditions.map</code> , <code>carBodyTypes.map</code> , <code>data.data.map</code> , <code>Array.from</code> .
Marketplace.onKeyDown (line 43)	event	No leading explanation comment; read the linked implementation. Direct named calls: <code>closeFilters</code> .
Marketplace.closeFilters (line 48)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>setFiltersOpen</code> , <code>filtersButtonRef.current?.focus</code> .
Marketplace.clearAll (line 49)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>setSearchInput</code> , <code>setSearchParams</code> , <code>resetFilters</code> .

apps/frontend/src/portals/buyer/pages/VehicleDetails.test.tsx

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
renderPage (line 21)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>render</code> .

Declared test groups and cases:

- describe: vehicle page (line 26)
- it: keeps Call, WhatsApp and Email one tap away, with the WhatsApp message ready (line 33)
- it: loads the small photo copy on phones, and lets the buyer swipe or tap through photos (line 41)
- it: ignores a mostly vertical finger movement (the page scrolling) (line 57)
- it: remembers the vehicle on this device for recommendations, and shows similar vehicles (line 65)

apps/frontend/src/portals/buyer/pages/VehicleDetails.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: VehicleDetails.

Function/component	Parameters	Explanation and direct calls
toAbsoluteUrl (line 17)	url	Dealers can enter a website without a scheme (e.g. "www.abcmotors.lk"); without one, the browser would treat the href as a relative path instead of an external link.
PhoneIcon (line 21)	None	No leading explanation comment; read the linked implementation.
MailIcon (line 27)	None	No leading explanation comment; read the linked implementation.
WebsiteIcon (line 34)	None	No leading explanation comment; read the linked implementation.
WhatsAppIcon (line 41)	None	No leading explanation comment; read the linked implementation.
VehicleDetails (line 47)	None	No leading explanation comment; read the linked implementation. Direct named calls: useParams, useBuyerListing, useI18n, Boolean, useBodyClass, useEffect, t, getEngineCapacityCc, getBatteryCapacityKwh, getBatteryRangeKm, getMileageKm, getEdition, getBodyType, tEnum, String, getCondition, getTransmission, getFuelType, number, whatsappLink, specs.filter, formatPrice, toAbsoluteUrl, dealer.website.replace, formatDate.
VehicleDetails.number (line 71)	value	No leading explanation comment; read the linked implementation.

apps/frontend/src/portals/dealer/dealer.routes.tsx

Route definitions: inspect the listed registrations and guards. Express routes are mounted by backend app.ts; frontend route arrays are metadata, while app/App.tsx mounts the actual React routes.

Exported declarations: dealerRoutes.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/frontend/src/portals/dealer/layout/DealerLayout.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: DealerLayout.

Function/component	Parameters	Explanation and direct calls
icon (line 5)	path	No leading explanation comment; read the linked implementation.
DealerLayout (line 35)	None	No leading explanation comment; read the linked implementation. Direct named calls: useAuth.

apps/frontend/src/portals/dealer/pages/DealerDashboard.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: DealerDashboard.

Function/component	Parameters	Explanation and direct calls
<code>uploadStatusBadge</code> (line 11)	<code>status</code>	No leading explanation comment; read the linked implementation.
<code>DealerDashboard</code> (line 13)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>useQuery</code> , <code>listings.slice</code> , <code>uploads.map</code> .
<code>DealerDashboard.queryFn</code> (line 14)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>listingApi.getMyListingStats</code> .
<code>DealerDashboard.queryFn</code> (line 15)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>listingApi.getMyListings</code> .
<code>DealerDashboard.queryFn</code> (line 16)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>inventoryApi.listUploads</code> .

apps/frontend/src/portals/dealer/pages/DealerProfile.test.tsx

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

Declared test groups and cases:

- describe: `DealerProfile` (line 16)
- it: shows the verified business name and registration number as read-only (line 19)
- it: saves edited details and confirms (line 26)
- it: shows the server message when saving fails (line 39)

apps/frontend/src/portals/dealer/pages/DealerProfile.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: `DealerProfile`.

Function/component	Parameters	Explanation and direct calls
<code>toForm</code> (line 11)	<code>dealer</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>dealer.brands.join</code> , <code>String</code> .
<code>DealerProfile</code> (line 19)	None	Lets an approved dealer keep the business details buyers see up to date (FR-DEALER-03). The business name and registration number were verified during review, so they are read-only. Direct named calls: <code>useState</code> , <code>useEffect</code> , <code>field</code> , <code>set</code> .
<code>DealerProfile.set</code> (line 33)	<code>field</code>	No leading explanation comment; read the linked implementation.
<code>DealerProfile.save</code> (line 35)	<code>event</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>event.preventDefault</code> , <code>setSaving</code> , <code>setStatus</code> , <code>updateMyDealerProfile</code> , <code>form.website.trim</code> , <code>form.brands.split</code> , <code>form.inventoryCount.trim</code> , <code>Number</code> , <code>setDealer</code> , <code>setForm</code> , <code>toForm</code> .
<code>DealerProfile.field</code> (line 52)	<code>label</code> , <code>name</code> , <code>props</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>set</code> .

apps/frontend/src/portals/dealer/pages/InventoryUpload.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: InventoryUpload.

Function/component	Parameters	Explanation and direct calls
statusBadgeClass (line 9)	status	No leading explanation comment; read the linked implementation.
InventoryUpload (line 14)	None	No leading explanation comment; read the linked implementation. Direct named calls: useNavigate, useState, useQuery, uploadableCategories.map, formatFileSize, template?.fields.map, uploads.map.
InventoryUpload.queryFn (line 17)	None	No leading explanation comment; read the linked implementation. Direct named calls: inventoryApi.listUploads.
InventoryUpload.handleFileChange (line 27)	e	No leading explanation comment; read the linked implementation. Direct named calls: setSelectedFile, setError.
InventoryUpload.handleStartupLoad (line 34)	None	No leading explanation comment; read the linked implementation. Direct named calls: setError, setIsUploading, setUploadProgress, inventoryApi.uploadCsv, navigate.
InventoryUpload.handleDownloadTemplate (line 48)	None	No leading explanation comment; read the linked implementation. Direct named calls: setIsDownloading, inventoryApi.downloadTemplate, URL.createObjectURL, document.createElement, document.body.appendChild, link.click, link.remove, URL.revokeObjectURL, setError.

apps/frontend/src/portals/dealer/pages/ListingForm.test.tsx

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
ImageCropModal (line 12)	{ file, onConfirm, onCancel }	No leading explanation comment; read the linked implementation.
saved (line 22)	overrides	No leading explanation comment; read the linked implementation.
renderForm (line 29)	path	No leading explanation comment; read the linked implementation. Direct named calls: render.
field (line 40)	label	No leading explanation comment; read the linked implementation. Direct named calls: screen.getByLabelText.
fillRequired (line 41)	None	No leading explanation comment; read the linked implementation. Direct named calls: userEvent.type, field, userEvent.selectOptions, userEvent.clear.

Declared test groups and cases:

- describe: dealer listing form (line 52)
- it: creates a car listing with only the fields that apply, then returns to the listings (line 55)
- it: asks for battery details for electric vehicles and the extra field of each vehicle type (line 73)
- it: sends the extras of each vehicle type (line 95)
- it: crops each chosen photo, uploads it after saving, and offers a retry when an upload fails (line 107)

- it: keeps no photos when every crop is skipped (line 134)
- it: edits an existing listing: loads it, reorders and removes photos, and saves (line 141)
- it: reports load and photo-order errors (line 164)
- it: reports a failed reorder and a failed save (line 170)

apps/frontend/src/portals/dealer/pages/ListingForm.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: ListingForm.

Function/component	Parameters	Explanation and direct calls
ListingForm (line 26)	None	No leading explanation comment; read the linked implementation. Direct named calls: useNavigate, useParams, useState, Boolean, engineRequiredFuels.has, batteryRequiredFuels.has, useEffect, existingImages.map, listableCategories.map, availableMakes.map, carBodyTypes.map, motorcycleTypes.map, vehicleConditions.map, listableFuelTypes.map, listableTransmissions.map.
ListingForm.buildAttributes (line 65)	None	No leading explanation comment; read the linked implementation. Direct named calls: Number.
ListingForm.handleFilesSelected (line 83)	files	No leading explanation comment; read the linked implementation. Direct named calls: setCropQueue, setCroppedSoFar.
ListingForm.handleCropConfirm (line 89)	croppedFile	No leading explanation comment; read the linked implementation. Direct named calls: cropQueue.slice, setImages, setCropQueue, setCroppedSoFar.
ListingForm.handleCropSkip (line 102)	None	No leading explanation comment; read the linked implementation. Direct named calls: cropQueue.slice, setImages, setCropQueue, setCroppedSoFar.
ListingForm.moveExistingImage (line 113)	index, direction	No leading explanation comment; read the linked implementation. Direct named calls: setError, setImageMessage, listingApi.reorderImages, nextImages.map, setExistingImages.
ListingForm.handleSubmit (line 129)	event	No leading explanation comment; read the linked implementation. Direct named calls: event.preventDefault, setError, setSuccessMessage, setIsSubmitting, buildAttributes, listingApi.updateListing, listingApi.createListing, setSavedListingId, listingApi.uploadImage, setImages, window.setTimeout.

apps/frontend/src/portals/dealer/pages/ListingManager.test.tsx

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
listing (line 13)	n, overrides	No leading explanation comment; read the linked implementation.
page (line 18)	data	No leading explanation comment; read the linked implementation.
renderAt (line 20)	path	No leading explanation comment; read the linked implementation. Direct named calls: render.

Declared test groups and cases:

- describe: dealer listing manager (line 25)
- it: reminds the dealer about stale stock and opens the stale list (line 32)
- it: publishes every selected draft in one request (line 43)
- it: offers one-click fixes on stale listings and shows how long ago each was confirmed (line 56)
- it: previews a price cut before applying it (line 69)
- it: explains skipped listings in plain words (line 84)

apps/frontend/src/portals/dealer/pages/ListingManager.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: `describeBulkResult`, `ListingManager`.

Function/component	Parameters	Explanation and direct calls
<code>describeBulkResult</code> (line 28)	<code>result</code>	"Published 5 listings. 2 were skipped because the action does not apply to their status."
<code>daysSince</code> (line 33)	<code>value</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>Math.floor</code> , <code>Date.now</code> .
<code>ListingManager</code> (line 37)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>useSearchParams</code> , <code>views.includes</code> , <code>searchParams.get</code> , <code>Math.max</code> , <code>Number</code> , <code>useState</code> , <code>useQueryClient</code> , <code>useEffect</code> , <code>useQuery</code> , <code>listings.every</code> , <code>tab</code> , <code>vehicleCategories.map</code> , <code>listings.map</code> .
<code>ListingManager.setParams</code> (line 51)	<code>changes</code>	Updates the URL (so views, filters and pages survive reloads and can be linked to), resetting the page. Direct named calls: <code>Object.entries</code> , <code>next.set</code> , <code>next.delete</code> , <code>setSearchParams</code> .
<code>ListingManager.queryFn</code> (line 66)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>listingApi.getMyListingStats</code> .
<code>ListingManager.queryFn</code> (line 73)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>listingApi.getMyListings</code> .
<code>ListingManager.refresh</code> (line 79)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>Promise.all</code> , <code>queryClient.invalidateQueries</code> .
<code>ListingManager.run</code> (line 85)	<code>action</code> , <code>ids</code> , <code>percent</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>setBusy</code> , <code>setMessage</code> , <code>listingApi.bulkAction</code> , <code>describeBulkResult</code> , <code>setSelected</code> , <code>refresh</code> .
<code>ListingManager.runSafely</code> (line 99)	<code>action</code> , <code>ids</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>run</code> .
<code>ListingManager.deleteForever</code> (line 100)	<code>ids</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>window.confirm</code> , <code>runSafely</code> .
<code>ListingManager.toggleAll</code> (line 107)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>setSelected</code> , <code>listings.map</code> .
<code>ListingManager.toggleOne</code> (line 108)	<code>id</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>setSelected</code> .
<code>ListingManager.tab</code> (line 110)	<code>value</code> , <code>label</code> , <code>count</code>	No leading explanation comment; read the linked implementation.

apps/frontend/src/portals/dealer/pages/UploadDetails.test.tsx

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
job (line 15)	overrides	No leading explanation comment; read the linked implementation.
renderPage (line 23)	None	No leading explanation comment; read the linked implementation. Direct named calls: render.

Declared test groups and cases:

- describe: UploadDetails (line 34)
- it: publishes all of this upload's drafts in one step (line 41)
- it: shows the failure reason and lets the dealer retry a failed CSV import (line 55)
- it: lets the dealer retry failed photo processing (line 68)
- it: shows an error message when the retry is refused (line 80)
- it: does not offer Retry for an upload that succeeded (line 90)

apps/frontend/src/portals/dealer/pages/UploadDetails.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: UploadDetails.

Function/component	Parameters	Explanation and direct calls
PublishDraftsCard (line 14)	{ uploadId }	CSV rows are imported as drafts so the dealer can check them first. This publishes every draft from this upload in one step, instead of opening each listing. Direct named calls: useQueryClient, useQuery, useState.
PublishDraftsCard.queryFn (line 16)	None	No leading explanation comment; read the linked implementation. Direct named calls: listingApi.getMyListings.
PublishDraftsCard.publishAll (line 21)	None	No leading explanation comment; read the linked implementation. Direct named calls: window.confirm, setIsPublishing, setMessage, listingApi.bulkAction, Promise.all, queryClient.invalidateQueries.
RetryButton (line 51)	{ uploadId, stage }	Re-queues a failed stage and refreshes the job so its new pending status shows immediately. Direct named calls: useQueryClient, useState.
RetryButton.retry (line 55)	None	No leading explanation comment; read the linked implementation. Direct named calls: setError, setIsRetrying, inventoryApi.retryUpload, inventoryApi.retryImages, queryClient.invalidateQueries.
VehiclePhotosCard (line 72)	{ uploadId, canUpload, imageProcessingStatus, imageZipFileName, imagesAttached, matchedListings, unmatchedFolders, imageFailureReason, }	No leading explanation comment; read the linked implementation. Direct named calls: useQueryClient, useState, unmatchedFolders.join, activeImageStatuses.has.

Function/component	Parameters	Explanation and direct calls
VehiclePhotosCard.handleUpload (line 81)	None	No leading explanation comment; read the linked implementation. Direct named calls: setError, setIsUploading, setUploadProgress, inventoryApi.uploadImagesZip, setSelectedZip, queryClient.invalidateQueries.
UploadDetails (line 150)	None	No leading explanation comment; read the linked implementation. Direct named calls: useParams, useQuery, formatFileSize, formatDateTime, rejectedRecords.map.
UploadDetails.queryFn (line 154)	None	No leading explanation comment; read the linked implementation. Direct named calls: inventoryApi.getUpload.
UploadDetails.refetchInterval (line 157)	query	Poll while either stage is queued or running (including after a Retry). Direct named calls: activeImageStatuses.has.
UploadDetails.queryFn (line 159)	None	No leading explanation comment; read the linked implementation. Direct named calls: inventoryApi.getRejectedRecords.

apps/frontend/src/portals/dealer/pages/dealerPages.test.tsx

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
job (line 15)	id, overrides	No leading explanation comment; read the linked implementation.
page (line 18)	data	No leading explanation comment; read the linked implementation.
renderAt (line 20)	path, element	No leading explanation comment; read the linked implementation. Direct named calls: render.

Declared test groups and cases:

- describe: bulk vehicle upload (line 34)
- it: shows the template fields for the chosen category and the upload history (line 37)
- it: uploads the chosen CSV, then opens its processing report (line 50)
- it: keeps the dealer on the page with a message when the upload or template download fails (line 62)
- it: downloads the category template as a file (line 74)
- it: shows an empty history and a history that failed to load (line 86)
- it: says when the upload history cannot be loaded (line 92)
- describe: dealer dashboard (line 99)
- it: shows totals by status, recent vehicles, recent uploads and the stale-stock reminder (line 102)
- it: says when the inventory cannot be loaded, and hides the reminder when nothing is stale (line 119)

apps/frontend/src/shared/components/PagerControls.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: PagerControls.

Function/component	Parameters	Explanation and direct calls
PagerControls (line 5)	{ meta, onPage, label }	Previous/next paging for admin tables, with the position and total shown in words. Direct named calls: Math.min.

apps/frontend/src/shared/components/PortalLayout.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: PortalNavLink, PortalNavSection, PortalLayout.

Function/component	Parameters	Explanation and direct calls
PortalLayout (line 23)	{ brand, sections, userName, userRole, onSignOut, toolbarLabel, sidebarStyle, avatarStyle }	Shared shell for the dealer and admin portals. On wide screens the sidebar is always shown; below 1024 px it becomes a drawer opened from the menu button, so every page stays reachable on phones and tablets. The drawer closes on navigation, on Escape, or by tapping outside it. Direct named calls: useState, useLocation, useRef, useEffect, sections.map, getInitials.
PortalLayout.onKeyDown (line 37)	event	No leading explanation comment; read the linked implementation. Direct named calls: setMenuOpen, menuButtonRef.current?.focus.

apps/frontend/src/shared/components/ResponsiveTable.tsx

React UI, page, or layout. Component-local functions handle interactions and state changes.

Exported declarations: ResponsiveTable.

Function/component	Parameters	Explanation and direct calls
ResponsiveTable (line 6)	{ className, children, ...props }	A data table that turns into stacked cards on phones (see .responsive-table in index.css). Each cell is labelled with its column heading, so a card row still reads "Price: Rs 6,000,000". Labels are copied from the elements after every render, so callers write a normal table. Direct named calls: useRef, useLayoutEffect.

apps/frontend/src/shared/components/mobileLayout.test.tsx

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
NotificationCenter (line 6)	None	No leading explanation comment; read the linked implementation.
renderPortal (line 11)	None	No leading explanation comment; read the linked implementation. Direct named calls: render, vi.fn.

Declared test groups and cases:

- describe: portal navigation on phones and tablets (line 25)
- it: opens the menu as a drawer and moves focus into it (line 26)
- it: closes on Escape (focus back on the button) and after choosing a page (line 38)
- describe: tables on phones (line 53)

- it: labels every cell with its column heading, so each row can be shown as a card (line 54)

apps/frontend/src/shared/constants/vehicle.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: availableMakes.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/frontend/src/shared/hooks/useBodyClass.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: useBodyClass.

Function/component	Parameters	Explanation and direct calls
useBodyClass (line 5)	className, active	Adds a class to while active, so fixed bars (compare tray, contact bar) can make room for each other and for the page content underneath them. Direct named calls: useEffect.

apps/frontend/src/shared/i18n/I18nProvider.tsx

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: languages, Language, TranslateValues, I18nContextValue, readableEnum, createTranslator, I18nContext, I18nProvider.

Function/component	Parameters	Explanation and direct calls
isLanguage (line 16)	value	No leading explanation comment; read the linked implementation. Direct named calls: languages.some.
interpolate (line 27)	text, values	No leading explanation comment; read the linked implementation. Direct named calls: text.replace.
readableEnum (line 31)	value	No leading explanation comment; read the linked implementation. Direct named calls: value.split.
createTranslator (line 35)	language	No leading explanation comment; read the linked implementation.
createTranslator.t (line 38)	key, values	No leading explanation comment; read the linked implementation. Direct named calls: interpolate.
createTranslator.tEnum (line 39)	value	No leading explanation comment; read the linked implementation. Direct named calls: readableEnum.
initialLanguage (line 48)	None	First visit: the browser's preferred language when it is Sinhala or Tamil, otherwise English. Direct named calls: readStored, navigator.language.toLowerCase, preferred.startsWith.
setLanguage (line 56)	None	No leading explanation comment; read the linked implementation.
I18nProvider (line 58)	{ children }	No leading explanation comment; read the linked implementation. Direct named calls: useState, useCallback, useEffect, useMemo.

apps/frontend/src/shared/i18n/LanguageSwitcher.tsx

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: LanguageSwitcher.

Function/component	Parameters	Explanation and direct calls
LanguageSwitcher (line 6)	{ className }	Each language is named in its own script, so a reader can find theirs whatever is showing now. Direct named calls: useI18n, t, languages.map.

apps/frontend/src/shared/i18n/i18n.test.tsx

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
useAuth (line 6)	None	No leading explanation comment; read the linked implementation. Direct named calls: vi.fn.
renderSite (line 14)	None	No leading explanation comment; read the linked implementation. Direct named calls: render.
placeholders (line 34)	text	No leading explanation comment; read the linked implementation. Direct named calls: text.match.

Declared test groups and cases:

- describe: Sinhala and Tamil support (line 24)
- it: translates every English message (none left empty or with a lost placeholder) (line 27)
- it: fills in placeholders and translates vehicle values (line 41)
- it: switches the whole site language, sets the page lang, and remembers the choice (line 50)
- it: starts in Tamil for a browser that prefers Tamil (line 64)

apps/frontend/src/shared/i18n/messages/en.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: en, MessageKey, Messages.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/frontend/src/shared/i18n/messages/si.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: si.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/frontend/src/shared/i18n/messages/ta.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `ta`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/frontend/src/shared/i18n/useI18n.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `useI18n`.

Function/component	Parameters	Explanation and direct calls
<code>useI18n</code> (line 4)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>useContext</code> .

apps/frontend/src/shared/services/apiClient.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `apiClient`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/frontend/src/shared/services/apiError.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `ApiResponseError`, `ApiError`.

Function/component	Parameters	Explanation and direct calls
constructor (line 8)	<code>statusCode</code> , <code>message</code> , <code>errors</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>super</code> .

apps/frontend/src/shared/services/apiServices.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
<code>ok</code> (line 29)	<code>data</code> , <code>meta</code>	No leading explanation comment; read the linked implementation.

Declared test groups and cases:

- describe: listing API (dealer) (line 31)
- it: turns listing records into the UI model: first photo primary, missing text filled in (line 34)
- it: uses the right endpoint for each listing operation (line 43)
- it: reads stats and sends bulk actions (line 72)

- describe: buyer API (line 81)
- it: browses and searches with filters and paging (line 84)
- it: loads similar vehicles, recommendations and one vehicle with its dealer (line 94)
- describe: inventory API (line 111)
- it: uploads a CSV with its category and reports progress (line 114)
- it: lists uploads and rejected rows, and retries (line 132)
- describe: dealer API (line 153)
- it: submits an application with its documents and checks the reply (line 156)
- it: rejects a reply that does not match the expected shape (line 169)
- it: reviews applications and updates the profile (line 174)
- it: opens a verification document in a new tab, and closes the tab if loading fails (line 188)
- describe: admin API (line 204)
- it: reads every admin collection with its filters and default page size (line 207)
- it: suspends users and removes listings (line 223)

apps/frontend/src/shared/services/queryClient.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `queryClient`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/frontend/src/shared/utils/formatters.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `formatPrice`, `formatDate`, `formatDateTime`, `getInitials`, `formatMileage`, `formatFileSize`.

Function/component	Parameters	Explanation and direct calls
<code>formatPrice</code> (line 1)	<code>price</code> , <code>currency</code>	No leading explanation comment; read the linked implementation.
<code>formatDate</code> (line 2)	<code>value</code>	No leading explanation comment; read the linked implementation.
<code>formatDateTime</code> (line 3)	<code>value</code>	No leading explanation comment; read the linked implementation.
<code>getInitials</code> (line 4)	<code>name</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>name.trim</code> .
<code>formatMileage</code> (line 5)	<code>mileageKm</code>	No leading explanation comment; read the linked implementation.
<code>formatFileSize</code> (line 6)	<code>bytes</code>	No leading explanation comment; read the linked implementation.

apps/frontend/src/shared/utils/phone.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

Declared test groups and cases:

- describe: WhatsApp numbers (line 4)
- it: adds the Sri Lankan country code to local numbers, however they are written (line 5)
- it: keeps foreign numbers and rejects ones that cannot be dialled (line 12)
- it: builds a link with the message ready to send (line 18)

apps/frontend/src/shared/utils/phone.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `toWhatsAppNumber`, `whatAppLink`.

Function/component	Parameters	Explanation and direct calls
<code>toWhatsAppNumber</code> (line 3)	<code>phone</code>	WhatsApp links need the full international number without "+" or spaces. Dealers usually enter Sri Lankan numbers in local form ("077 123 4567"), so those get the 94 country code. Direct named calls: <code>phone.replace</code> , <code>digits.slice</code> , <code>digits.startsWith</code> .
<code>whatAppLink</code> (line 13)	<code>phone</code> , <code>message</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>toWhatsAppNumber</code> , <code>encodeURIComponent</code> .

apps/frontend/src/shared/utils/safeStorage.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `readStored`, `writeStored`, `removeStored`, `isStringArray`.

Function/component	Parameters	Explanation and direct calls
<code>readStored</code> (line 4)	<code>key</code> , <code>fallback</code> , <code>isValid</code>	Browser storage for small per-device conveniences (language, compare list, recently viewed). Storage can be missing or throw (private windows, blocked site data), so every access is guarded and the app simply falls back to defaults. Direct named calls: <code>window.localStorage.getItem</code> , <code>JSON.parse</code> , <code>isValid</code> .
<code>writeStored</code> (line 15)	<code>key</code> , <code>value</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>window.localStorage.setItem</code> , <code>JSON.stringify</code> .
<code>removeStored</code> (line 19)	<code>key</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>window.localStorage.removeItem</code> .
<code>isStringArray</code> (line 23)	<code>value</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>Array.isArray</code> , <code>value.every</code> .

apps/frontend/src/test/setup.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/frontend/src/vite-env.d.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/frontend/vite.config.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Function/component	Parameters	Explanation and direct calls
manualChunks (line 15)	id	No leading explanation comment; read the linked implementation. Direct named calls: <code>id.includes</code> .

apps/frontend/vitest.config.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/worker/src/benchmarks/importThroughput.bench.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
csv (line 17)	records	No leading explanation comment; read the linked implementation. Direct named calls: <code>Array.from</code> , <code>rows.join</code> .

apps/worker/src/config/database.ts

Configuration, validated environment settings, or a shared external-service client.

Exported declarations: `connectWorkerDatabase`, `disconnectWorkerDatabase`.

Function/component	Parameters	Explanation and direct calls
<code>connectWorkerDatabase</code> (line 5)	None	Opens the worker's independent MongoDB connection before queue consumption. Direct named calls: <code>mongoose.connect</code> .
<code>disconnectWorkerDatabase</code> (line 8)	None	Closes MongoDB after the BullMQ consumer stops accepting work. Direct named calls: <code>mongoose.disconnect</code> .

apps/worker/src/config/env.ts

Configuration, validated environment settings, or a shared external-service client.

Exported declarations: env.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/worker/src/config/mailer.ts

Configuration, validated environment settings, or a shared external-service client.

Exported declarations: mailer, mailerConfig.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/worker/src/config/queue.ts

Configuration, validated environment settings, or a shared external-service client.

Exported declarations: inventoryQueueProducer.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/worker/src/config/redis.ts

Configuration, validated environment settings, or a shared external-service client.

Exported declarations: redisConnection, disconnectWorkerRedis.

Function/component	Parameters	Explanation and direct calls
disconnectWorkerRedis (line 8)	None	Releases the worker's shared Redis connection during graceful shutdown. Direct named calls: redisConnection.quit.

apps/worker/src/config/storage.ts

Configuration, validated environment settings, or a shared external-service client.

Exported declarations: workerStorageClient, workerStorageConfig.

Function/component	Parameters	Explanation and direct calls
mimeTypeForExtension (line 18)	extension	No leading explanation comment; read the linked implementation. Direct named calls: extension.toLowerCase, allowedMimeTypes.has.

apps/worker/src/drills/seedImport.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/worker/src/health.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `startWorkerHealthServer`.

Function/component	Parameters	Explanation and direct calls
<code>startWorkerHealthServer</code> (line 12)	None	Minimal health server for the worker: - <code>/health/live</code> : the process is running and its event loop responds. Use this for container restarts. It deliberately ignores MongoDB and Redis: restarting every worker because a shared dependency is briefly down does not fix anything and only interrupts jobs. - <code>/health/ready</code> (and <code>/health</code> , kept for compatibility): MongoDB and Redis are connected, for dashboards and alarms. Both checks read connection state only, so they never block. Direct named calls: <code>createServer</code> , <code>server.listen</code> .

apps/worker/src/jobs/documentRetention.job.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
<code>deletedKeys</code> (line 13)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>mocks.storageSend.mock.calls.map</code> .

Declared test groups and cases:

- `describe`: dealer document retention (line 15)
- `it`: only looks at decisions made more than 90 days ago (line 24)
- `it`: deletes every stored file, then clears the record with the deletion date (line 32)
- `it`: keeps the record when a storage delete fails, so the next cycle retries it (line 43)

apps/worker/src/jobs/documentRetention.job.ts

Background job handler or scheduled maintenance operation.

Exported declarations: `runDocumentRetention`.

Function/component	Parameters	Explanation and direct calls
<code>runDocumentRetention</code> (line 13)	<code>now</code>	Deletes dealer verification documents (ID cards, business registrations) once the review decision is older than <code>DEALER_DOCUMENT_RETENTION_DAYS</code> . Files are removed from storage first; the database record is cleared only when every file of that dealer is gone, so a storage failure is simply retried on the next cycle. S3 deletes are idempotent, so retries are safe. Direct named calls: <code>now.getTime</code> , <code>findDealersWithExpiredDocuments</code> , <code>workerStorageClient.send</code> , <code>markDealerDocumentsDeleted</code> , <code>console.error</code> , <code>String</code> , <code>console.log</code> .

apps/worker/src/jobs/emailOutbox.job.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
notification (line 13)	emailAttempts	No leading explanation comment; read the linked implementation.

Declared test groups and cases:

- describe: email outbox (line 15)
- it: sends each due email once and marks it sent (line 22)
- it.each([[1, 60_000], [2, 5 * 60_000], [4, 2 * 60 * 60_000]]): after failed attempt %i it retries %i ms later (line 34)
- it: gives up after the fifth failed attempt and marks the email failed (line 45)
- it: marks an email failed at once when the recipient account no longer exists (line 54)
- it: processes a bounded number of emails per cycle (line 65)

apps/worker/src/jobs/emailOutbox.job.ts

Background job handler or scheduled maintenance operation.

Exported declarations: runEmailOutbox.

Function/component	Parameters	Explanation and direct calls
runEmailOutbox (line 15)	now	Sends queued notification emails (the outbox). Business actions only record a pending email; this job delivers it, retrying with increasing delays, so an SMTP outage never fails or reverses the action that caused the email. Delivery is at-least-once: a crash between sending and recording "sent" can repeat one email, which is preferable to losing it. Direct named calls: claimNextDueEmail, now, findAuthUserById, markEmailFailed, mailer.sendMail, notificationEmailText, notificationEmailHtml, markEmailSent, String, scheduleEmailRetry, console.log.

apps/worker/src/jobs/inventoryImages.job.ts

Background job handler or scheduled maintenance operation.

Exported declarations: processInventoryImagesJob.

Function/component	Parameters	Explanation and direct calls
processInventoryImagesJob (line 8)	job	Validates the queue payload before starting durable vehicle-photos extraction. Direct named calls: payloadSchema.parse, processInventoryImages.

apps/worker/src/jobs/inventoryUpload.job.ts

Background job handler or scheduled maintenance operation.

Exported declarations: processInventoryUploadJob.

Function/component	Parameters	Explanation and direct calls
processInventoryUploadJob (line 8)	job	Validates the queue payload before starting durable inventory extraction. Direct named calls: payloadSchema.parse, extractInventoryUpload.

apps/worker/src/jobs/reaper.job.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
bullJob (line 21)	state	No leading explanation comment; read the linked implementation. Direct named calls: <code>vi.fn</code> .

Declared test groups and cases:

- describe: queue reconciliation (reaper) (line 23)
- it: re-queues a pending upload whose queue message was never written (e.g. Redis was down) (line 29)
- it: re-queues photo processing whose BullMQ attempts were exhausted, under the images job id (line 40)
- it: fails a job whose durable attempt budget is used up instead of re-queuing it forever (line 52)
- it.each(['waiting', 'delayed', 'active']): leaves a job alone while its queue message is %s (line 62)

apps/worker/src/jobs/reaper.job.ts

Background job handler or scheduled maintenance operation.

Exported declarations: `ensureQueued`, `runLeaseReaper`.

Function/component	Parameters	Explanation and direct calls
ensureQueued (line 16)	jobName, bullJobId, uploadJobId	Makes sure exactly one live queue message exists for a job. A message that is still waiting, delayed (backing off), or running is left alone; one that is missing, finished, or exhausted is replaced. BullMQ dedups <code>add()</code> by jobId, so the old finished message must be removed first. Direct named calls: <code>inventoryQueueProducer.getJob</code> , <code>ALIVE_STATES.has</code> , <code>existing.getState</code> , <code>existing.remove</code> , <code>inventoryQueueProducer.add</code> .
exhausted (line 26)	attempts	No leading explanation comment; read the linked implementation.
runLeaseReaper (line 33)	now	Reconciles MongoDB (the durable source of truth) with the queue: 1. Expired leases: the worker holding the job crashed or was killed mid-job. 2. Stale pending jobs: the queue message was never written (Redis down or a crash right after the MongoDB insert), was lost, or ran out of BullMQ attempts after temporary failures. Either way the job is re-queued, until its durable attempt budget is used up. Direct named calls: <code>upsertWorkerHeartbeat</code> , <code>now.getTime</code> , <code>Promise.all</code> , <code>findExpiredLeaseUploadJobs</code> , <code>findExpiredLeaseImageJobs</code> , <code>findStalePendingUploadJobs</code> , <code>findStalePendingImageJobs</code> , <code>String</code> , <code>failUploadJob</code> , <code>exhausted</code> , <code>ensureQueued</code> , <code>inventoryBullJobId.csv</code> , <code>failImageProcessing</code> , <code>inventoryBullJobId.images</code> .

apps/worker/src/jobs/staleListingReminder.job.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

Declared test groups and cases:

- describe: stale listing reminders (line 15)

- it: counts listings not confirmed for 60 days, and reminds at most once a week (line 23)
- it: reminds each dealer with their own count, only when this worker wins the claim (line 31)
- it: keeps going when one dealer fails, so the others are still reminded (line 44)

apps/worker/src/jobs/staleListingReminder.job.ts

Background job handler or scheduled maintenance operation.

Exported declarations: `runStaleListingReminders`.

Function/component	Parameters	Explanation and direct calls
<code>runStaleListingReminders</code> (line 13)	<code>now</code>	Tells each dealer how many of their published listings have not been confirmed for <code>STALE_LISTING_DAYS</code> days, at most once every <code>STALE_REMINDER_REPEAT_DAYS</code> days. Safe to run on several worker copies at once: the per-dealer claim lets only one of them send the reminder. Direct named calls: <code>now.getTime</code> , <code>countStaleListingsByDealer</code> , <code>claimStaleListingReminder</code> , <code>notifyStaleListings</code> , <code>console.error</code> , <code>String</code> , <code>console.log</code> .

apps/worker/src/pipeline/categorize.ts

Worker transformation stage or helper. Follow imports from `uploadJob.service.ts` to establish which stages actually run.

Exported declarations: `categorize`.

Function/component	Parameters	Explanation and direct calls
<code>categorize</code> (line 1)	<code>value</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>value.toLowerCase</code> .

apps/worker/src/pipeline/detectDuplicates.ts

Worker transformation stage or helper. Follow imports from `uploadJob.service.ts` to establish which stages actually run.

Exported declarations: `createListingDuplicateKey`, `detectExactDuplicates`.

Function/component	Parameters	Explanation and direct calls
<code>createListingDuplicateKey</code> (line 9)	<code>row</code>	The registration number is a much stronger identity than title/make/model/year (which two genuinely different vehicles of the same trim could share) — it uniquely names one vehicle. Scoped globally (not per-dealer): a plate number identifies one physical vehicle regardless of which dealer account lists it, matching the manual listing form's duplicate rule. Direct named calls: <code>normalizeRegistrationNumber</code> .
<code>detectExactDuplicates</code> (line 12)	<code>rows</code> , <code>seenKeys</code>	Separates unique rows from duplicates found in this upload or in currently listed (draft/active) inventory. Direct named calls: <code>rows.map</code> , <code>findActivelyListedRegistrations</code> , <code>createListingDuplicateKey</code> , <code>seenKeys.has</code> , <code>existingKeys.has</code> , <code>duplicates.push</code> , <code>seenKeys.add</code> , <code>unique.push</code> .

apps/worker/src/pipeline/enrich.ts

Worker transformation stage or helper. Follow imports from `uploadJob.service.ts` to establish which stages actually run.

Exported declarations: `enrich`.

Function/component	Parameters	Explanation and direct calls
<code>enrich</code> (line 1)	<code>row</code> , <code>make</code> , <code>model</code> , <code>year</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>String</code> .

`apps/worker/src/pipeline/extract.test.ts`

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

Declared test groups and cases:

- `describe`: CSV extraction (line 5)
- `it`: streams records in bounded batches and reports cumulative progress (line 6)
- `it`: rejects rows whose column count does not match the header (line 13)

`apps/worker/src/pipeline/extract.ts`

Worker transformation stage or helper. Follow imports from `uploadJob.service.ts` to establish which stages actually run.

Exported declarations: `ExtractedInventoryRow`, `extractCsvBatches`.

Function/component	Parameters	Explanation and direct calls
<code>extractCsvBatches</code> (line 9)	<code>stream</code> , <code>batchSize</code> , <code>onBatch</code> , <code>skipRecords</code>	Streams CSV records into bounded batches without loading the entire file into memory. <code>skipRecords</code> resumes after a checkpoint: those leading records are parsed but not re-processed, and <code>processedRecords</code> keeps counting from the start of the file so row numbers stay stable. Direct named calls: <code>stream.pipe</code> , <code>parse</code> , <code>batch.push</code> , <code>onBatch</code> .

`apps/worker/src/pipeline/generateEmbedding.ts`

Worker transformation stage or helper. Follow imports from `uploadJob.service.ts` to establish which stages actually run.

Exported declarations: `generateEmbedding`.

Function/component	Parameters	Explanation and direct calls
<code>generateEmbedding</code> (line 6)	<code>text</code>	Uses the configured semantic model, while keeping local imports operational without a hosted key. Direct named calls: <code>text.trim</code> , <code>createLocalSearchEmbedding</code> , <code>axios.post</code> , <code>encodeURIComponent</code> , <code>normalizeEmbeddingResponse</code> .

`apps/worker/src/pipeline/normalize.test.ts`

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

Declared test groups and cases:

- describe: inventory normalization (line 4)
- it: normalizes common fields, enums, numbers, and supported price suffixes for a car row (line 5)
- it: normalizes hyphenated and underscored enum aliases the same way (line 16)
- it: restores a CSV title truncated to a prefix of make and model (line 21)
- it: preserves a title with additional information instead of replacing it (line 26)
- it: leaves optional powertrain fields unset when the CSV cell is blank (line 31)
- it: builds motorcycle-specific attributes instead of car attributes (line 37)

apps/worker/src/pipeline/normalize.ts

Worker transformation stage or helper. Follow imports from uploadJob.service.ts to establish which stages actually run.

Exported declarations: `normalizeCommonFields`, `normalizeAttributes`, `normalizeInventoryRow`.

Function/component	Parameters	Explanation and direct calls
<code>normalizeNumber</code> (line 7)	<code>value</code>	Converts supported plain, million, and lakh number representations into a JS number. Returns undefined for blank cells so optional fields (e.g. battery specs on a petrol car) stay unset instead of failing validation as NaN. Direct named calls: <code>compact.endsWith</code> , <code>Number</code> , <code>compact.replace</code> , <code>Number.isNaN</code> .
<code>titleCase</code> (line 16)	<code>value</code>	Produces consistent title case for searchable names and locations. Direct named calls: <code>value.trim</code> .
<code>normalizeEnumValue</code> (line 21)	<code>value</code>	Normalizes CSV enum aliases into the snake_case values used across <code>vehicleCategories</code> , <code>fuelTypes</code> , <code>transmissionTypes</code> , <code>vehicleConditions</code> , <code>carBodyTypes</code> , and <code>motorcycleTypes</code> — "Plug-in Hybrid" / "plug in hybrid" / "PLUG_IN_HYBRID" all resolve to "plug_in_hybrid".
<code>normalizeCsvTitle</code> (line 23)	<code>title</code> , <code>make</code> , <code>model</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>title.trim</code> , <code>fullVehicleName.toLowerCase</code> , <code>normalizedTitle.toLowerCase</code> .
<code>normalizeCommonFields</code> (line 32)	<code>row</code>	Common fields shared by every category's CSV row. Direct named calls: <code>row.description?.trim</code> , <code>titleCase</code> , <code>normalizeCsvTitle</code> , <code>normalizeNumber</code> , <code>row.currency?.trim</code> .
<code>pickNumber</code> (line 49)	<code>key</code> , <code>raw</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>normalizeNumber</code> .
<code>normalizePowertrainAttributes</code> (line 51)	<code>row</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>normalizeEnumValue</code> , <code>pickNumber</code> .
<code>normalizeAttributes</code> (line 64)	<code>category</code> , <code>row</code>	Builds the category-specific attributes object for one CSV row. Unrecognized/irrelevant columns for the selected category are simply not read — the CSV template only includes the columns that apply, but stray extra columns from a hand-edited file are harmlessly ignored. Direct named calls: <code>normalizePowertrainAttributes</code> , <code>row.edition?.trim</code> , <code>normalizeEnumValue</code> , <code>pickNumber</code> .
<code>normalizeInventoryRow</code> (line 89)	<code>category</code> , <code>row</code>	Converts one extracted CSV row into the shape <code>listingRowSchema</code> (from <code>@motorx/shared-contracts</code>) validates: common fields at the top level, plus the category-discriminated attributes object. Direct named calls: <code>normalizeCommonFields</code> , <code>normalizeAttributes</code> .

apps/worker/src/pipeline/persist.ts

Worker transformation stage or helper. Follow imports from uploadJob.service.ts to establish which stages actually run.

Exported declarations: persistValidRows, persistRejectedRows.

Function/component	Parameters	Explanation and direct calls
persistValidRows (line 9)	dealerId, uploadJobId, rows	Applies trusted ownership fields before inserting validated rows as drafts. Direct named calls: Promise.all, Array.from, Math.min, insertImportedListings.
persistRejectedRows (line 26)	records	Persists rejected rows without allowing one invalid record to block valid listings. Direct named calls: insertRejectedRecords.

apps/worker/src/pipeline/transform.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

Declared test groups and cases:

- describe: inventory batch transformation (line 4)
- it: separates valid and invalid rows while retaining CSV row numbers (line 5)
- it: validates a full electric-car CSV row example end to end (line 13)
- it: rejects a mixed batch by row without discarding the valid rows in it (line 21)

apps/worker/src/pipeline/transform.ts

Worker transformation stage or helper. Follow imports from uploadJob.service.ts to establish which stages actually run.

Exported declarations: PreparedInventoryRow, InvalidInventoryRow, prepareInventoryBatch.

Function/component	Parameters	Explanation and direct calls
prepareInventoryBatch (line 11)	category, rows, firstRowNumber	Normalizes and validates a batch (against the upload's chosen vehicle category) while preserving source rows for correction reports. Direct named calls: rows.forEach.

apps/worker/src/pipeline/validate.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
carRow (line 5)	attributes	No leading explanation comment; read the linked implementation.

Declared test groups and cases:

- describe: car row validation (line 7)
- it: accepts a petrol car with an engine capacity (line 8)

- it: accepts a diesel car with an engine capacity (line 12)
- it: accepts a hybrid car with engine capacity and no battery specs (line 16)
- it: accepts an electric car with battery specs and no engine capacity (line 20)
- it: accepts a plug-in hybrid with both engine capacity and battery specs (line 24)
- it: rejects a petrol car missing engine capacity (line 28)
- it: rejects an electric car missing battery capacity and range (line 34)
- it: rejects an invalid body type (line 43)
- it: returns multiple field-level errors for a broadly invalid row (line 48)
- describe: other category row validation (line 55)
- it: accepts a valid motorcycle (line 58)
- it: rejects a motorcycle with an invalid bikeType (line 62)
- it: accepts a valid van (line 66)
- it: accepts a valid truck (line 70)
- it: rejects a truck missing engine capacity for a combustion fuel type (line 74)
- it: accepts a valid three-wheeler (line 78)
- it: accepts an electric three-wheeler without engine capacity (line 82)
- it: accepts a valid bus (line 86)

apps/worker/src/pipeline/validate.ts

Worker transformation stage or helper. Follow imports from uploadJob.service.ts to establish which stages actually run.

Exported declarations: ValidInventoryRow, validateInventoryRow.

Function/component	Parameters	Explanation and direct calls
validateInventoryRow (line 7)	row	Validates one normalized CSV row against the same category-discriminated schema (common fields + conditional fuel-type rules) the backend's manual listing form uses. Direct named calls: <code>listingRowSchema.safeParse</code> , <code>result.error.issues.map</code> .

apps/worker/src/repositories/authUser.repository.ts

Persistence operations: reads or writes stored records for the owning feature.

Exported declarations: WorkerAuthUser, findAuthUserById, findAdminUserIds, claimStaleListingReminder.

Function/component	Parameters	Explanation and direct calls
findAuthUserById (line 12)	userId	Looks up the dealer's contact details for the completion email. auth-users is backend-owned; the worker's only write is the stale-stock reminder marker below. Direct named calls: <code>AuthUserModel.findById</code> .
findAdminUserIds (line 17)	None	Fans admin-only advisory notifications (high rejection rate) out to every active administrator. Direct named calls: <code>AuthUserModel.find</code> , <code>admins.map</code> .

Function/component	Parameters	Explanation and direct calls
claimStaleListingReminder (line 26)	dealerUserId, notRemindedSince, now	Records that a dealer is being reminded about stale stock, but only if they were not reminded since notRemindedSince. Atomic, so two worker copies running the same cycle never both send the reminder: only the one whose update matched goes on to notify. (staleReminderSentAt is the one field the worker writes here; the backend model ignores it.) Direct named calls: AuthUserModel.updateOne.

apps/worker/src/repositories/dealerDocument.repository.ts

Persistence operations: reads or writes stored records for the owning feature.

Exported declarations: RetainedDealerDocuments, findDealersWithExpiredDocuments, markDealerDocumentsDeleted.

Function/component	Parameters	Explanation and direct calls
findDealersWithExpiredDocuments (line 15)	cutoff, limit	Reviewed applications whose decision is older than the cutoff and still have stored files. Direct named calls: DealerModel.find.
markDealerDocumentsDeleted (line 24)	dealerId, deletedAt	Clears the document list only after every file was deleted from storage, and records when. Direct named calls: DealerModel.updateOne.

apps/worker/src/repositories/jobSafety.db.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
jobs (line 14)	None	No leading explanation comment; read the linked implementation. Direct named calls: mongoose.connection.collection.
createPendingJob (line 23)	None	No leading explanation comment; read the linked implementation. Direct named calls: jobs, String.

Declared test groups and cases:

- describe.skipIf(!safeUri): job safety guarantees in MongoDB (line 12)
- it: lets only the current lease owner write once an expired lease is taken over (line 28)
- it: does not let a second worker claim a job whose lease is still valid (line 42)
- it: never stores two listings for the same CSV row of the same upload (line 49)
- it: records each rejected row once, even when the batch is processed again (line 61)

apps/worker/src/repositories/listing.repository.ts

Persistence operations: reads or writes stored records for the owning feature.

Exported declarations: ImportableListing, WorkerListingImage, findActivelyListedRegistrations, countOpenDealerListings, findImportedRowNumbers, insertImportedListings, findListingsByUploadJob, appendListingImages, countStaleListingsByDealer.

Function/component	Parameters	Explanation and direct calls
<code>findActivelyListedRegistrations</code> (line 24)	<code>normalizedRegistrationNumbers</code>	Returns which of the given normalized registration numbers already belong to a currently listed (draft/active) vehicle — archived/sold listings don't block a CSV row from importing. Direct named calls: <code>ListingModel.find</code> , <code>matches.map</code> .
<code>countOpenDealerListings</code> (line 31)	<code>dealerId</code>	Drafts and active listings count toward the per-dealer listing limit. Direct named calls: <code>ListingModel.countDocuments</code> .
<code>findImportedRowNumbers</code> (line 36)	<code>uploadJobId</code> , <code>rowNumbers</code>	Returns which of these CSV row numbers an earlier attempt of this upload already imported. Direct named calls: <code>ListingModel.find</code> , <code>rows.map</code> .
<code>insertImportedListings</code> (line 43)	<code>rows</code>	Inserts validated draft listings as one ordered batch owned by the upload's dealer. Direct named calls: <code>ListingModel.insertMany</code> , <code>rows.map</code> .
<code>findListingsByUploadJob</code> (line 49)	<code>uploadJobId</code>	Loads every listing this exact upload job created, for matching against zip folder names. Direct named calls: <code>ListingModel.find</code> .
<code>appendListingImages</code> (line 55)	<code>listingId</code> , <code>images</code> , <code>maximum</code>	Appends photos to one listing, capped at the configured per-listing image limit. Refuses the whole append if any of these keys is already attached, so overlapping attempts cannot duplicate. Direct named calls: <code>ListingModel.updateOne</code> , <code>images.map</code> .
<code>countStaleListingsByDealer</code> (line 65)	<code>cutoff</code> , <code>limit</code>	Dealers with active listings not confirmed since <code>cutoff</code> , with how many. Listings saved before <code>lastConfirmedAt</code> existed fall back to their last update time (same rule as the dealer's stale list). Direct named calls: <code>ListingModel.aggregate</code> .

apps/worker/src/repositories/models.ts

Database shapes, Mongoose models, and indexes. Models describe stored documents; they do not open the database connection.

Exported declarations: `UploadJobModel`, `ListingModel`, `AuditLogModel`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

apps/worker/src/repositories/notification.repository.ts

Persistence operations: reads or writes stored records for the owning feature.

Exported declarations: `NotificationChannel`, `NotificationEmailStatus`, `NotificationDetails`, `WorkerNotification`, `createNotification`, `claimNextDueEmail`, `markEmailSent`, `scheduleEmailRetry`, `markEmailFailed`.

Function/component	Parameters	Explanation and direct calls
<code>createNotification</code> (line 47)	<code>input</code>	Records the notification. The email leg (if any) is only queued here: pending rows are sent by the outbox job, so creating a notification never waits on, or fails because of, SMTP. Direct named calls: <code>input.channels.includes</code> , <code>NotificationModel.create</code> .
<code>claimNextDueEmail</code> (line 56)	<code>now</code>	Atomically takes the oldest due email, so several workers never send the same one at once. A claim that is never finished (the worker died mid-send) becomes available again after <code>CLAIM_MS</code> . Direct named calls: <code>NotificationModel.findOneAndUpdate</code> , <code>now.getTime</code> .
<code>markEmailSent</code> (line 70)	<code>notificationId</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>NotificationModel.updateOne</code> .

Function/component	Parameters	Explanation and direct calls
<code>scheduleEmailRetry</code> (line 74)	<code>notificationId</code> , <code>nextAttemptAt</code> , <code>error</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>NotificationModel.updateOne</code> , <code>error.slice</code> .
<code>markEmailFailed</code> (line 78)	<code>notificationId</code> , <code>error</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>NotificationModel.updateOne</code> , <code>error.slice</code> .

apps/worker/src/repositories/rejectedRecord.repository.ts

Persistence operations: reads or writes stored records for the owning feature.

Exported declarations: `RejectedInventoryRecord`, `insertRejectedRecords`.

Function/component	Parameters	Explanation and direct calls
<code>insertRejectedRecords</code> (line 16)	<code>records</code>	Persists invalid and duplicate rows for dealer correction later. Insert-once per (upload, row): re-running a batch after a crash leaves the existing records untouched. Direct named calls: <code>RejectedRecordModel.bulkWrite</code> , <code>records.map</code> .

apps/worker/src/repositories/uploadJob.repository.ts

Persistence operations: reads or writes stored records for the owning feature.

Exported declarations: `ImageProcessingStatus`, `WorkerUploadJob`, `JOB_LEASE_MS`, `ProcessingCounts`, `claimPendingUploadJob`, `renewUploadLease`, `updateUploadProgress`, `completeUploadJob`, `failUploadJob`, `retryUploadJob`, `claimPendingImageProcessing`, `renewImageLease`, `completeImageProcessing`, `failImageProcessing`, `retryImageProcessing`, `findExpiredLeaseUploadJobs`, `findExpiredLeaseImageJobs`, `findStalePendingUploadJobs`, `findStalePendingImageJobs`.

Function/component	Parameters	Explanation and direct calls
<code>leaseUntil</code> (line 31)	<code>now</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>now.getTime</code> .
<code>claimPendingUploadJob</code> (line 41)	<code>uploadJobId</code> , <code>leaseOwner</code>	----- CSV extraction stage. Every write made while processing requires the caller's lease-owner token, so a worker whose lease expired (and was taken over) can no longer change the job. ----- Claims pending work, or work whose lease expired because its worker crashed. Direct named calls: <code>UploadJobModel.findOneAndUpdate</code> , <code>leaseUntil</code> .
<code>renewUploadLease</code> (line 51)	<code>uploadJobId</code> , <code>leaseOwner</code>	Extends the lease while this owner still holds it; false means the job was taken over. Direct named calls: <code>UploadJobModel.updateOne</code> , <code>leaseUntil</code> .
<code>updateUploadProgress</code> (line 57)	<code>uploadJobId</code> , <code>leaseOwner</code> , <code>counts</code>	Saves cumulative counters after a batch is durably persisted: the checkpoint a retry resumes from. Direct named calls: <code>UploadJobModel.updateOne</code> , <code>leaseUntil</code> .
<code>completeUploadJob</code> (line 63)	<code>uploadJobId</code> , <code>leaseOwner</code> , <code>counts</code>	Marks a fully processed upload using error-aware terminal status semantics. Direct named calls: <code>UploadJobModel.updateOne</code> .
<code>failUploadJob</code> (line 74)	<code>uploadJobId</code> , <code>failureReason</code> , <code>leaseOwner</code>	Records a terminal failure. With a lease owner, only that owner may fail the job; without one (the reaper), only a job still in the expected state is changed. Direct named calls: <code>UploadJobModel.updateOne</code> , <code>failureReason.slice</code> .

Function/component	Parameters	Explanation and direct calls
retryUploadJob (line 81)	uploadJobId, leaseOwner, failureReason	Hands the job back as pending (temporary failure or shutdown) so a later attempt resumes it. Direct named calls: <code>UploadJobModel.updateOne, failureReason.slice</code> .
claimPendingImageProcessing (line 93)	uploadJobId, leaseOwner	----- Image stage: same lease rules, with its own attempt counter. ----- Direct named calls: <code>UploadJobModel.findOneAndUpdate, leaseUntil</code> .
renewImageLease (line 102)	uploadJobId, leaseOwner	No leading explanation comment; read the linked implementation. Direct named calls: <code>UploadJobModel.updateOne, leaseUntil</code> .
completeImageProcessing (line 109)	uploadJobId, leaseOwner, result	Marks image processing complete, recording how many photos were attached and which folders in the zip didn't match any listing this upload job created. Direct named calls: <code>UploadJobModel.updateOne</code> .
failImageProcessing (line 118)	uploadJobId, failureReason, leaseOwner	No leading explanation comment; read the linked implementation. Direct named calls: <code>UploadJobModel.updateOne, failureReason.slice</code> .
retryImageProcessing (line 124)	uploadJobId, leaseOwner, failureReason	No leading explanation comment; read the linked implementation. Direct named calls: <code>UploadJobModel.updateOne, failureReason.slice</code> .
findExpiredLeaseUploadJobs (line 137)	now	----- Reaper / reconciliation queries. ----- Jobs whose lease outlived the worker that claimed them (it crashed or was killed mid-job). Direct named calls: <code>UploadJobModel.find</code> .
findExpiredLeaseImageJobs (line 141)	now	No leading explanation comment; read the linked implementation. Direct named calls: <code>UploadJobModel.find</code> .
findStalePendingUploadJobs (line 148)	pendingSince	Jobs that have been pending longer than expected: their queue message may never have been written (Redis down or a crash right after the MongoDB insert), or it was lost or exhausted. The MongoDB record is the durable source of truth, so these are re-enqueued from here. Direct named calls: <code>UploadJobModel.find</code> .
findStalePendingImageJobs (line 152)	pendingSince	No leading explanation comment; read the linked implementation. Direct named calls: <code>UploadJobModel.find</code> .

apps/worker/src/repositories/workerHeartbeat.repository.ts

Persistence operations: reads or writes stored records for the owning feature.

Exported declarations: `upsertWorkerHeartbeat`.

Function/component	Parameters	Explanation and direct calls
upsertWorkerHeartbeat (line 7)	None	Writes a single timestamped doc the backend's admin system-health endpoint reads to approximate worker liveness (a separate container it has no other way to observe).

apps/worker/src/services/activeLeases.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `LeaseStage`, `trackLease`, `untrackLease`, `listActiveLeases`.

Function/component	Parameters	Explanation and direct calls
trackLease (line 7)	uploadJobId, stage, owner	No leading explanation comment; read the linked implementation. Direct named calls: <code>active.set</code> .
untrackLease (line 8)	owner	No leading explanation comment; read the linked implementation. Direct named calls: <code>active.delete</code> .
listActiveLeases (line 9)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>active.values</code> .

apps/worker/src/services/emailTemplate.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `notificationEmailHtml`, `notificationEmailText`.

Function/component	Parameters	Explanation and direct calls
escapeHtml (line 3)	value	No leading explanation comment; read the linked implementation. Direct named calls: <code>value.replace</code> .
detailRows (line 9)	details	No leading explanation comment; read the linked implementation. Direct named calls: <code>Object.entries</code> .
notificationEmailHtml (line 15)	title, message, details	The one MotorX notification email layout, used for every email the outbox sends. Direct named calls: <code>detailRows</code> , <code>escapeHtml</code> .
notificationEmailText (line 21)	title, message, details	No leading explanation comment; read the linked implementation. Direct named calls: <code>Object.entries</code> , <code>lines.join</code> .

apps/worker/src/services/imageProcessing.service.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
fakeEntry (line 12)	path, type, content	No leading explanation comment; read the linked implementation.
fakeEntry.buffer (line 13)	None	No leading explanation comment; read the linked implementation.
fakeEntry.stream (line 13)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>Readable.from</code> .
mimeTypeForExtension (line 20)	extension	No leading explanation comment; read the linked implementation. Direct named calls: <code>extension.toLowerCase</code> .
storedObjects (line 39)	None	Uploaded objects are every storage call after the initial zip download. Direct named calls: <code>mocks.storageSend.mock.calls.slice</code> .
transformToByteArray (line 46)	None	No leading explanation comment; read the linked implementation.
buffer (line 137)	None	No leading explanation comment; read the linked implementation.
stream (line 138)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>Readable.from</code> .
files (line 232)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>fakeEntry</code> .

Function/component	Parameters	Explanation and direct calls
<code>transformToByteArray</code> (line 242)	None	No leading explanation comment; read the linked implementation.

Declared test groups and cases:

- describe: inventory image processing (line 41)
- it: matches a zip folder to a listing by normalized registration number and attaches images (line 51)
- it: groups entries whose zip path uses backslashes (PowerShell Compress-Archive) the same as forward slashes (line 68)
- it: matches images when the zip contains an extra wrapper folder (line 78)
- it: records a folder that matches no listing from this upload job as unmatched (line 88)
- it: skips root-level files with no folder and non-image files (line 98)
- it: caps attached images at the configured per-listing limit, accounting for existing images (line 114)
- it: fails immediately (no retry) on an entry that exceeds the expanded size limit, without buffering it in full (line 130)
- it: stores a re-encoded WebP with all metadata (including GPS location) removed (line 151)
- it: skips a file that only starts like a JPEG but cannot be decoded (line 166)
- it: skips an image above the pixel cap even when the file itself is small (line 177)
- it: shrinks large photos to the maximum stored dimension (line 190)
- it: also stores an 800 px small copy for cards and phones, and links it on the listing (line 201)
- it: retries a temporary storage failure with backoff instead of failing the job (line 215)
- it: fails immediately when the upload is not a zip archive at all (line 223)
- it: is idempotent: a retry after a crash does not attach the same photos again (line 230)
- it: derives the same key for the same photo and a different key for different content (line 253)
- it: fails at once with an actionable reason (not a silent 0/0) when no photo is inside a folder (line 261)

apps/worker/src/services/imageProcessing.service.ts

Business operations and orchestration; see each function and its direct calls below.

Exported declarations: `InvalidArchiveError`, `deterministicImageKey`, `processInventoryImages`.

Function/component	Parameters	Explanation and direct calls
constructor (line 18)	message	No leading explanation comment; read the linked implementation. Direct named calls: <code>super</code> .
<code>downloadZipBuffer</code> (line 22)	storageKey	Downloads the private zip as one buffer. Bounded by the backend's zip size cap at upload time, so buffering the whole archive here (rather than streaming) keeps the extraction logic simple. Direct named calls: <code>workerStorageClient.send</code> , <code>Buffer.from</code> , <code>object.Body.transformToByteArray</code> .
<code>fullObjectKey</code> (line 32)	key	No leading explanation comment; read the linked implementation.
<code>thumbObjectKey</code> (line 33)	key	No leading explanation comment; read the linked implementation.
<code>imageMimeType</code> (line 35)	buffer	No leading explanation comment; read the linked implementation. Direct named calls: <code>buffer.subarray</code> , <code>Buffer.from</code> .

Function/component	Parameters	Explanation and direct calls
<code>readEntryWithinLimit</code> (line 46)	<code>entry</code> , <code>limitBytes</code>	Reads a zip entry while enforcing <code>limitBytes</code> as it decompresses, rather than after fully inflating it — a hand-crafted entry can lie about its declared size, so the only way to cap actual memory use against a decompression bomb is to stop reading mid-stream once the real byte count crosses the limit, instead of trusting metadata or buffering first.
<code>extractImageEntries</code> (line 73)	<code>zipBuffer</code>	Extracts image entries from the zip, grouped by the folder that directly contains each photo (the vehicle's registration number), so both <code>CAX-1234/photo.jpg</code> and an extra wrapper folder such as <code>Photos/CAX-1234/photo.jpg</code> work. Root-level files (no folder), non-image files, and images that cannot be decoded are skipped — a hand-built zip may contain extras, and one bad file shouldn't fail the whole batch. Direct named calls: <code>unzipper.Open.buffer</code> , <code>entry.path.split</code> , <code>fileName.split</code> , <code>workerStorageConfig.mimeTypeForExtension</code> , <code>Math.max</code> , <code>Math.min</code> , <code>readEntryWithinLimit</code> , <code>imageMimeType</code> , <code>reencodeListingPhoto</code> , <code>normalizeRegistrationNumber</code> , <code>grouped.get</code> , <code>list.push</code> , <code>segments.join</code> , <code>createHash</code> , <code>makeListingThumb</code> , <code>grouped.set</code> .
<code>deterministicImageKey</code> (line 121)	<code>uploadJobId</code> , <code>listingId</code> , <code>entry</code>	The same photo from the same upload always gets the same storage key, so a retry after a crash overwrites the object it already wrote (no orphans) and recognises photos it already attached. Formatted like a UUID to satisfy the public image-key format. Direct named calls: <code>createHash</code> , <code>hex.slice</code> .
<code>uploadImage</code> (line 127)	<code>key</code> , <code>order</code> , <code>entry</code>	Uploads one re-encoded photo (and its small copy) under its deterministic key and returns its <code>ListingImage</code> metadata. Direct named calls: <code>Promise.all</code> , <code>put</code> , <code>fullObjectKey</code> , <code>thumbObjectKey</code> , <code>encodeURIComponent</code> .
<code>uploadImage.put</code> (line 128)	<code>objectKey</code> , <code>body</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>workerStorageClient.send</code> .
<code>processInventoryImages</code> (line 141)	<code>uploadJobId</code>	Downloads, extracts, matches, and attaches a vehicle-photos zip to the listings this exact upload job created — matched by normalized registration number (the zip's folder names), so a zip can never attach photos to another dealer's or another job's listings. Safe to retry at any point: photos already attached by an earlier attempt are recognised by their key. Direct named calls: <code>randomUUID</code> , <code>claimPendingImageProcessing</code> , <code>console.warn</code> , <code>holdLease</code> , <code>trackLease</code> , <code>downloadZipBuffer</code> , <code>extractImageEntries</code> , <code>findListingsByUploadJob</code> , <code>listings.map</code> , <code>lease.assertHeld</code> , <code>listingsByRegistration.get</code> , <code>unmatchedFolders.push</code> , <code>String</code> , <code>existing.map</code> , <code>entries.map</code> , <code>keyed.filter</code> , <code>Math.max</code> , <code>Promise.all</code> , <code>toAttach.map</code> , <code>appendListingImages</code> , <code>Promise.allSettled</code> , <code>images.flatMap</code> , <code>completeImageProcessing</code> , <code>notifyImageProcessingResult</code> , <code>isTransientError</code> , <code>retryImageProcessing</code> , <code>failImageProcessing</code> , <code>lease.stop</code> , <code>untrackLease</code> .

apps/worker/src/services/imageReencode.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `reencodeListingPhoto`, `makeListingThumb`.

Function/component	Parameters	Explanation and direct calls
<code>reencodeListingPhoto</code> (line 10)	<code>input</code>	Rebuilds a listing photo as a fresh WebP from its decoded pixels (same rules as the backend's direct uploads): drops hidden trailing data and all metadata such as EXIF GPS location, keeps the photo upright, caps its size, and refuses images above the pixel limit before decoding. Returns null for anything that is not a decodable JPEG, PNG, or WebP so callers can skip it. Direct named calls: <code>sharp</code> , <code>image.metadata</code> , <code>decodableFormats.has</code> .

Function/component	Parameters	Explanation and direct calls
<code>makeListingThumb</code> (line 26)	<code>cleanWebp</code>	Makes the small copy (max 800 px) shown on cards and phones from an already cleaned photo. Direct named calls: <code>sharp</code> .

apps/worker/src/services/jobLease.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

Declared test groups and cases:

- `describe: holdLease` (line 7)
- `it: renews the lease every third of its duration while work continues` (line 11)
- `it: reports the lease as lost once another worker owns the job` (line 23)
- `it: keeps the lease through a temporary renewal error and tries again next time` (line 32)
- `it: stops renewing when the job ends` (line 43)

apps/worker/src/services/jobLease.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `LeaseLostError`, `HeldLease`, `holdLease`.

Function/component	Parameters	Explanation and direct calls
<code>constructor</code> (line 6)	<code>uploadJobId</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>super</code> .
<code>holdLease</code> (line 18)	<code>uploadJobId</code> , <code>owner</code> , <code>renew</code> , <code>intervalMs</code>	Renews the lease every third of its duration while the job runs. A failed renewal caused by a database hiccup is retried on the next tick; only "another worker owns it" marks the lease lost. Direct named calls: <code>setInterval</code> , <code>timer.unref</code> .
<code>holdLease.assertHeld</code> (line 28)	None	No leading explanation comment; read the linked implementation.
<code>holdLease.stop</code> (line 29)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>clearInterval</code> .

apps/worker/src/services/notification.service.ts

Business operations and orchestration; see each function and its direct calls below.

Exported declarations: `notifyUploadJobResult`, `notifyUploadHighRejectionRate`, `notifyImageProcessingResult`, `notifyStaleListings`.

Function/component	Parameters	Explanation and direct calls
<code>notify</code> (line 8)	<code>userId</code> , <code>type</code> , <code>title</code> , <code>message</code> , <code>channels</code> , <code>details</code>	Records one notification. Emails are queued (sent later by the outbox job), and a failure to record a notification is logged, never thrown: a notification must not fail or appear to fail the import or photo processing that has already completed. Direct named calls: <code>createNotification</code> , <code>console.error</code> , <code>String</code> .

Function/component	Parameters	Explanation and direct calls
notifyMany (line 13)	userIds, type, title, message, channels	No leading explanation comment; read the linked implementation. Direct named calls: <code>Promise.all</code> , <code>userIds.map</code> .
notifyUploadJobResult (line 19)	dealerUserId, uploadJobId, status, counts, failureReason	Notifies the dealer once a CSV upload job reaches a terminal state, matching the delivery strategy: clean completion stays in-app only, failures/partial failures also send an email. Direct named calls: <code>notify</code> .
notifyUploadHighRejectionRate (line 30)	uploadJobId, rejectionRate	Advisory-only signal for administrators when a job's rejection rate crosses the threshold — not urgent enough to leave the app for, per the delivery strategy, so in-app only. Direct named calls: <code>findAdminUserIds</code> , <code>notifyMany</code> , <code>Math.round</code> .
notifyImageProcessingResult (line 36)	dealerUserId, status, unmatchedFolders, failureReason	Notifies the dealer once vehicle-photo zip processing reaches a terminal state. Direct named calls: <code>notify</code> .
notifyStaleListings (line 44)	dealerUserId, staleListings, days	Reminds a dealer about listings nobody has confirmed for a while, so buyers do not contact them about cars already sold. In-app only: it repeats weekly, which would be too often for email. Direct named calls: <code>notify</code> .

apps/worker/src/services/transientError.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

Declared test groups and cases:

- describe: `isTransientError` (line 4)
- it.each([['a dropped connection', Object.assign(new Error('socket hang up'), { code: 'ECONNRESET' })], ['S3 throttling', Object.assign(new Error('Please reduce your request rate.'), { name: 'SlowDown' })], ['an S3 5xx response', Object.assign(new Error('Internal'), { \$metadata: { httpStatusCode: 503 } })], ['an SDK-marked retryable error', Object.assign(new Error('x'), { \$retryable: { throttling: false } })], ['MongoDB being unreachable', Object.assign(new Error('Server selection timed out'), { name: 'MongoServerError' })], ['a wrapped network error', new Error('download failed', { cause: Object.assign(new Error('timeout'), { code: 'ETIMEDOUT' }) })],]): retries %s (line 5)
- it.each([['a missing file', Object.assign(new Error('The specified key does not exist.'), { name: 'NoSuchKey', \$metadata: { httpStatusCode: 404 } })], ['access denied', Object.assign(new Error('Access Denied'), { name: 'AccessDenied', \$metadata: { httpStatusCode: 403 } })], ['a CSV parse error', Object.assign(new Error('Invalid Record Length'), { code: 'CSV_RECORD_INCONSISTENT_COLUMNS' })], ['a duplicate key error', Object.assign(new Error('E11000 duplicate key error'), { name: 'MongoServerError', code: 11000 })], ['a programming error', new TypeError('Cannot read properties of undefined')], ['a non-error value', 'boom'],]): does not retry %s (line 16)

apps/worker/src/services/transientError.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `isTransientError`.

Function/component	Parameters	Explanation and direct calls
<code>isTransientError</code> (line 12)	<code>error</code>	Decides whether a failure is worth retrying: network blips, storage throttling or 5xx errors, and database connectivity loss. Anything else (bad CSV, invalid archive, missing file, programming errors) is permanent, so retrying would

Function/component	Parameters	Explanation and direct calls
		only repeat the same failure. Direct named calls: <code>TRANSIENT_ERROR_NAMES.has</code> , <code>TRANSIENT_NETWORK_CODES.has</code> , <code>candidate.hasErrorLabel</code> , <code>isTransientError</code> .

apps/worker/src/services/uploadJob.service.test.ts

Automated tests; the test names below state the behavior being checked. A test declaration is not evidence of a passing run.

Function/component	Parameters	Explanation and direct calls
car (line 24)	registration	No leading explanation comment; read the linked implementation.
insertedRowNumbers (line 26)	None	No leading explanation comment; read the linked implementation. Direct named calls: <code>mocks.insertListings.mock.calls.flatMap</code> .

Declared test groups and cases:

- describe: inventory upload ETL service (line 28)
- it: persists valid rows, records invalid rows, and completes with accurate counters (line 39)
- it: uses one lease-owner token for the claim and every later write (line 47)
- it: fails permanently (and tells the dealer) when the CSV itself cannot be parsed (line 56)
- it: hands a temporary storage failure back as pending and rethrows so BullMQ retries with backoff (line 64)
- it: fails a temporary failure once the durable attempt budget is used up (line 71)
- it: routes duplicate registration numbers to rejected records instead of listings (line 78)
- it: resumes after a crash from the last checkpoint without importing any row twice (line 85)
- it: rejects rows beyond the dealer's listing limit instead of importing them (line 100)
- it: stops without writing when another worker has taken over the lease (line 111)

apps/worker/src/services/uploadJob.service.ts

Business operations and orchestration; see each function and its direct calls below.

Exported declarations: `extractInventoryUpload`.

Function/component	Parameters	Explanation and direct calls
downloadInventoryStream (line 23)	storageKey	Downloads the private original CSV as a Node stream for bounded-memory parsing. Direct named calls: <code>workerStorageClient.send</code> .
processInventoryBatch (line 32)	uploadJobId, lease, dealerId, category, rows, processedRecords, counts, seenKeys	Transforms and persists one batch, then checkpoints the cumulative counters. Safe to run again for the same rows: rows an earlier attempt already imported are counted, not re-inserted, and rejected rows are insert-once. Direct named calls: <code>lease.assertHeld</code> , <code>prepareInventoryBatch</code> , <code>findImportedRowNumbers</code> , <code>prepared.valid.map</code> , <code>prepared.valid.filter</code> , <code>seenKeys.add</code> , <code>createListingDuplicateKey</code> , <code>detectExactDuplicates</code> , <code>Math.max</code> , <code>countOpenDealerListings</code> , <code>duplicateResult.unique.splice</code> , <code>overLimit.map</code> , <code>prepared.invalid.map</code> , <code>duplicateResult.duplicates.map</code> , <code>Promise.all</code> , <code>persistValidRows</code> , <code>persistRejectedRows</code> , <code>updateUploadProgress</code> .

Function/component	Parameters	Explanation and direct calls
extractInventoryUpload (line 60)	uploadJobId	Claims, downloads, and extracts one upload. A retry resumes after the last checkpoint. Temporary failures hand the job back as pending (BullMQ retries it with backoff); permanent ones, or running out of attempts, fail it and tell the dealer. Direct named calls: randomUUID, claimPendingUploadJob, console.warn, holdLease, trackLease, downloadInventoryStream, extractCsvBatches, lease.assertHeld, completeUploadJob, notifyUploadJobResult, notifyUploadHighRejectionRate, isTransientError, retryUploadJob, failUploadJob, lease.stop, untrackLease.

apps/worker/src/worker.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Function/component	Parameters	Explanation and direct calls
releaseActiveLeases (line 19)	None	Returns every job this process still holds to pending, using its own lease token so a job another worker has already taken over is never touched. Returns how many were released. Direct named calls: listActiveLeases, Promise.allSettled, leases.map, results.filter.
processInventoryJob (line 28)	job	Both job types share one queue; this dispatches by name to the right handler. Direct named calls: processInventoryUploadJob, processInventoryImagesJob.
startWorker (line 35)	None	Opens durable dependencies before the worker begins consuming queue messages. Direct named calls: connectWorkerDatabase, startWorkerHealthServer, setInterval, runRetention, runStaleReminders, worker.on, process.on, console.log.
startWorker.runRetention (line 47)	None	Deletes verification documents whose retention period has ended (runs once at startup too). Direct named calls: runDocumentRetention.
startWorker.runStaleReminders (line 52)	None	Reminds dealers about listings they have not confirmed for a while (runs once at startup too). Direct named calls: runStaleListingReminders.
startWorker.shutdown (line 73)	signal	Stops taking new jobs, gives running jobs until SHUTDOWN_TIMEOUT_MS to finish, then hands any still running back as pending (their progress checkpoints stay, so the next attempt resumes) and exits before the orchestrator's own kill deadline. Direct named calls: console.log, setTimeout, hardExit.unref, clearInterval, Promise.race, worker.close, clearTimeout, releaseActiveLeases, console.warn, Promise.allSettled, inventoryQueueProducer.close, results.push, disconnectWorkerDatabase, disconnectWorkerRedis, process.exit, results.some.

apps/worker/vitest.config.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

packages/shared-contracts/src/dtos/index.ts

Type contracts; these describe values for TypeScript and are not database writes or runtime checks by themselves.

Exported declarations: AuthUserDto, UpdateAuthProfileInput, DealerVerificationDocumentDto, DealerApplicationDto, CreateDealerApplicationInput, NotificationDto, ListingImageDto, ListingDto, CreateListingInput, UpdateListingInput, UpdateListingStatusInput, ReorderListingImagesInput, STALE_LISTING_DAYS, bulkListingActions, BulkListingAction, BULK_LISTING_MAX_IDS, BULK_PRICE_REDUCTION_MAX_PERCENT, BulkListingActionInput, BulkListingActionResult, DealerListingStatsDto.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

packages/shared-contracts/src/enums/index.ts

Module entry point or re-export barrel; inspect exported symbols/import targets.

Exported declarations: userRoles, UserRole, userStatuses, UserStatus, dealerApplicationStatuses, DealerApplicationStatus, notificationTypes, NotificationType, notificationChannels, NotificationChannel, notificationEmailStatuses, NotificationEmailStatus, listingStatuses, ListingStatus, fuelTypes, FuelType, engineRequiredFuelTypes, batteryRequiredFuelTypes, transmissionTypes, TransmissionType, vehicleCategories, VehicleCategory, vehicleConditions, VehicleCondition, carBodyTypes, CarBodyType, motorcycleTypes, MotorcycleType.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

packages/shared-contracts/src/index.ts

Module entry point or re-export barrel; inspect exported symbols/import targets.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

packages/shared-contracts/src/interfaces/index.ts

Type contracts; these describe values for TypeScript and are not database writes or runtime checks by themselves.

Exported declarations: PaginationMeta, ListResponseMeta, ApiResponseSuccess, ApiResponseError, ApiResponse.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

packages/shared-contracts/src/media/imageConstraints.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: LISTING_IMAGE_ASPECT_RATIO, LISTING_IMAGE_ASPECT_RATIO_LABEL, LISTING_IMAGE_MIN_WIDTH_PX, LISTING_IMAGE_MIN_HEIGHT_PX, LISTING_IMAGE_ASPECT_RATIO_TOLERANCE, LISTING_IMAGE_MAX_INPUT_PIXELS, LISTING_IMAGE_MAX_DIMENSION_PX, LISTING_IMAGE_OUTPUT_QUALITY, LISTING_IMAGE_OBJECT_PREFIX, LISTING_IMAGE_THUMB_MAX_DIMENSION_PX, LISTING_IMAGE_THUMB_QUALITY, LISTING_IMAGE_THUMB_SUBPATH.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

packages/shared-contracts/src/queue/inventoryJobs.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: INVENTORY_JOB_OPTIONS, inventoryBulkJobId.

Function/component	Parameters	Explanation and direct calls
csv (line 14)	uploadJobId	No leading explanation comment; read the linked implementation.
images (line 15)	uploadJobId	No leading explanation comment; read the linked implementation.

packages/shared-contracts/src/search/embedding.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: SEARCH_EMBEDDING_DIMENSIONS, createLocalSearchEmbedding, normalizeEmbeddingResponse, composeListingSearchText.

Function/component	Parameters	Explanation and direct calls
hashToken (line 3)	token	No leading explanation comment; read the linked implementation. Direct named calls: <code>character.charCodeAtAt</code> , <code>Math.imul</code> .
normalize (line 9)	values	No leading explanation comment; read the linked implementation. Direct named calls: <code>Math.sqrt</code> , <code>values.reduce</code> , <code>values.map</code> .
createLocalSearchEmbedding (line 15)	text	Provides an offline, model-compatible dimension fallback based on token features. Direct named calls: <code>text.toLocaleLowerCase</code> , <code>hashToken</code> , <code>normalize</code> .
normalizeEmbeddingResponse (line 26)	payload	HF feature extraction can return one vector or one vector per token; both become one normalized vector. Direct named calls: <code>Array.isArray</code> , <code>payload.every</code> , <code>normalize</code> , <code>rows.some</code> , <code>Array.from</code> .
composeListingSearchText (line 38)	listing	No leading explanation comment; read the linked implementation. Direct named calls: <code>Object.values</code> .

packages/shared-contracts/src/utils/mongoUri.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: findProductionMongoUriProblems.

Function/component	Parameters	Explanation and direct calls
findProductionMongoUriProblems (line 8)	uri	Lists the reasons a MongoDB URI is unsafe for production, so a production deployment cannot silently run against a local, dev, or test database or over an unencrypted connection. Returns an empty array when the URI is acceptable. Direct named calls: <code>MONGO_URI_PATTERN.exec</code> , <code>uri.trim</code> , <code>params.get</code> , <code>scheme.toLowerCase</code> , <code>problems.push</code> , <code>hostList.split</code> , <code>decodeURIComponent</code> , <code>NON_PRODUCTION_DB_PATTERN.test</code> .

packages/shared-contracts/src/utils/registrationNumber.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: normalizeRegistrationNumber.

Function/component	Parameters	Explanation and direct calls
<code>normalizeRegistrationNumber</code> (line 3)	<code>value</code>	Normalizes a vehicle registration/license plate number into a comparable identity key. "CAX-1234", "CAX 1234", and "cax1234" all normalize to "CAX1234". Direct named calls: <code>value.trim</code> .

packages/shared-contracts/src/vehicle/attributeSchemas.ts

Validation rules and related types; these reject or normalize unsupported input.

Exported declarations: `carAttributesSchema`, `motorcycleAttributesSchema`, `vanAttributesSchema`, `truckAttributesSchema`, `threeWheelerAttributesSchema`, `busAttributesSchema`, `otherVehicleAttributesSchema`, `vehicleDetailsSchema`, `CarAttributesInput`, `MotorcycleAttributesInput`, `VanAttributesInput`, `TruckAttributesInput`, `ThreeWheelerAttributesInput`, `BusAttributesInput`, `VehicleDetailsInput`.

Function/component	Parameters	Explanation and direct calls
<code>refinePowertrain</code> (line 25)	<code>data</code> , <code>ctx</code>	Requires <code>engineCapacityCc</code> for combustion-bearing fuel types and both <code>battery</code> fields for battery-bearing fuel types (conventional hybrid is combustion-required but battery-optional). Direct named calls: <code>ctx.addIssue</code> .

packages/shared-contracts/src/vehicle/csvTemplates.ts

Supporting application module; use the symbols, comments, and direct calls below to follow its role.

Exported declarations: `CsvFieldDescriptor`, `CategoryCsvTemplate`, `carCsvTemplate`, `motorcycleCsvTemplate`, `vanCsvTemplate`, `truckCsvTemplate`, `threeWheelerCsvTemplate`, `busCsvTemplate`, `csvTemplatesByCategory`, `buildCsvTemplateContent`.

Function/component	Parameters	Explanation and direct calls
<code>buildCsvTemplateContent</code> (line 98)	<code>template</code>	Renders a template's field list + example rows into an actual downloadable CSV file body. Direct named calls: <code>template.fields.map</code> , <code>headers.join</code> , <code>template.exampleRows.map</code> , <code>lines.join</code> .
<code>buildCsvTemplateContent.escape</code> (line 100)	<code>value</code>	No leading explanation comment; read the linked implementation. Direct named calls: <code>value.replace</code> .

packages/shared-contracts/src/vehicle/index.ts

Module entry point or re-export barrel; inspect exported symbols/import targets.

Exported declarations: `PowertrainAttributes`, `CarAttributes`, `MotorcycleAttributes`, `VanAttributes`, `TruckAttributes`, `ThreeWheelerAttributes`, `BusAttributes`, `OtherVehicleAttributes`, `VehicleDetails`, `VehicleAttributesByCategory`, `AnyVehicleAttributes`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.

packages/shared-contracts/src/vehicle/listingSchemas.ts

Validation rules and related types; these reject or normalize unsupported input.

Exported declarations: `commonListingFieldsSchema`, `listingRowSchema`, `ListingRowInput`.

No named function declarations: this file contains configuration, types, data, re-exports, top-level setup, or anonymous callbacks.